



DI Lena Klambauer¹

**Automata Learning of Websites
Using AALpy and Selenium**

MASTER'S THESIS

to achieve the university degree of

Diplom-Ingenieur

Master's degree programme: Computer Science

submitted to

Graz University of Technology

Supervisor

Ao.Univ.-Prof. Dipl.-Ing. Dr.techn. Bernhard K. Aichernig

Institute of Software Engineering and Artificial Intelligence

Graz, 23 October 2025

¹ E-mail: lena.klambauer@gmx.at
© Copyright 2025 by the author

Abstract

This master's thesis explores the use of automata learning techniques to create behavioural models of websites. To this end, a learning application was implemented. AALpy, an automata learning library written in Python, and Selenium, a framework to automate website interactions, were used as the main tools.

The user must specify the initial URL as well as the URL scope of the learning process. The learning application analyses all webpages in the scope to find interactive HTML elements, which are then used to generate the input alphabet.

The learning application can be configured to learn the website system with one of two different methods: active or passive learning.

Active learning uses AALpy's implementation of the L^* algorithm. It actively queries the website with inputs, tracks the received outputs, and constructs a hypothesis automaton model of the system. The hypothesis is compared to the system under learning using an equivalence oracle. If no counter-example is found, the hypothesis is accepted and the learning terminates. Otherwise, it integrates the new information and looks for a new hypothesis. AALpy supports the use of different equivalence oracles, each of which can be configured using specific parameters.

For passive learning, the learning application first creates input-output traces from queries to the website, which are then used as the basis for AALpy's state-merging in the red-blue framework. Since interactions with the website are slow, the trace generation process attempts to minimize the number of interactions. As a result, some information may be lost, which must be compensated for with domain knowledge to obtain correct results. Two different approaches for this compensation were implemented in the application.

Both the active learning process as well as the trace generation for passive learning assume that the system under learning is input enabled. A website is not so by default, as HTML elements can only be interacted with if the webpage they are on is currently loaded. To address this, the application supports two strategies for handling invalid inputs: either creating a self-loop in the learned automaton, or transitioning to an artificially inserted sink state.

The learning application was evaluated on two custom-built case study websites, which include different types of HTML elements and structures, as well as hidden information that affects behaviour stored in cookies. The results show that, for all configurations with sufficiently high parameter settings, the learning application was able to learn correct automaton models of the case study websites. Passive learning proved more efficient for learning less complex websites, where all observable behaviour could be found close to the initial state. In contrast, active learning was more efficient for more complex websites, where behaviour was deeply hidden within the system.

Out of the two methods to make the system input enabled, using self-loops proved far more suitable than inserting a sink state.

Keywords: automata learning, website testing, AALpy, Selenium, active learning, L^* algorithm, passive learning, state-merging, red-blue framework

Kurzfassung

Diese Masterarbeit untersucht den Einsatz von Verfahren des Automatenlernens zur Erstellung von Verhaltensmodellen von Websites. Zu diesem Zweck wurde eine Lernanwendung implementiert. Als Hauptwerkzeuge kamen AALpy, eine in Python geschriebene Automatenlern-Bibliothek, sowie Selenium, ein Framework zur Automatisierung von Website-Interaktionen, zum Einsatz.

Der Benutzer muss den initialen URL definieren, sowie den Lernprozess über eine Liste an URLs eingrenzen. Die Lernanwendung analysiert alle Webseiten innerhalb des angegebenen Bereichs, um interaktive HTML-Elemente zu finden, die anschließend zur Erzeugung des Eingabealphabets verwendet werden.

Die Lernanwendung kann das Website-System mit einer von zwei unterschiedlichen Methoden erlernen: aktivem oder passivem Lernen.

Beim aktiven Lernen wird der in AALpy implementierte L^* -Algorithmus eingesetzt. Dabei stellt die Anwendung aktiv Eingabeabfragen an die Website, verfolgt die erhaltenen Ausgaben und konstruiert daraus ein Hypothesenmodell des Systems. Diese Hypothese wird mit dem zu lernenden System mittels eines Äquivalenzorakels verglichen. Wird kein Gegenbeispiel gefunden, wird die Hypothese akzeptiert und der Lernprozess beendet. Andernfalls wird die neue Information integriert und eine neue Hypothese aufgestellt. AALpy unterstützt die Verwendung verschiedener Äquivalenzorakel, die jeweils durch spezifische Parameter konfiguriert werden können.

Für das passive Lernen erstellt die Lernanwendung zuerst Eingabe-Ausgabe-Sequenzen aus Anfragen an die Website, die anschließend als Grundlage für die Zustandsverschmelzung im Rot-Blau-Framework von AALpy dienen. Da die Interaktionen mit Websites langsam sind, versucht der Sequenz-Generierungsprozess die Anzahl der Interaktionen zu minimieren. Dadurch können jedoch Informationen verloren gehen, die durch Domänenwissen kompensiert werden müssen, um korrekte Ergebnisse zu erzielen. In der Anwendung wurden zwei unterschiedliche Ansätze für diese Kompensation implementiert.

Sowohl der aktive Lernprozess als auch die Sequenz-Generierung für passives Lernen setzen voraus, dass das zu lernende System immer alle Eingaben akzeptiert. Eine Website kann dies jedoch standardmäßig nicht, da mit HTML-Elementen nur interagiert werden kann, wenn die Webseite auf der sie sich befinden geladen ist. Um dieses Problem zu lösen, unterstützt die Anwendung zwei Strategien zum Umgang mit ungültigen Eingaben: Entweder wird im gelernten Automaten eine Selbstschleife erzeugt, oder es erfolgt ein Übergang in einen künstlich eingefügten Senkenzustand.

Die Lernanwendung wurde anhand von zwei eigens erstellten Fallstudien-Websites evaluiert, die unterschiedliche Arten von HTML-Elementen und Strukturen, sowie versteckte, in Cookies gespeicherte Informationen, die das Verhalten beeinflussen, enthalten. Die Ergebnisse zeigen, dass die Lernanwendung bei allen Konfigurationen mit hinreichend hohen Parametereinstellungen korrekte Automatenmodelle der Fallstudien-Websites lernen konnte. Passives Lernen erwies sich als effizienter beim Erlernen weniger komplexer Websites, deren gesamtes beobachtbares Verhalten sich in der Nähe des Initialzustands befand. Im Gegensatz dazu war aktives Lernen effizienter bei komplexeren Websites, deren Verhalten tief im System verborgen war.

Von den beiden Methoden, das System eingabefähig zu machen, erwies sich die Verwendung von Selbstschleifen als besser geeignet als das Einfügen eines Senkenzustands.

Schlagworte: Automatenlernen, Website-Testen, AALpy, Selenium, aktives Lernen, L^* -Algorithmus, passives Lernen, Zustandsverschmelzung, Rot-Blau-Framework

Acknowledgements

The completion of this master's thesis was made possible with the support of many people.

First and foremost, I would like to thank my supervisor, Ao.Univ.-Prof. Dipl.-Ing. Dr.techn. Bernhard K. Aichernig, whose insightful courses sparked my interest in formal methods in general, and specifically in automata learning techniques.

I would also like to thank DI Benjamin von Berg, who kindly made himself available at short notice to answer questions about passive learning, and whose advice helped me quickly overcome an obstacle that might otherwise have kept me occupied for much longer.

My gratitude also goes to the system administrators of the Institute for Software Engineering and Artificial Intelligence for providing the necessary hardware for various computations. Special thanks go to Mario Wellik, who managed the setup and provided ongoing support.

I am grateful to my friends for always being willing to listen, and for their advice and support, both personal and technical.

Finally, I would like to wholeheartedly thank my family, who have supported me at every level throughout my entire education. Without them, completing my studies would not have been possible.

Lena Klambauer
Graz, Austria, 23 October 2025

Danksagung

Die Erstellung der vorliegenden Masterarbeit wurde durch die Unterstützung zahlreicher Menschen möglich gemacht.

Allen voran danke ich meinem Betreuer Ao.Univ.-Prof. Dipl.-Ing. Dr.techn. Bernhard K. Aichernig, der durch seine aufschlussreichen Lehrveranstaltungen mein Interesse an formalen Methoden generell, und im Speziellen den Verfahren des Automatenlernens, geweckt hat.

Weiters geht mein Dank an DI Benjamin von Berg, der auch kurzfristig für Fragen zum passiven Lernen zur Verfügung stand, und mir mit seinen Ratschlägen half, eine Hürde schnell zu überwinden, die mich sonst vermutlich noch länger beschäftigt hätte.

Ebenso möchte ich mich bei den System Administratoren des Instituts für Software Engineering und Artificial Intelligence für die Bereitstellung der benötigten Hardware für diverse Berechnungen bedanken. Spezieller Dank geht hierbei an Mario Wellik, der das Setup und die Betreuung übernahm.

Danke an meine Freunde für ihr offenes Ohr, sowie Beratung und Unterstützung sowohl persönlicher als auch technischer Natur.

Abschließend möchte ich von Herzen meiner Familie danken, die mich auf allen Ebenen durch meine gesamte Ausbildung begleitet hat. Ohne sie wäre dieses Studium für mich nicht möglich gewesen.

Lena Klambauer
Graz, Österreich, 23. Oktober 2025

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Research Problem and Goals	1
1.3	Structure.....	2
2	Preliminaries.....	3
2.1	Definitions	3
2.2	Web Testing	3
2.3	System Modelling and Representation.....	4
2.4	Automata Learning	5
2.4.1	Active Automata Learning	5
2.4.2	Passive Automata Learning.....	9
2.5	AALpy.....	11
2.6	Selenium.....	12
3	Methodology.....	13
3.1	Running Example	13
3.2	Accessing the Website SUL.....	14
3.3	Automata Characteristics	14
3.3.1	Input Alphabet – Scope.....	14
3.3.2	Initial State – Reset	15
3.3.3	Input Enabled Handling	15
3.3.4	Output – Current URL	15
3.4	Active Learning	16
3.4.1	Active Learning Algorithm	16
3.4.2	Equivalence Oracle	16
3.5	Passive Learning.....	17
3.5.1	Trace Generation.....	17
3.5.2	Passive Learning Algorithm.....	20
4	Implementation.....	25
4.1	Tool Versions	26
4.2	Configuration File.....	26
4.3	WebDriver Configuration	29
4.4	Preprocessing of User-Defined URLs.....	30
4.5	Generating the Input Alphabet	30

4.5.1	Searching for Interactive HTML Elements.....	30
4.5.2	Creation of <i>InputElement</i> Objects	32
4.5.3	Creation of Unique Input Letter Strings	34
4.6	Reset to Initial State	34
4.7	HTML Elements and Interactions.....	35
4.7.1	Getting a Selenium Reference to the Element	35
4.7.2	Interacting with the Element.....	36
5	Challenges	38
5.1	Interaction Speed.....	38
5.2	Simulating User Interactions	39
5.3	Choice of Initial URL.....	39
5.4	Output of Interactions in Edge States	40
5.5	Balancing Information Depth and Human Interpretability	42
5.6	Mouse Reset on <i>HoverEnd</i> Interactions	42
5.7	Mouse Pointer Tracking	44
5.8	Mouse Pointer Visualisation.....	44
5.9	Possible Issues During the Interaction Sequence.....	44
5.9.1	Non-Determinism Caused by Variable Internet Speed	44
5.9.2	Interaction with Covered Elements	45
5.9.3	Disabled and Enabled Elements Wrapped in Enabled Elements	45
5.9.4	Automatic Scrolling Triggers Hover	45
5.9.5	<i>HoverEnd</i> without <i>HoverStart</i>	45
5.9.6	Elements Change Position.....	45
5.9.7	Bounding Box Approximation	47
5.9.8	User Interference	47
6	Case Study Websites.....	48
6.1	Car Alarm System Demo Website	48
6.2	Test Website	50
7	First Prototype	52
7.1	Configuration	52
7.2	Evaluation of the Prototype	53
7.2.1	Learned Automata	54
7.2.2	Impact of Equivalence Oracle Choice on Performance	54
8	Evaluation	57
8.1	Active Learning Equivalence Oracle.....	57

8.2	Seeds for Active Learning.....	57
8.3	Test Website.....	57
8.3.1	Single Page Test Website SULs	57
8.3.2	Whole Test Website SUL	61
8.4	Car Alarm System Demo Website.....	63
8.4.1	Active Learning – Depth First and Input Enabled Handling.....	64
8.4.2	Passive Learning – Passive Approach and Input Enabled Handling	69
8.4.3	Active versus Passive Learning	70
9	Concluding Remarks.....	71
9.1	Summary.....	71
9.2	Conclusion.....	71
9.3	Future Work.....	72
	Appendices	74
	Bibliography	81

List of Figures

Figure 1: Example of a Mealy machine represented as a directed labelled graph	5
Figure 2: Basic procedure of the L* algorithm.....	6
Figure 3: State Merging – Merge incompatibility of two states due to resulting non-determinism	10
Figure 4: State Merging – Implied merge due to duplicate transition.....	10
Figure 5: AALpy’s interface and structure [8].....	11
Figure 6: Overview of the website learning process.....	13
Figure 7: Running Example – Schematic of the website structure	13
Figure 8: Running Example – Definition of initial page and scope, and resulting input alphabet to be used in active learning	17
Figure 9: Running Example – Final learned model using active learning (NOCP = <i>NotOnCurrentPage</i> , DE = <i>DeadEnd</i> , GS = <i>GoSink</i> , SS = <i>StaySink</i>).....	17
Figure 10: Running Example – Definition of initial page and scope, and resulting input alphabet to be used in trace generation	19
Figure 11: Running Example – Trace generation initialisation.....	19
Figure 12: Running Example – Trace generation after iteration 1	19
Figure 13: Running Example – Trace generation after iteration 2	20
Figure 14: Running Example – Trace generation after iteration 3	20
Figure 15: Running Example – PTA before and after preprocessing (NOCP = <i>NotOnCurrentPage</i> , DE = <i>DeadEnd</i> , GS = <i>GoSink</i> , SS = <i>StaySink</i>).....	22
Figure 16: Running Example – Self-loops created via the custom scoring function (NOCP = <i>NotOnCurrentPage</i> , DE = <i>DeadEnd</i> , GS = <i>GoSink</i> , SS = <i>StaySink</i>).....	22
Figure 17: Running Example – Final learned model using passive learning with PTA preprocessing and the custom scoring function (NOCP = <i>NotOnCurrentPage</i> , DE = <i>DeadEnd</i> , GS = <i>GoSink</i> , SS = <i>StaySink</i>).....	23
Figure 18: Running Example – Intermediate learned model while passive learning with the custom compatibility function (NOCP = <i>NotOnCurrentPage</i> , DE = <i>DeadEnd</i> , GS = <i>GoSink</i> , SS = <i>StaySink</i>)	24
Figure 19: Running Example – Final learned model using passive learning with the custom compatibility function and postprocessing (NOCP = <i>NotOnCurrentPage</i> , DE = <i>DeadEnd</i> , GS = <i>GoSink</i> , SS = <i>StaySink</i>)	24
Figure 20: Sequence and data movement of the execution of the learning application	25
Figure 21: Example of an alert overlay	29
Figure 22: Example of a Select element.....	34
Figure 23: Different partial automata resulting from a different choice of initial URL	39
Figure 24: Output Example – Schematic of website SUL	40

Figure 25: Output Example – Self-loop automaton <i>V1_loop</i> resulting if pages that are at the boundary of the scope and pages without interactive elements are not handled separately.....	40
Figure 26: Output Example – Sink state automaton <i>V1_sink</i> resulting if pages that are at the boundary of the scope and pages without interactive elements are not handled separately.....	41
Figure 27: Output Example – Automaton <i>V2</i> resulting if pages that are at the boundary of the scope and pages without interactive elements are handled separately, but their output does not include the current URL.....	41
Figure 28: Output Example – Automaton <i>V3</i> resulting if pages that are at the boundary of the scope and pages without interactive elements are handled separately and their output includes the current URL.....	41
Figure 29: Hover Example – Schematic of website SUL.....	42
Figure 30: Hover Example – Self-loop automaton <i>V1_hover_basic</i> without hover detection	43
Figure 31: Hover Example – Self-loop automaton <i>V2_hover_advanced</i> including hover detection	43
Figure 32: Moving Elements Example – Schematic of website SUL	46
Figure 33: Moving Elements Example – Three different states of Page 1	46
Figure 34: Moving Elements Example – Sequence of transitioning from hovering above H1 to hovering above H2.....	46
Figure 35: Moving Elements Example – Learned self-loop automaton.....	47
Figure 36: State machine of the Car Alarm System demo website	48
Figure 37: Frontend of the Car Alarm System demo website (4-door version).....	49
Figure 38: Frontend of the Car Alarm System demo website (reduced version).....	50
Figure 39: Prototype – Learned automaton for the 1-door Car Alarm System demo website	54
Figure 40: Test Website Single Page SULs – Efficiency of active versus passive learning	59
Figure 41: Whole Test Website – Efficiency of active versus passive learning.....	62
Figure 42: Car Alarm System Demo Website – Efficiency of active learning with chosen parameters (df = depth first, wps = walks per state, wl = walk length)	68
Figure 43: Car Alarm System Demo Website – Efficiency of active versus passive learning.....	70

List of Tables

Table 1: Structure of an L^* observation table	7
Table 2: Running Example – Resulting input alphabets for different scope definitions	14
Table 3: Allowed string values for <i>website_to_learn</i> in the configuration file.....	26
Table 4: Specifying <i>website_to_learn</i> manually in the configuration file.....	26
Table 5: Configuration options – Description.....	27
Table 6: Configuration options – Allowed and default values	28
Table 7: Attributes the learning application looks for to find interactive elements	32
Table 8: <i>InputElement</i> objects created from found HTML elements	33
Table 9: Interaction Speed Test – Device specifications and network speed	38
Table 10: Additional configuration options for the system source.....	38
Table 11: Prototype – Device specifications and network speed.....	53
Table 12: Prototype – URL settings.....	53
Table 13: Prototype – Car Alarm System demo website analysis	53
Table 14: Prototype – 4-door Car Alarm System demo website, performance of State Prefix equivalence oracle.....	55
Table 15: Prototype – 2-door Car Alarm System demo website, performance of State Prefix equivalence oracle.....	55
Table 16: Prototype – 1-door Car Alarm System demo website, performance of State Prefix equivalence oracle.....	55
Table 17: Prototype – 2-door Car Alarm System demo website, performance of Random Walk equivalence oracle.....	56
Table 18: Test Website – Size of scope and input alphabet.....	57
Table 19: Test Website Single Page SULs – Structure of correct learned automata.....	58
Table 20: Test Website Single Page SULs – Minimum parameter values needed to learn correct automata.....	58
Table 21: Test Website Single Page SULs – Number of SUL interactions and resets using minimal sufficient parameter values.....	59
Table 22: Test Website Single Page SULs – Passive learning statistics using minimal sufficient trace length for SULs with low necessary trace lengths.....	60
Table 23: Whole Test Website – Structure of correct learned automata	61
Table 24: Whole Test Website – Minimum parameter values needed to learn the correct automata	62
Table 25: Whole Test Website – Number of SUL interactions and resets using minimal sufficient parameter values.....	62
Table 26: Whole Test Website – Passive learning statistics using minimal sufficient trace length	63
Table 27: Car Alarm System Demo Website – Initial URL, scope, and input alphabet	64

Table 28: Car Alarm System Demo Website – Structure of correct learned automata.....	64
Table 29: Car Alarm System Demo Website – Overview of minimal necessary parameter values for active learning.....	65
Table 30: Car Alarm System Demo Website – Detailed evaluation of minimal necessary parameter values for active learning with self-loop input enabled handling.....	66
Table 31: Car Alarm System Demo Website – Detailed evaluation of minimal necessary parameter values for active learning with sink state input enabled handling	67
Table 32: Car Alarm System Demo Website – Active learning statistics with chosen parameters .	68
Table 33: Car Alarm System Demo Website – Passive learning statistics using minimal sufficient trace length	69
Table 34: Car Alarm System Demo Website – Number of SUL interactions and resets for most efficient passive and active configuration using minimal consistently sufficient parameter values	70

List of Algorithms

Algorithm 1: L* algorithm [15]	6
Algorithm 2: State merging in the red-blue framework [19]	11
Algorithm 3: Trace generation for passive learning	18
Algorithm 4: PTA preprocessing	21

List of Acronyms

API	Application Programming Interface
CSS	Cascading Style Sheets
DFA	Deterministic Finite-state Automaton
DOM	Document Object Model
HTML	Hypertext Markup Language
IEH	Input Enabled Handling
IO	Input-Output
PTA	Prefix Tree Automaton
PDF	Portable Document Format
SUL	System Under Learning
URL	Uniform Resource Locator

1 Introduction

1.1 Motivation

Using web applications has become commonplace in our current day and age. The number of internet users worldwide has risen to 5.5 billion in 2024. The average daily time spent online by these users is about 6.5 hours [1]. The global web market revenue is predicted to reach 435.6 billion USD for 2025 [2]. All this is to show that websites are widely known and used, and that their design, performance, and security play a crucial role in shaping digital experiences. As more aspects of daily life – from communication and entertainment to shopping and education – continue to shift online, the demand for reliable, efficient, and user-friendly web applications will only increase.

The web frontend is the first point of interaction with a web application a user experiences. As the interface between the user and the underlying functionality of the web application, it significantly impacts user perception and satisfaction, and as a consequence user retention [3]. A user is not likely to return to a website that has broken links, where dropdown menus cannot be opened, or where moving the mouse pointer across an image suddenly loads a new page.

This means that web frontend testing is vital to ensure that web applications provide a seamless and reliable user experience. It helps to identify and resolve potential issues, enhances user satisfaction, and upholds the application’s reputation by preventing bugs and inconsistencies. Sufficient and successful web testing can be very challenging and resource intensive due to the dynamic and diverse nature of web applications, the complexity of interactive elements, asynchronous behaviours and varying responsiveness [3].

Extensive specification and documentation writing is the first to go when developers are forced to reduce the time spent on a project. With undocumented legacy code, the increasing frequency of updates driven by market demands, and rapid and incremental development, manually writing a test suite with sufficient coverage and keeping it up-to-date becomes a high effort endeavour. Often there are no models which could be used to automatically generate a test suite, sometimes the code is not available, or it is a collection of scripts, programs, and services, written in different languages, and relying on functionalities provided by the hosting platform [4].

Automata learning, which uses observations to construct a behavioural model of a system, has proven itself as a viable way to overcome these or similar obstacles in other domains [5] [6] [7]. The resulting learned model can be analysed for faults and makes further testing techniques possible. This thesis therefore applies different common automata learning approaches to website systems and analyses the learned models. It further evaluates the suitability of these different approaches for the specific domain of website systems.

1.2 Research Problem and Goals

The goal of this thesis is to use automata learning to learn a model of a website. To this end, a learning application that includes an adapter to access a website was implemented. The application uses AALpy [8] as a learning library and Selenium [9] to interact with the HTML elements of the website.

Automata learning is a process in which a behavioural model of a system is learned from the system’s data. This can either be done from preexisting data like log files (passive automata learning), or by actively querying the system with inputs and receiving outputs (active automata learning) [10].

Both passive and active learning as they are implemented in the learning application assume that the system under learning is input enabled. This means that in any state of the website, all inputs that are part of the input alphabet will be accepted and handled. By default, a website is not input enabled,

as elements can only be interacted with if the specific webpage they are on is currently loaded. Making the system input enabled can either be achieved by creating self-loops in the learned automaton whenever an invalid input is received, or by transitioning to an artificial sink state.

Different learning configurations using active or passive learning algorithms were used on two case study websites, and the results evaluated to answer the following three research questions:

- 1. Can an automaton that correctly represents the whole behaviour of a website be learned using AALpy and Selenium?**
- 2. Is passive or active learning more suitable for learning an automaton of a website system?**
- 3. Is it preferable to use self-loops or an artificial sink state to make the website system input enabled?**

1.3 Structure

This thesis consists of nine chapters.

In Chapter 2 the preliminary knowledge for this thesis is presented. This includes definitions of common terms that are used, some basics about website testing, descriptions of the modelling formalisms used to represent learned website behaviour, the basic algorithm of the active as well as the passive learning approach, and a short introduction to the two main tools used in the implementation, AALpy and Selenium.

Chapter 3 reports on the methodology behind different aspects of the website automata learning, as well as any design decisions and assumptions that were made in that context. This includes the definition of the initial state and scope of the learning process, and how this relates to the input alphabet. Additionally, it defines two different ways a website can be made input enabled, and which outputs are used in the learning process. Finally, details of how the active and the passive learning algorithm are applied are described.

Chapter 4 presents some details of the implementation of the learning application. It contains information about which tool versions were used, how the learning application can be executed with different configurations, how the input alphabet is generated, how the system is reset to its initial state, and how interactions with HTML elements are simulated.

Chapter 5 gives an overview of domain specific challenges that were encountered during the course of this thesis, and how they were resolved. It also includes a list of possible issues that might occur when learning an unknown website.

In Chapter 6, two case study websites are described that were created specifically for this thesis project.

Chapter 7 describes a first prototype of the learning application that was created as proof-of-concept for this thesis. It lists the limited functionality of the prototype when compared to the final learning application, and evaluates results of trying to learn an early version of one of the case study websites with active learning.

Chapter 8 contains the evaluation of the final version of the learning application on the two case study websites. This includes comparisons of different configurations for active and passive learning, as well as comparisons between active and passive learning.

Finally, Chapter 9 concludes this thesis with a short summary, the conclusions that were drawn from the project, and possible extensions to future work.

The code for the learning application can be found under <https://github.com/lklamb/aalpy-selenium-website-learning>.

2 Preliminaries

This chapter provides background knowledge for the subjects discussed in the other parts of this thesis.

2.1 Definitions

In this thesis the following definitions are used:

Webpage

A webpage (or simply page) is a single document that can be viewed in a web browser.

Website

A website is a collection of related webpages that are grouped together under a single domain name.

Connected Webpages

A webpage is connected to another webpage if it can be reached via interactions from the first webpage. A collection of connected webpages could be a single website, but could also span multiple domains. Even though this thesis is about learning a website, the principles are also applicable to any other system of connected webpages.

Uniform Resource Locator (URL)

A Uniform Resource Locator is used to locate and access a specific resource, e.g., a webpage, on the internet.

Initial URL

This URL is used as the starting point for the learning application to learn the website in active learning and for trace generation during passive learning.

URL Scope

This is a collection of URLs to specific webpages that is used to generate the input alphabet for active learning and for trace generation during passive learning.

Boundary of Scope

The boundary of the URL scope includes all webpages / URLs that can be reached via a sequence of interactions from the initial webpage / URL, but are not part of the URL scope themselves.

Current URL

This is the URL that the system is currently at, which matches the webpage currently displayed in the web browser.

HTML Element

This is an HTML element on a webpage, e.g., a button, an image, an anchor, or a paragraph.

System Under Learning (SUL)

The System Under Learning is the website or system of connected webpages that is being learned.

2.2 Web Testing

Web testing is defined as the testing of a web application. As any other testing process, its goal is to identify any issues and points of failure and their underlying reasons, and to help resolve them. Successful and sufficient web testing ensures that the application operates effectively [11].

As the definition of “effectiveness” is broad, many different types of web testing exist, depending on which type of effectiveness should be tested. They can be structured into two main categories: functional testing, and non-functional testing.

Functional testing includes anything that is to do with the actual functional requirements of the web application. This includes but is not limited to all links working properly, forms working correctly, data being stored as intended without any inconsistencies etc [12] [13].

Non-functional testing is testing of any non-functional requirements. This includes:

- performance testing (e.g., response time, service availability)
- load testing (performance testing under a specific load level)
- stress testing (evaluation beyond the specified requirements, e.g., recovery from errors or crashes)
- compatibility testing (trying to uncover failures due to different web server platforms or browsers)
- usability testing (ease of use for the user, e.g., clarity of messages and prompts)
- accessibility testing (usability testing on systems with limited hardware or software, usability testing for users with disabilities)
- security testing (e.g., effectiveness of defences against unauthorized access, capability to preserve system resources from improper use) [11] [12] [13]

A web application may be tested with unit tests (e.g., for a single webpage), integration tests (e.g., a subset of webpages of a website), or system tests (tests for the whole website system) [13].

Testing strategies include white box, black box, and grey box testing. In white box testing strategies, test cases are designed on the basis of the code representation of the web application. Black box testing strategies are usually used to identify failures in the externally visible behaviour of the system, and are responsibility based, meaning the test defines what should happen, not how it should happen. Grey box testing strategies combine black box and white box testing approaches: they use some knowledge of the internal workings of the web application for testing it against its specifications [13].

From the perspective of providing a reliable and seamless experience to the user, among others, web frontend testing is essential. Bugs or inconsistencies have a big impact on user satisfaction, and might cause users to abandon a web application. The dynamic and diverse nature of web applications can be very challenging for web frontend testing. The complexity of interactive elements, asynchronous behaviour, and responsiveness make sufficient and successful testing a high effort endeavour [3].

2.3 System Modelling and Representation

A model of a system is a formal representation of its behaviour. In this thesis, the real-life system is a website. This is a *reactive system*, meaning it reacts to inputs from the environment based on some sort of control mechanism and never terminates, as opposed to a *transformational system*, which transforms an input into an output and then terminates.

Mealy machines are well suited to modelling reactive systems. A Mealy machine is a finite state machine where the output depends on the current state and the current input. Formally, a Mealy machine is a 6-tuple $\langle S, s_0, \Sigma, \Omega, \delta, \lambda \rangle$, where

- S is a finite set of states,
- $s_0 \in S$ is the initial state,
- Σ is the input alphabet, a finite set of input symbols,
- Ω is the output alphabet, a finite set of output symbols,
- $\delta : S \times \Sigma \rightarrow S$ is the transition function, specifying the transitions from every state to its successor states, and
- $\lambda : S \times \Sigma \rightarrow \Omega$ is the output function, producing an output symbol for every transition.

When processing an input symbol $i \in \Sigma$, a Mealy machine will proceed into a successor state as denoted by the δ function and produce an output symbol $o \in \mathcal{Q}$ while doing the transition, as denoted by λ . This can be extended to the processing of sequences of input symbols, which returns a sequence of output symbols [10] [14].

In a Mealy machine, two states $s_u, s_v \in S$ are equivalent if both states show exactly the same behaviour for all futures, i.e., every sequence of input symbols produces the same sequence of output symbols for both. States that are equivalent can be merged to arrive at a minimal Mealy machine [14].

A Mealy machine is assumed to be input enabled, meaning that for each combination of state $s \in S$ and input symbol $i \in \Sigma$, both the transition function $\delta(s, i)$ and the output function $\lambda(s, i)$ must be defined. Additionally, as δ is defined as a non-set function, a Mealy machine is by definition deterministic [10].

A Mealy machine can be represented as a directed labelled graph. Each node corresponds to a state, whereas each directed edge corresponds to a transition. Each edge is labelled with an input and the output this input produces.

Figure 1 shows an example of a Mealy machine as a graph. The states of the system are s_0 and s_1 . Each transition is labelled in the format *input/output*. The arrow not originating from a node denotes the initial state.

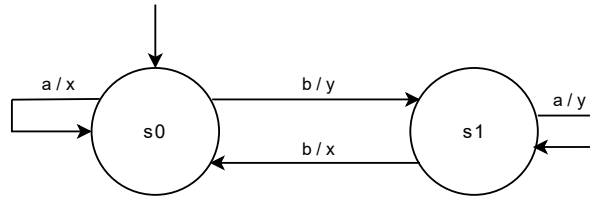


Figure 1: Example of a Mealy machine represented as a directed labelled graph

2.4 Automata Learning

Automata learning is the process of inferring a behavioural model of a system (SUL) from the system's data. It is generally assumed that the internals of the system behaviour cannot be accessed directly, so that the behaviour must be learned from input and output data ("black box system"). There are two different general approaches to automata learning, *active* and *passive*, which are distinguished by how this data is generated [10].

2.4.1 Active Automata Learning

Active automata learning actively queries the system during the learning process. For this, an adapter for the learning application to interact with the SUL is needed.

2.4.1.1 L* Algorithm

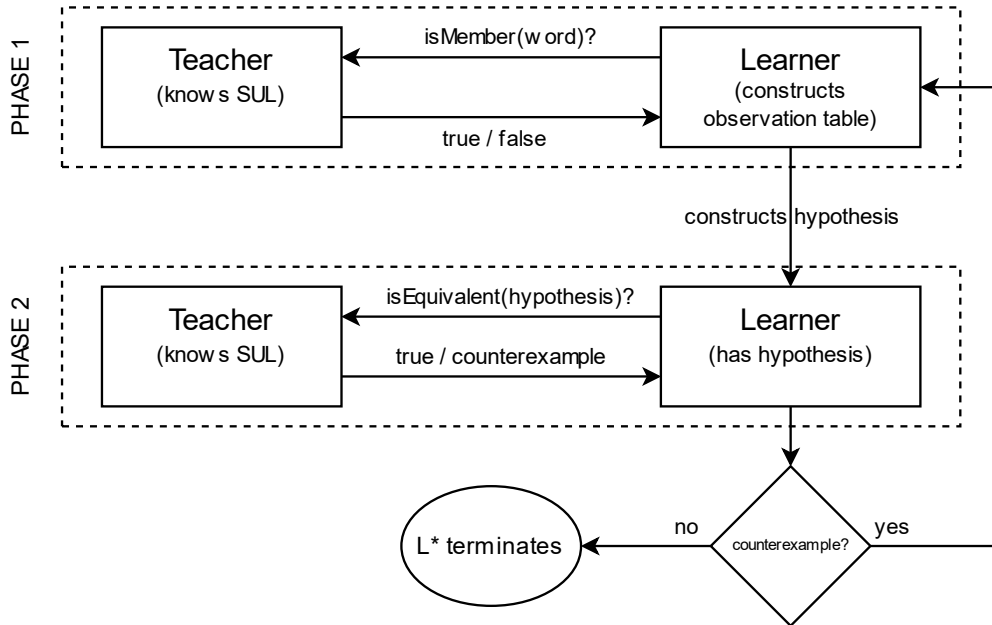
Active automata learning is often based on the L* algorithm proposed by Angluin [15] (see Algorithm 1). Using this algorithm, a *Learner* can learn an initially unknown regular language from a so-called *minimally adequate Teacher*. Under appropriate abstraction, the input-output behaviour of a reactive system can be captured by regular languages, so the L* algorithm is suitable to learn a reactive system like a website [8].

L* Algorithm

```

1:  $P \leftarrow \emptyset$ 
2:  $X \leftarrow \emptyset$ 
3: Ask membership queries for  $\epsilon$  and each  $i \in \Sigma$ .
4: Construct the initial observation table  $(P, X, T)$ .
5: repeat
6:   while  $(P, X, T)$  is not closed or not consistent do
7:     if  $(P, X, T)$  is not consistent then
8:       Find  $t_u, t_v \in P$ ,  $i \in \Sigma$ , and  $x \in X$  such that  $\text{row}(t_u) = \text{row}(t_v)$  and  $T(t_u \cdot i \cdot x) \neq T(t_v \cdot i \cdot x)$ 
9:       Add  $i \cdot x$  to  $X$ 
10:      Extend  $T$  to  $(P \cup P \cdot \Sigma) \cdot X$  using membership queries
11:    end if
12:    if  $(P, X, T)$  is not closed then
13:      Find  $t_u \in P, i \in \Sigma$  such that  $\text{row}(t_u \cdot i)$  is different from  $\text{row}(t_v)$  for all  $t_v \in P$ 
14:      Add  $t_u \cdot i$  to  $P$ 
15:      Extend  $T$  to  $(P \cup P \cdot \Sigma) \cdot X$  using membership queries
16:    end if
17:  end while
18:  Once  $(P, X, T)$  is closed and consistent, let  $\mathcal{M} = M(P, X, T)$ 
19:  Make the conjecture  $\mathcal{M}$ 
20:  if Teacher replies with a counterexample  $t$  then
21:    Add  $t$  and all its prefixes to  $P$ 
22:    Extend  $T$  to  $(P \cup P \cdot \Sigma) \cdot X$  using membership queries
23:  end if
24: until Teacher replies yes to the conjecture  $\mathcal{M}$ 
25: return  $\mathcal{M}$ 

```

Algorithm 1: L* algorithm [15]**Figure 2:** Basic procedure of the L* algorithm

To be minimally adequate, the Teacher must be able to answer two types of queries about the unknown regular language:

- A *membership query*, which is answered with *true* or *false* depending on if a given word is a member of the unknown regular language, and
- an *equivalence query*, which is answered with *true* if the current hypothesis the Learner has of the language is equivalent to the real unknown language. Otherwise, the Teacher will provide a word that is a member of the hypothesis, but not of the real unknown language, or vice versa, as a counterexample. That such a word exists shows that the hypothesis and the real unknown language are not equivalent.

In Angluin's paper [15] both the real unknown language the Teacher knows, and the Learner's hypothesis are represented as deterministic finite-state automata (DFA). To answer a membership query, the Teacher traces the given word through the DFA of the real language. The time complexity for this process is polynomial in the number of states of the DFA and the length of the given word.

To answer an equivalence query, the Teacher may directly determine the equivalence of two DFAs using a standard algorithm (if the DFA of the real language is known), or may use some external mechanism (e.g., model checking, conformance testing). Under the assumption that this is a polynomial-time process, the L* algorithm as a whole has a polynomial time complexity.

The Learner learns the unknown regular language as an iterative process in multiple *learning rounds*. A learning round consists of two phases, which are shown in Figure 2.

In the first phase the Learner asks multiple membership queries of the Teacher. The information received through these membership queries is organized in an *observation table*, which is defined by three parts:

- A non-empty prefix closed set P of strings. Prefix-closed means that every prefix of a string in P is also a member of P .
- A non-empty suffix closed set X of strings. Suffix-closed means that every suffix of a string in X is also a member of X .
- A function T that maps $((P \cup P \cdot \Sigma) \cdot X)$ to $\{true, false\}$, depending on the result of a membership query.

	X
P	$T((P \cup P \cdot \Sigma) \cdot X)$
$P \cdot \Sigma$	

Table 1: Structure of an L* observation table

This can be represented as a 2D array, the above-mentioned observation table, as shown in Table 1, with $(P \cup P \cdot \Sigma)$ being the header of the rows of the table, X being the header of the columns of the table, and the results of T being the values in the corresponding cells.

The strings in P are essentially access sequences to established states. $P \cdot \Sigma$ is created by taking each string in P and appending a single input symbol from the input alphabet Σ . This means that $P \cdot \Sigma$ contains strings that are access sequences to potential successor states of the states reached through P [14].

To be able to construct a hypothesis DFA, the observation table must be *closed* and *consistent*.

Closedness

An observation table is closed, if and only if for every string t_u in $P \cdot \Sigma$ there exists a string t_v in P such that $row(t_u) = row(t_v)$ [15]. This means that all transitions must lead to already established states.

Closing an observation table that is not closed is achieved by moving a row in the $P \cdot \Sigma$ section whose content does not match any rows already present in the P section, to the P section. Then its potential successors are added to $P \cdot \Sigma$, and the observation table is filled using new membership queries [14].

Consistency

An observation table is consistent if and only if for any two strings t_u and t_v in P where $\text{row}(t_u) = \text{row}(t_v)$, for any $i \in \Sigma$, $\text{row}(t_u \cdot i) = \text{row}(t_v \cdot i)$ [15]. This means that all equivalent states (i.e. rows with the same row content) have equivalent successors for every transition. Otherwise, at least one state has two distinct successors for the same input symbol i , meaning the automaton shows non-determinism. As the L^* algorithm assumes determinism, it can be inferred that the offending states cannot be equivalent. The inconsistency is resolved by adding a new string $i \cdot x$ to X , where x is one of the strings previously already in X , and filling the observation table using new membership queries [15]. This results in distinct states (i.e. rows do not have the same row content any longer) [14] [16].

When the observation table is closed and consistent, the Learner constructs a hypothesis DFA, where each row in the P section corresponds to a state, and each row in the $P \cdot \Sigma$ section defines a transition to a successor state. In the second phase of the learning round, the Learner then asks an equivalence query of the Teacher. If the hypothesis DFA is equivalent to the unknown regular language DFA, the algorithm terminates. Otherwise, the Learner learns a counterexample to its current hypothesis, adds this information to the observation table, and returns to Phase 1, asking more membership queries to construct a new hypothesis.

2.4.1.2 Equivalence Oracles

As already mentioned, in the L^* algorithm, the Teacher must be able to answer *equivalence queries*, which specify whether the Learner's hypothesis is equivalent to the SUL. In AALpy [8], the learning library used in this thesis project (see Section 2.5), these equivalence queries are addressed via conformance testing. A test suite is generated from the hypothesis and executed on both the SUL and the hypothesis. If a test leads to different results, a counterexample has been found.

Different *equivalence oracles* exist. The choice of oracle and its parameters determine the amount and type of testing performed in an equivalence query [8].

In this thesis, mainly the State Prefix equivalence oracle was used. Additionally, the Random Walk equivalence oracle was used for the evaluation of the prototype (see Chapter 7) only.

State Prefix Equivalence Oracle

The State Prefix equivalence oracle explores the surroundings of each discovered state by starting random walks of a specified maximum length a specified number of times. Each state is first accessed using the shortest possible path on the hypothesis. Then the state is used as the root of the random exploration. This avoids excessive testing of initial states.

It can also be configured whether states should be handled in descending order of the length of their access sequence, or in a random order [17].

Random Walk Equivalence Oracle

The Random Walk equivalence oracle starts a random walk from the initial state. After every step, the system is reset with a specified probability and a new random walk started from the initial state. The total number of steps to be taken and whether the step counter should be reset after a counterexample has been found can also be configured [18].

2.4.2 Passive Automata Learning

Passive automata learning learns the system from previously generated and documented data. This data represents traces through the system. Each trace is commonly given as a sequence of input-output pairs. Traces may be generated from real user interactions via system logs, or through intentionally using specific sequences of inputs to actively query the SUL in advance of the actual passive learning process.

2.4.2.1 State-Merging in the Red-Blue Framework [19]

Passive automata learning is often done using state-merging algorithms. State-merging algorithms consist of the following steps:

- A *Prefix Tree Automaton (PTA)* is created out of the traces. It is assumed that all traces start in the same initial state. Each trace is added as a sequence of transitions and states from this common root node. States that share the same prefix can be grouped together, resulting in a tree-shaped automaton that describes exactly the behaviour observed in the traces.
- The automaton is generalized by *iteratively merging compatible states* until no more merges are possible.

To reduce the number of compatibility checks between states, the red-blue framework restricts the states that are considered for a merge in each iteration of the algorithm. States are assigned to three distinct sets:

- *Red states* are considered mutually distinct. This set is initialised with the initial state, which corresponds to the root node of the PTA.
- *Blue states* are all states that are reachable from a red state in a single transition.
- *White states* are all states that are neither red or blue.

Only pairs of a red state and a blue state are considered *merge candidates*. Merging two states results in a single state that contains the behaviour of both original states.

All merge candidate pairs are checked for compatibility. As the goal is to learn a deterministic Mealy machine, if the merge of two states s_0 and s_1 would result in a state that has two outgoing transitions with the same input but different outputs (non-determinism), the two states are considered incompatible and the merge candidate pair is rejected. Figure 3 shows an example of this.

If the merge would result in a state that has two outgoing transitions with the same input and output, but leading to two different states, this implies that these successor states are in reality the same state and must also be merged. This is called an *implied merge*. Figure 4 shows an example of a merge resulting in an implied merge. Two states can only be merged if all related implied merges would also be compatible. Additional compatibility checks may be added to the algorithm based on domain knowledge.

If a blue state is incompatible with all red states, it is promoted to red. If there exist multiple such blue states in the same iteration of the algorithm, ties are broken using a specified node order.

If no blue state was promoted in this iteration, all compatible merge candidate pairs are scored, taking any implied merges into account, and the merge with the best score is executed. Different scoring functions may be used, which also may include domain knowledge. The same specified node order as above is used to break ties if there are multiple merge candidate pairs with the same score.

When all states are red, the algorithm terminates because no more merges are possible.

Algorithm 2 shows the pseudocode for state merging in the red-blue framework.

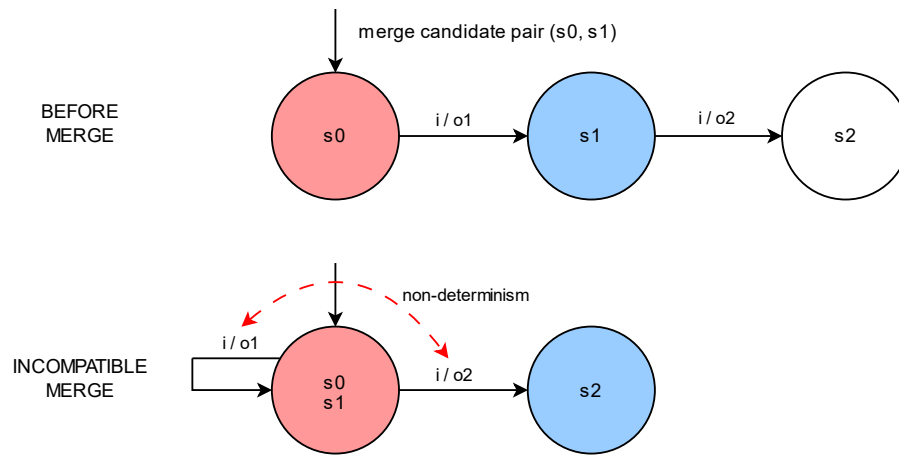


Figure 3: State Merging – Merge incompatibility of two states due to resulting non-determinism

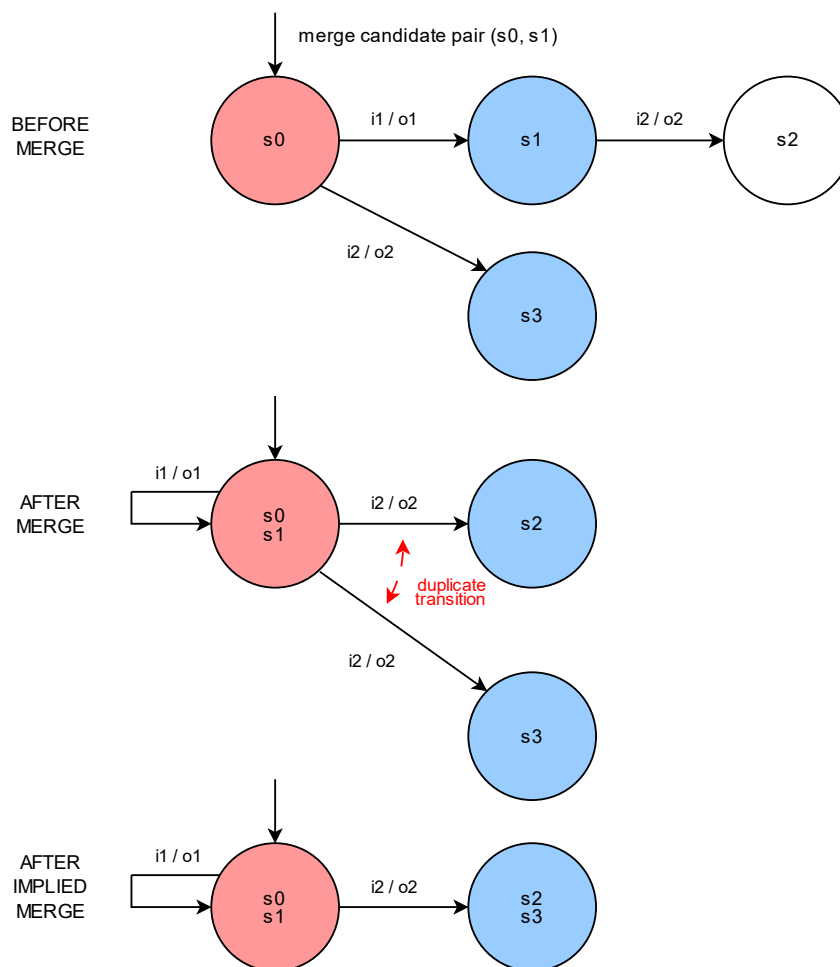


Figure 4: State Merging – Implied merge due to duplicate transition

State Merging in the Red-Blue Framework

Input: collection of traces $\mathcal{TR}$ **Parameters:** scoring function $score$, state order $<$ **Output:** automaton model $\mathcal{M}$

```

1:  $\mathcal{M} \leftarrow$  PTA created from  $\mathcal{TR}$ 
2:  $S_r \leftarrow \{s_0^{\mathcal{M}}\}$  ▷ Initialize red states to initial state
3: loop
4:    $S_b \leftarrow$  set of blue states derived from  $S_r$ 
5:   if  $S_b = \emptyset$  then
6:     return  $\mathcal{M}$  ▷ Terminate if no more merge candidates are available
7:   end if
8:    $S_x \leftarrow \{b \in S_b \mid \forall r \in S_r : score(\mathcal{M}, r, b) = -\infty\}$  ▷ Unmergeable blue states
9:   if  $S_x \neq \emptyset$  then
10:     $S_r \leftarrow S_r \cup \{\min_{<} Q\}$  ▷ Promote minimal unmergeable blue state
11:   else
12:     $(r, b) \leftarrow \arg \max_{(r,b) \in S_r \times S_b} score(\mathcal{M}, r, b)$  ▷ Select best merge wrt.  $score$ 
13:     $\mathcal{M} \leftarrow merge(\mathcal{M}, r, b)$  ▷ Update model
14:   end if
15: end loop

```

Algorithm 2: State merging in the red-blue framework [19]

2.5 AALpy

AALpy (Active Automata Learning in Python) [8] [20] is an automata learning library written in Python. It supports a wide range of modelling formalisms, including deterministic, non-deterministic, and stochastic automata. For deterministic systems, it can be used to learn deterministic finite automata, Moore machines, and Mealy machines. For systems that show non-deterministic or stochastic behaviour, AALpy can be used to learn observable non-deterministic finite-state machines, Markov decision processes, or stochastic Mealy machines [21].

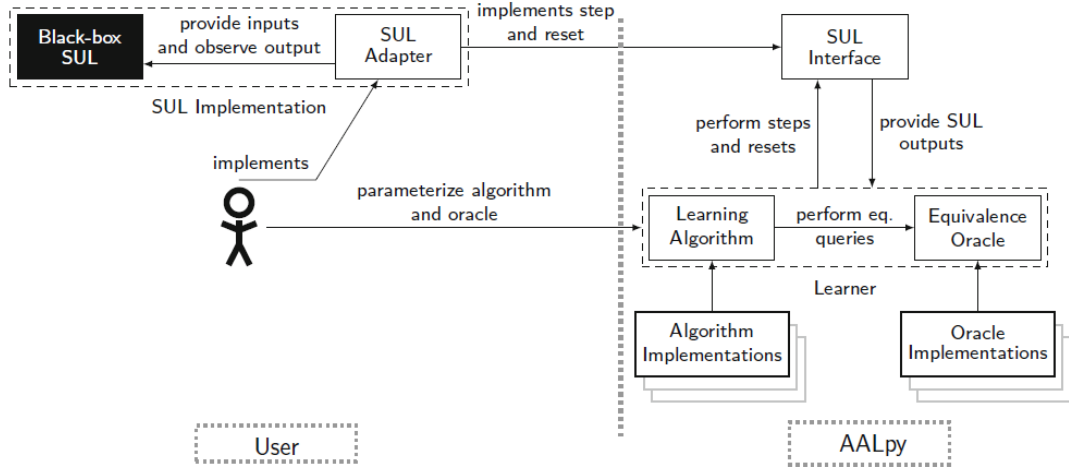
**Figure 5:** AALpy's interface and structure [8]

Figure 5 shows the basic structure of how AALpy works for active learning, which, as the name suggests, was AALpy's original focus. The user needs to implement an adapter to the SUL, which must at least contain the following functionalities:

- a way to submit a single input to the SUL and read the resulting output (step)
- a way to reset the SUL to its initial state (reset)

To actively learn a system, the user must then choose an active learning algorithm (e.g., the L* algorithm) and its parameters, including an equivalence oracle. The learning itself is provided by the AALpy library.

Later, the AALpy library was extended to also include passive automata learning with state-merging algorithms in the red-blue framework [19]. As passive learning learns from preexisting data, the user needs to provide input-output traces. Additionally, the state-merging algorithm can be configured depending on, e.g., domain knowledge.

The same adapter to the SUL that was created for active learning can be used to generate the traces to be used as basis for passive learning if there are no preexisting traces, though the trace generation itself is not provided by the AALpy library.

2.6 Selenium

Selenium [9] is a suite of tools to automate web applications. It is available for Java, Python, C#, Ruby, JavaScript and Kotlin, and supports Chrome, Edge, Firefox, Internet Explorer, and Safari browsers. Selenium WebDriver is designed as a simple and concise programming interface and has a compact object-oriented API. It has a modular structure that allows for cross-browser and cross-platform automation. Its framework consists of three parts:

- A collection of language-specific bindings that define the user interface
- A language-neutral and browser-neutral protocol for controlling the behaviour of a web browser
- A browser-specific driver component responsible for delegating down to the browser, which handles communication between Selenium and the browser

At a basic level, everything Selenium WebDriver does is send the browser commands to do something, or send requests for information. Its main functionality consists of eight different components:

1. Starting a browser session

This defines the type of browser to use, and can be configured with different options. It opens a new window with the specified browser, which is under control by the WebDriver object.

2. Taking an action on the browser

For example, loading a new URL, or deleting the cookies for the current domain.

3. Requesting browser information

For example, getting the current URL, or getting the current cookies.

4. Establishing a waiting strategy

Synchronizing the code with the current state of the browser is one of the biggest challenges with Selenium. An HTML element needs to be on the page before it can be located, and it needs to be in an interactable state before it can be interacted with. The waiting strategy defines how the WebDriver should handle it if this is not (yet) the case.

5. Finding an element

The WebDriver offers different locator strategies (e.g., by tag, by XPath) to get a reference to HTML elements on the current page.

6. Interacting with an element

For example, moving the mouse pointer to the element, or clicking the element.

7. Requesting element information

For example, getting the element position, or checking if it is currently displayed.

8. Ending the browser session

This closes the browser and ends the driver process.

3 Methodology

In this thesis, AALpy’s implementation of different learning algorithms and Selenium’s website frontend interaction functionality are combined to learn an automaton model of a website system.

First, the SUL is analysed for information necessary to the learning process, such as which interactive elements exist on the website. Then, depending on the configuration of the learning application, either an active or a passive learning process is started. The active learning uses AALpy’s implementation of the L^* algorithm. For passive learning, first traces are generated, and then AALpy’s implementation of state-merging in the red-blue framework² is used. An overview of this process can be seen in Figure 6.

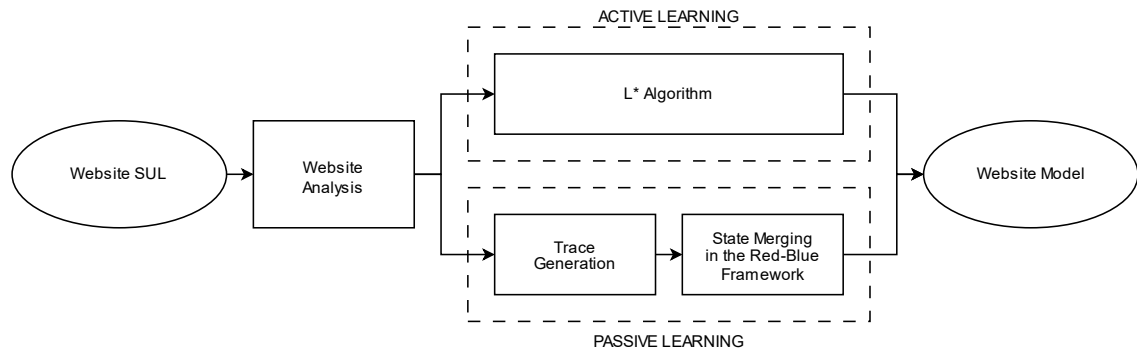


Figure 6: Overview of the website learning process

Some design decisions were made about how to handle certain aspects of this automata learning process, which are described in more detail below.

3.1 Running Example

To explain parts of the methodology of this thesis project, a running example is used, which is shown in Figure 7. It consists of a website with three connected pages. Page 1 contains Button A which loads Page 2 when clicked. Page 2 contains Buttons B and C. Clicking B leads back to Page 1. Clicking C loads Page 3. Page 3 does not contain any HTML elements except a header.

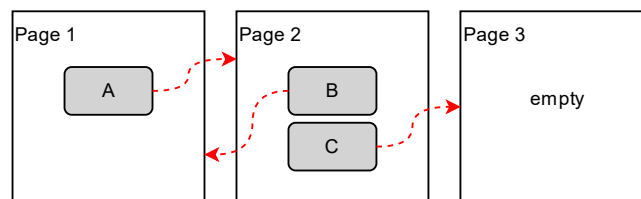


Figure 7: Running Example – Schematic of the website structure

The running example can be found under https://lklamb.github.io/Test_Website/running-example_page1.html, and loaded in the learning application under the name `RUNNING_EXAMPLE`.

Page 1, Page 2 and Page 3 is sometimes abbreviated as P1, P2, and P3 for convenience.

² GeneralizedStateMerging

3.2 Accessing the Website SUL

Selenium's Chrome WebDriver is used to access the website SUL and interact with it. The WebDriver opens a new Chrome browser session, which is then used to load URLs, get the current URL, access elements on the currently loaded webpage, delete cookies etc.

3.3 Automata Characteristics

AALpy's learning algorithms support the learning of different types of automata. The learning application is set up to learn a deterministic Mealy machine for all configurations. The characteristics of the learned Mealy machine, as well as how they are defined and used in the learning application, are described in the following sections.

3.3.1 Input Alphabet – Scope

Both the active L^* algorithm as well as the trace generation for passive learning need a predefined input alphabet to know which inputs to test on the SUL. For this purpose, a separate phase was implemented, in which the website SUL is analysed in advance of the learning process (see Figure 6).

The learning application must be configured with a list of URLs to include in the exploration, which defines the scope of the learning algorithm. Each of the URLs in this scope is loaded in the WebDriver. Then, the corresponding page is analysed to find interactive elements, which are added to the input alphabet with their corresponding interactions. All interactions that trigger behaviour on a webpage in the scope are part of the input alphabet. This is not influenced by whether the webpages are connected or not.

If at any point the exploration of the system under learning reaches a webpage whose URL is not contained in this scope, it will not continue from this point, because the valid inputs on webpages outside the scope are unknown and therefore not part of the input alphabet. This URL at the boundary of the scope will add an additional state to the learned automaton, and be documented so that it might be added to the scope in future executions.

Contrary to the definition of a website, the learning process is not limited to webpages under the same domain. All webpages that are defined in the URL scope and can be reached from the initial URL will be explored.

Running Example

The input alphabet depends on the scope that was defined. Table 2 shows how the definition of the scope influences the input alphabet on the running example.

<i>Scope</i>	<i>Resulting Input Alphabet</i>
<i>Page 1</i>	<i>{click A on P1}</i>
<i>Page 2</i>	<i>{click B on P2, click C on P2}</i>
<i>Page 3</i>	<i>{}</i>
<i>Page 1, Page 2</i>	<i>{click A on P1, click B on P2, click C on P2}</i>
<i>Page 1, Page 3</i>	<i>{click A on P1}</i>
<i>Page 2, Page 3</i>	<i>{click B on P2, click C on P2}</i>
<i>Page 1, Page 2, Page 3</i>	<i>{click A on P1, click B on P2, click C on P2}</i>

Table 2: Running Example – Resulting input alphabets for different scope definitions

3.3.2 Initial State – Reset

The learning application uses a predefined starting point for the exploration of the SUL. This starting point needs to be specified prior to execution in the form of an initial URL. This initial URL will automatically be added to the exploration scope mentioned above.

Before each exploratory walk through the SUL, either during active learning or during trace generation for passive learning, the website is (re)set to this initial URL.

Running Example

https://lklamb.github.io/Test_Website/running-example_page1.html is used as the initial URL for the running example.

3.3.3 Input Enabled Handling

As already mentioned, the learning process needs a predefined input alphabet to know which inputs to test on the SUL. AALpy's extended implementation of L^* to learn a Mealy machine assumes that the SUL is input enabled [8], so for every state and every input in the input alphabet the output function must be defined. The implemented trace generation for passive learning assumes that the SUL is input enabled as well.

A website in general is not input enabled, e.g., a specific button can only be clicked if the webpage it is on is currently loaded. Additionally, an element can only be interacted with if it is currently displayed³ in the browser. Problems might also occur if the element to interact with is covered by a different element.

To handle this discrepancy, two different approaches were implemented in the learning application, one of which must be chosen before running the application.

3.3.3.1 Self-Loops

The first approach creates a self-loop whenever the learning application tries to interact with an element that cannot currently be interacted with. This is done by returning various specific output strings whenever the application encounters this problem. The output string includes the reason the interaction failed (see also Section 3.3.4). As the SUL is assumed to be deterministic, it will react with this same output every time this happens, creating a self-loop. No additional states are created.

3.3.3.2 Sink State

The second approach inserts an artificial sink state into the learned automaton. Whenever the learning application tries to interact with an element that is not on the current page, the same output corresponding to a transition to this sink state will be returned.

On all subsequent steps on this walk through the system, an output that corresponds to staying in this sink state will be returned. Only if a new walk is started will different outputs be possible again.

3.3.4 Output – Current URL

To keep the learning as general as possible, the current URL after an interaction is used as an output. This means that some hidden states of the website can only be observed if they consequently trigger a change of the URL further down the trace. Interactions that do not affect the URL at all (e.g., changing an embedded image) cannot be observed. It would be possible to observe more specific

³ visible, not hidden

states of the website by accessing the state of different HTML elements directly, but then the implementation would have to be more closely tailored to the website.

The prototype of the learning application used strictly the current URL as an output (see Chapter 7). The current version has been improved to handle different settings and cover more edge cases, and therefore also includes some specific output strings. The set of outputs that are used depends on the type of input enabled handling that was chosen.

All additional output strings are listed below. If multiple conditions apply, the output that is higher in the list will be used. Only if none apply will the output be the current URL after the interaction.

1. *StaySink*, if the walk has reached the sink state through previous interactions (sink state input enabled handling only)
2. *Boundary- $\langle$ current URL $\rangle$* , if the current URL before an interaction is not inside the defined URL scope, so none of the interactions in the input alphabet will be found on this page
3. *DeadEnd- $\langle$ current URL $\rangle$* , if the current page before an interaction does not have any interactive elements
4. *NotOnCurrentPage* (for self-loop input enabled handling) or *GoSink* (for sink state input enabled handling), if the element to interact with is not on the current page
5. *NotInteractable*, if the element to interact with cannot be interacted with, e.g., because it is currently not displayed
6. *InteractionIntercepted*, if the interaction with the element was intercepted, e.g., due to the element being covered by another element

It is assumed that if the interaction returned *NotInteractable* or *InteractionIntercepted* as an output, the page the SUL is currently on is the same as before the interaction.

3.4 Active Learning

The configuration described in this section is used in the active automata learning process.

3.4.1 Active Learning Algorithm

AALpy's implementation of the L^* algorithm is used for active learning.

3.4.2 Equivalence Oracle

AALpy supports the use of different equivalence oracles in its implementation of the L^* algorithm. Each of these equivalence oracles can be further configured by setting parameters.

For the first prototype described in Chapter 7, experiments were run with the State Prefix equivalence oracle and the Random Walk equivalence oracle with different parameter values. The State Prefix equivalence oracle showed much better results, so usage of the Random Walk equivalence oracle was abandoned.

Running Example

Figure 8 defines the initial page, scope, and input alphabet for the running example to be used in active learning. If the parameters for the State Prefix equivalence oracle are chosen in a sufficient way, AALpy's L^* algorithm learns a correct model of the SUL as shown in Figure 9. The structure of the automaton is dependent on the type of input enabled handling (IEH) that was chosen.

The outputs are abbreviations for the output strings listed in Section 3.3.4. Page specifiers on the inputs have been omitted for easier readability.

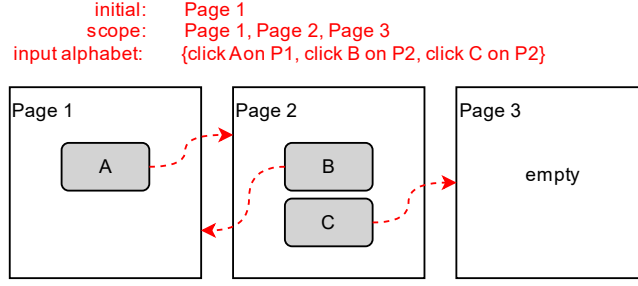


Figure 8: Running Example – Definition of initial page and scope, and resulting input alphabet to be used in active learning

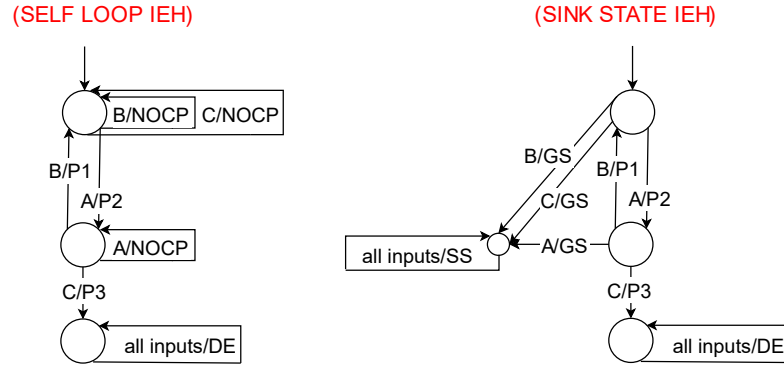


Figure 9: Running Example – Final learned model using active learning (NOCP = *NotOnCurrentPage*, DE = *DeadEnd*, GS = *GoSink*, SS = *StaySink*)

3.5 Passive Learning

The configuration described in this section is used in the passive automata learning process.

3.5.1 Trace Generation

Passive learning learns from preexisting data. For a website, even if sufficient logs for user traces exist, there is no way to determine whether these traces cover the whole behaviour without knowing the website in advance. Therefore, the learning application instead intentionally creates the data to be used in passive learning. This trace generation process is parametrized by a specified trace length, which defines how deeply the system should be explored.

Random trace generation has the same drawbacks as real user data regarding coverage, whereas exhaustively generating all possible input sequences up to the specified trace length quickly runs into time and/or memory limits.

The trace generation therefore takes domain knowledge into account to reduce the number of traces necessary to cover the whole behaviour. This is achieved by actively exploring the SUL in a breadth-first manner while tracking the webpage the SUL is currently on at the end of each branch. Only valid prefixes are stored and explored further until the specified trace length is reached. Valid prefixes contain only inputs that match the webpage that the SUL is on at any point in time.

If a trace reaches a webpage where no inputs are valid, it is not explored further, but added to the IO traces for learning as is. When the specified trace length has been reached, all remaining valid prefixes are added to the IO traces for learning. The trace generation terminates prematurely if all traces have reached a webpage where no further exploration is necessary.

This also reduces the number of necessary interactions with the SUL significantly, which is the bottleneck of the learning process. The extent of the reduction is dependent on the structure of the website SUL. In the extreme case of a trace length of one with all inputs being on the initial webpage, there is no reduction at all, though in this case the existing behaviour to be learned is also limited.

Algorithm 3 shows the pseudocode for trace generation.

Trace Generation for Passive Learning

Input: website system SUL , input alphabet Σ , initial page $page_{init}$

Parameters: trace length len

Output: set of IO traces TR

```

1:  $TR \leftarrow \emptyset$ 
2:  $Pre_{valid} \leftarrow \{(\epsilon)\}$  ▷ Initialize valid prefixes with a prefix of length 0
3: for  $i = 1$  to  $len$  do
4:    $Pre_{new} \leftarrow \emptyset$ 
5:   for all  $prefix \in Pre_{valid}$  do
6:      $page \leftarrow$  page at end of  $prefix$  ▷ Initial page or page after last input
7:     if  $\forall input \in \Sigma; input \text{ not on } page$  then ▷ Reached a dead end
8:       add  $prefix$  to  $TR$ 
9:     else
10:      for all  $input \in \Sigma$  where  $input$  is on  $page$  do
11:        reset  $SUL$ 
12:        execute  $prefix$  on  $SUL$ 
13:         $output \leftarrow$  process  $input$  on  $SUL$ 
14:         $trace_{new} \leftarrow$  append  $(input, output)$  to  $prefix$ 
15:        add  $trace_{new}$  to  $Pre_{new}$ 
16:      end for
17:    end if
18:  end for
19:  if  $Pre_{new} = \emptyset$  then
20:    break ▷ Terminate early if all valid traces have reached a dead end
21:  end if
22:   $Pre_{valid} \leftarrow Pre_{new}$  ▷ Store valid prefixes to explore further
23: end for
24: add all  $prefix \in Pre_{valid}$  to  $TR$ 
25: return  $TR$ 

```

Algorithm 3: Trace generation for passive learning

Running Example

Figure 10 defines the initial page, scope, and input alphabet for the running example to be used in trace generation. The following shows an execution of the trace generation algorithm on the running example using a specified trace length of 3. In the graphs, green nodes have already been visited at least once during the trace generation algorithm. They have been marked with the page the SUL is on in this state.

Figure 11 shows the initialisation. The initial webpage P1, which corresponds to the root node of the exploration, is known. Only the empty trace is a known valid prefix.

In the first iteration (see Figure 12) only A is a valid input, because only A is on P1. It results in the output P2, so it is known that the next node in this branch is a state on webpage P2. All other inputs are not on the initial page, so these branches are pruned immediately. The valid prefixes to be explored further are therefore $\{(A/P2)\}$.

B and C are valid inputs on P2, so in the second iteration (see Figure 13) these two inputs are added to all valid prefixes from the previous iteration. B results in the output P1. C results in the output P3. A is not a valid input in this state, so this branch is pruned. The valid prefixes to be explored further are therefore $\{(A/P2, B/P1), (A/P2, C/P3)\}$.

In the third iteration (see Figure 14) A is a valid input on P1, so this input is added to the prefix that terminated on P1 in the previous iteration. It results in the output P2. P3 does not contain any valid inputs, so the trace that ends here is added directly to the IO traces for learning because it does not need to be explored further. The valid prefixes to potentially be explored further are therefore $\{(A/P2, B/P1, A/P2), B/P1, A/P2)\}$.

After the specified trace length of three has been reached, all remaining valid prefixes are now moved to the IO traces for learning, and the trace generation terminates. This results in the following traces, which will be used to create the Prefix Tree Automaton:

$$IO\ traces = \{(A/P2, C/P3), (A/P2, B/P1, A/P2)\}$$

This approach reduces the number of traces necessary to cover the whole behaviour of the running example system from $3 \text{ (size of input alphabet)}^3 \text{ (trace length)} = 27$ to 2.

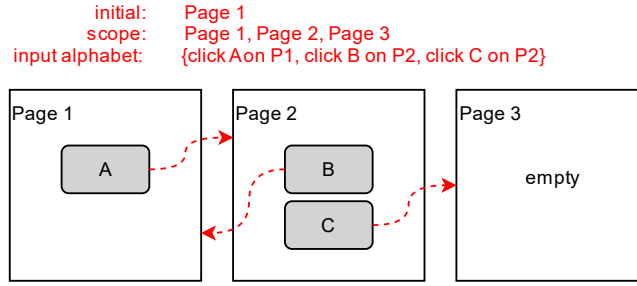


Figure 10: Running Example – Definition of initial page and scope, and resulting input alphabet to be used in trace generation

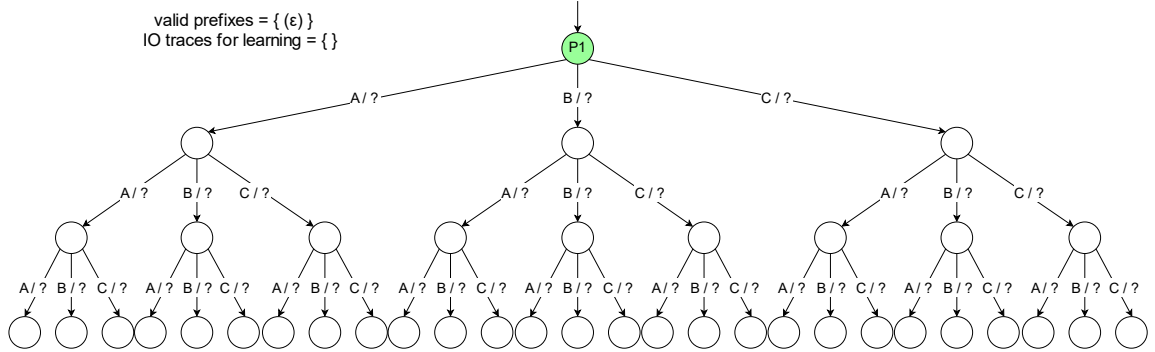


Figure 11: Running Example – Trace generation initialisation

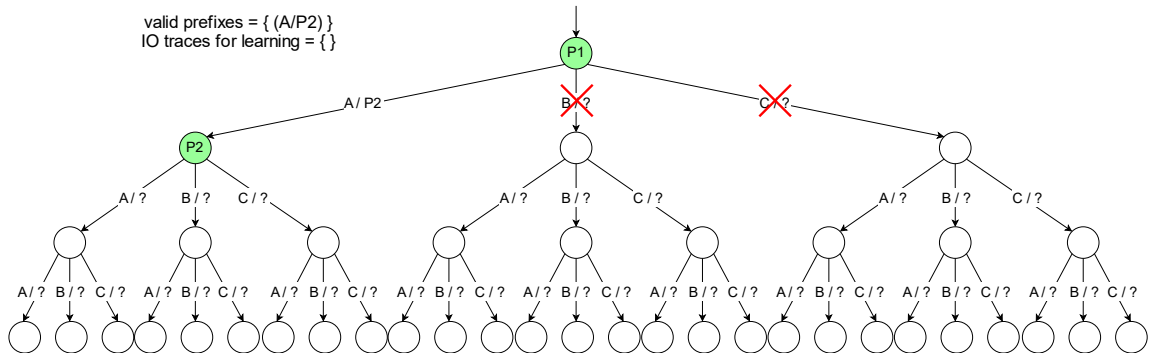


Figure 12: Running Example – Trace generation after iteration 1

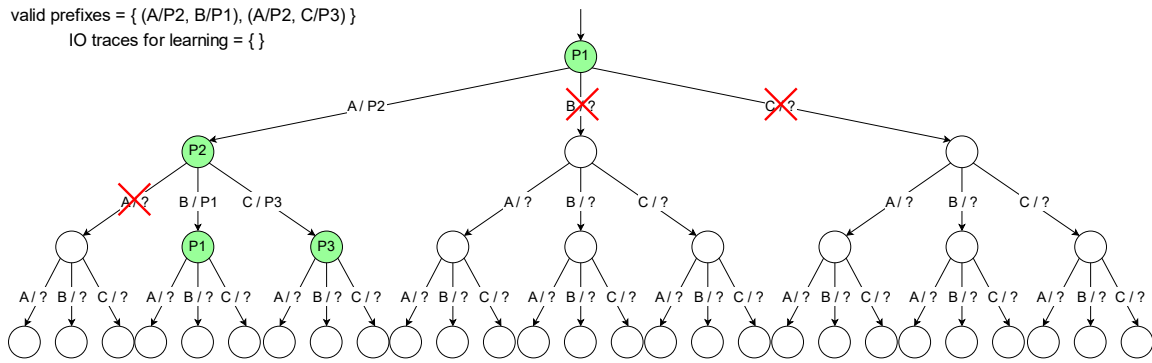


Figure 13: Running Example – Trace generation after iteration 2

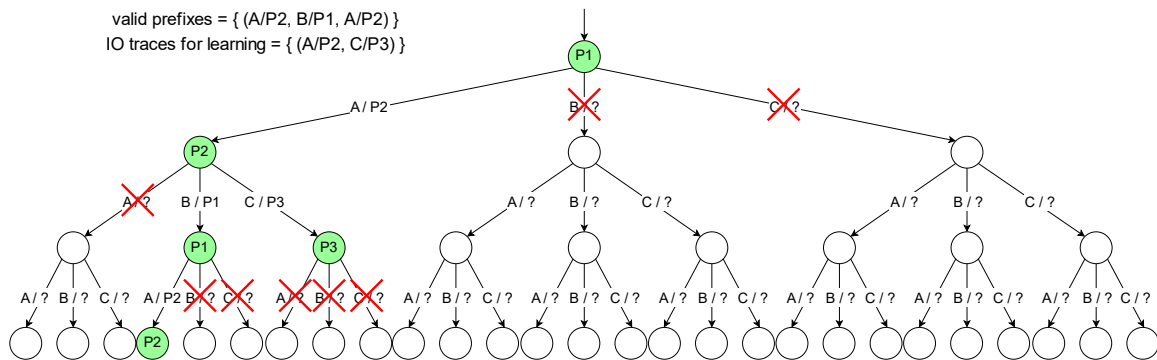


Figure 14: Running Example – Trace generation after iteration 3

3.5.2 Passive Learning Algorithm

From the IO traces generated in the previous step, a Prefix Tree Automaton is created, which is then used as the basis for state-merging in the red-blue framework.

Because only traces with valid inputs for their current webpage at any point in the sequence were generated, two states on two different webpages will have distinct input sets. This consequently means that these two states are merge compatible, because they can never have outgoing transitions with different outputs for the same input. As a result, they might be merged during the state-merging algorithm, which leads to completely nonsensical learned automata.

To prevent this, the information that was lost while reducing the number of necessary traces must be added back in somehow, to keep states on different webpages from merging. The learning application can be configured to use one of two approaches to solve this problem, which are described below.

3.5.2.1 PTA Preprocessing plus Custom Scoring

In this approach, transitions to additional nodes are added to the PTA prior to the execution of the state-merging algorithm in a preprocessing step. As the current page is known for each node of the PTA, and it is known which inputs are on which page, it is possible to add child nodes for all inputs that are not on the current page and have therefore not been covered during trace generation. This includes adding these child nodes to nodes at the very end of each trace. Additionally, if the input enabled handling is set to create a sink state, another layer of nodes is added to each newly created child node to signify the SUL staying in the sink state for all inputs once it has entered it.

The edge cases for pages at the boundary of the scope and webpages without any interactive elements are covered by adding transitions to child nodes with an output specific to the edge case to the node.

The preprocessing step introduces additional overhead to the learning process, and it would have been possible to get these nodes directly from traces by executing one (for self-loop input enabled handling) or two (for sink state input enabled handling) more steps along each of the pruned branches during trace generation. As interacting with the SUL is extremely slow when compared to adding a node artificially, it is still preferable to reduce the number of necessary SUL interactions and resets in any way possible.

Algorithm 4 shows the pseudocode for PTA preprocessing.

PTA Preprocessing

Input: PTA from IO traces PTA , input alphabet Σ , initial page $page_{init}$,

pages in scope $scope$, pages without interactive elements $deadends$, type of input enabled handling $type_{IEH}$

Output: PTA after Preprocessing

```

1: for all  $node \in PTA$  do
2:    $page \leftarrow$  page the SUL is on in this state           ▷ Initial page or page after preceding interaction
3:   if  $page \notin scope$  or  $page \in deadends$  then           ▷ Edge cases with specific outputs
4:     for all  $input \in \Sigma$  do
5:        $output \leftarrow$  specific output string
6:       add transition to new child node with  $output$  to  $node$ 
7:     end for
8:   else
9:     for all  $input \in \Sigma$  where  $input$  not on  $page$  do       ▷ Missing information in traces
10:       $output \leftarrow$  output string signifying that the input is not on the current page
11:      add transition to new child node with  $output$  to  $node$ 
12:      if  $type_{IEH} = \text{SINK STATE}$  then
13:        for all  $input \in \Sigma$  do
14:           $output \leftarrow$  output string for staying in sink state
15:          add transition to new grandchild node with  $output$  to child node
16:        end for
17:      end if
18:    end for
19:  end if
20: end for
21: return  $PTA$ 

```

Algorithm 4: PTA preprocessing

On the preprocessed PTA the state-merging algorithm is then executed, which greedily merges any states that are considered merge compatible using AALpy's default node order. After preprocessing, the PTA contains a lot of nodes without any outgoing transitions, which are therefore merge compatible with any other node. To force these nodes to merge into the correct states, a custom scoring function is used, that penalizes the intended merge if these nodes are not merged into their parent nodes. This then creates self-loops either directly at the node generated from traces (for self-loop input enabled handling) or at the node representing the sink state (for sink state input enabled handling).

Edge cases like pages at the boundary of the scope or pages without interactive elements are also handled via the scoring function, and always forced to create self-loops, no matter the type of input enabled handling.

Running Example

Using the traces from the trace generation in the previous section, a PTA is created (Figure 15, left graph). Depending on which type of input enabled handling (IEH) the learning application is

configured to use, transitions with specific outputs to new nodes are added (Figure 15, middle and right graph). All nodes generated from traces have been marked with the page the SUL is on in this state.

Figure 16 shows what the automaton would look like with all necessary self-loops created correctly. During the learning process this exact status might not occur, as these merges do not necessarily occur before all other merges. The sequence of merges is dependent on AALpy's default node order.

Figure 17 shows the final learned model using passive learning with PTA preprocessing and the custom scoring function. These learned automata match the automata created by the active learning process.

The outputs in the graphs are abbreviations for the output strings listed in Section 3.3.4. Page specifiers on the inputs have been omitted for easier readability.

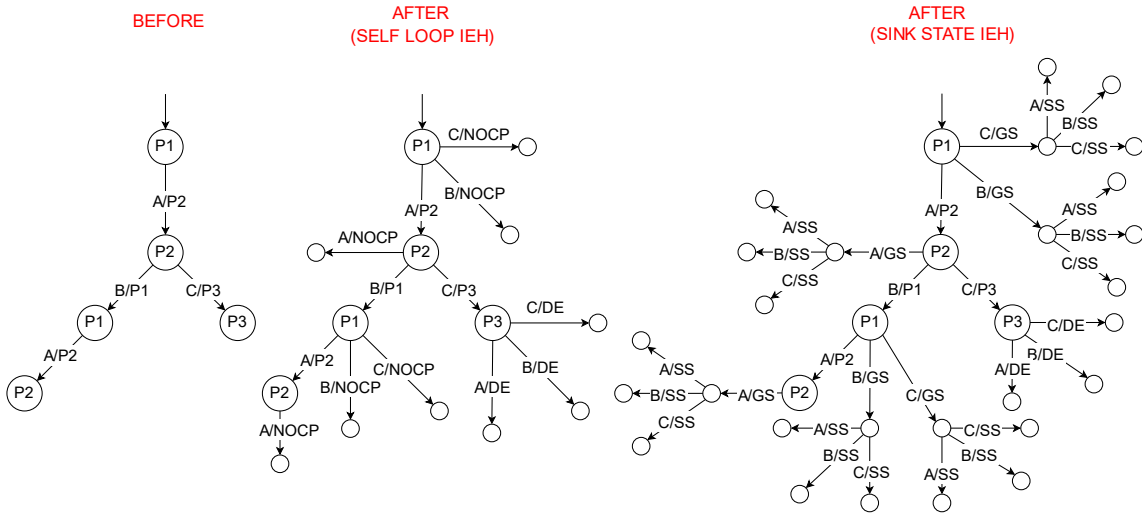


Figure 15: Running Example – PTA before and after preprocessing (NOCP = *NotOnCurrentPage*, DE = *DeadEnd*, GS = *GoSink*, SS = *StaySink*)

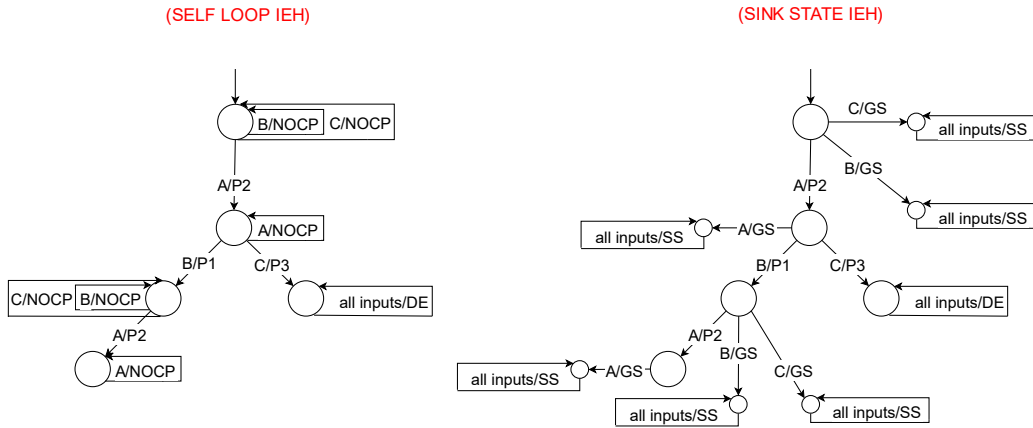


Figure 16: Running Example – Self-loops created via the custom scoring function (NOCP = *NotOnCurrentPage*, DE = *DeadEnd*, GS = *GoSink*, SS = *StaySink*)

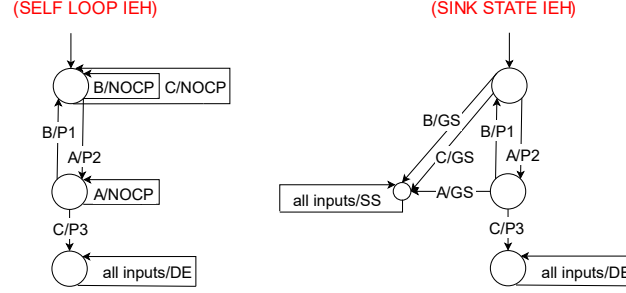


Figure 17: Running Example – Final learned model using passive learning with PTA preprocessing and the custom scoring function (NOCP = *NotOnCurrentPage*, DE = *DeadEnd*, GS = *GoSink*, SS = *StaySink*)

3.5.2.2 Custom Compatibility Function plus Postprocessing

AALpy's implementation of the state-merging algorithm also offers the possibility to define a custom compatibility function, which will be evaluated to find whether two states are merge compatible. As for each node in the PTA it is known which page the SUL is on in this state, this can be used to directly forbid states which are on different pages from merging.

This approach is much more efficient than PTA preprocessing, as the size of the PTA that the state-merging algorithm needs to work on is not increased, but as the information that was lost during trace generation is not added back in, it will also not be present in the final learned automaton. After the state-merging, the learned model is the same no matter the input enabled handling type, and is not input complete. Transitions are only defined for the valid inputs in each state. All valid behaviour is learned, but the automaton does not show the location of edge cases, and therefore does not contain all the information of the automaton created by active learning or by passive learning with PTA preprocessing.

To make the results of this passive approach comparable to the other methods, it is therefore necessary to do some postprocessing on the automaton learned from the state-merging algorithm. As for any state the current page is known, it is possible to make the automaton input complete by adding transitions for those inputs that are not on the current page, either with self-loops or by transitioning to a newly created sink state.

The edge cases for pages at the boundary of the scope and webpages without any interactive elements are covered by adding self-loops with the edge case-specific output strings.

Running Example

Using the traces from the trace generation in the previous section, a PTA is created (Figure 18, left graph). Only states that are on the same page are merged, if they are otherwise merge compatible. This results in an automaton that is not input complete (Figure 18, right graph).

In a postprocessing step, transitions for invalid inputs are added to the automaton dependent on the type of input enabled handling. Figure 19 shows the final learned model using passive learning with the custom compatibility function and postprocessing. These learned automata match the automata created by the active learning process and by passive learning with PTA preprocessing.

The outputs in the graphs are abbreviations for the output strings listed in Section 3.3.4. Page specifiers on the inputs have been omitted for easier readability.

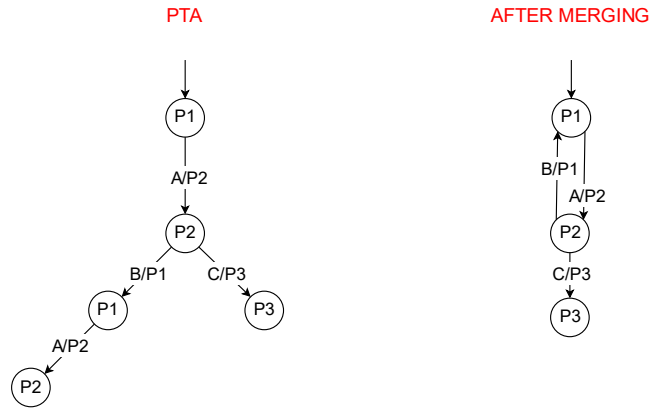


Figure 18: Running Example – Intermediate learned model while passive learning with the custom compatibility function

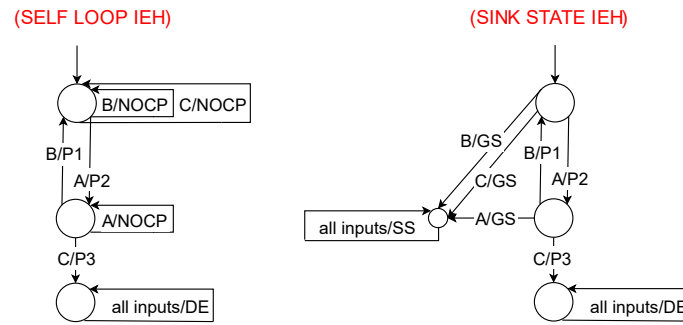


Figure 19: Running Example – Final learned model using passive learning with the custom compatibility function and postprocessing (NOCP = *NotOnCurrentPage*, DE = *DeadEnd*, GS = *GoSink*, SS = *StaySink*)

4 Implementation

In this chapter specifics of the implementation of the learning application are described. The code for the learning application can be found under <https://github.com/lklamb/aalpy-selenium-website-learning>. Figure 20 shows the sequence and data movement of an execution. The details of each step are explained in the following sections.

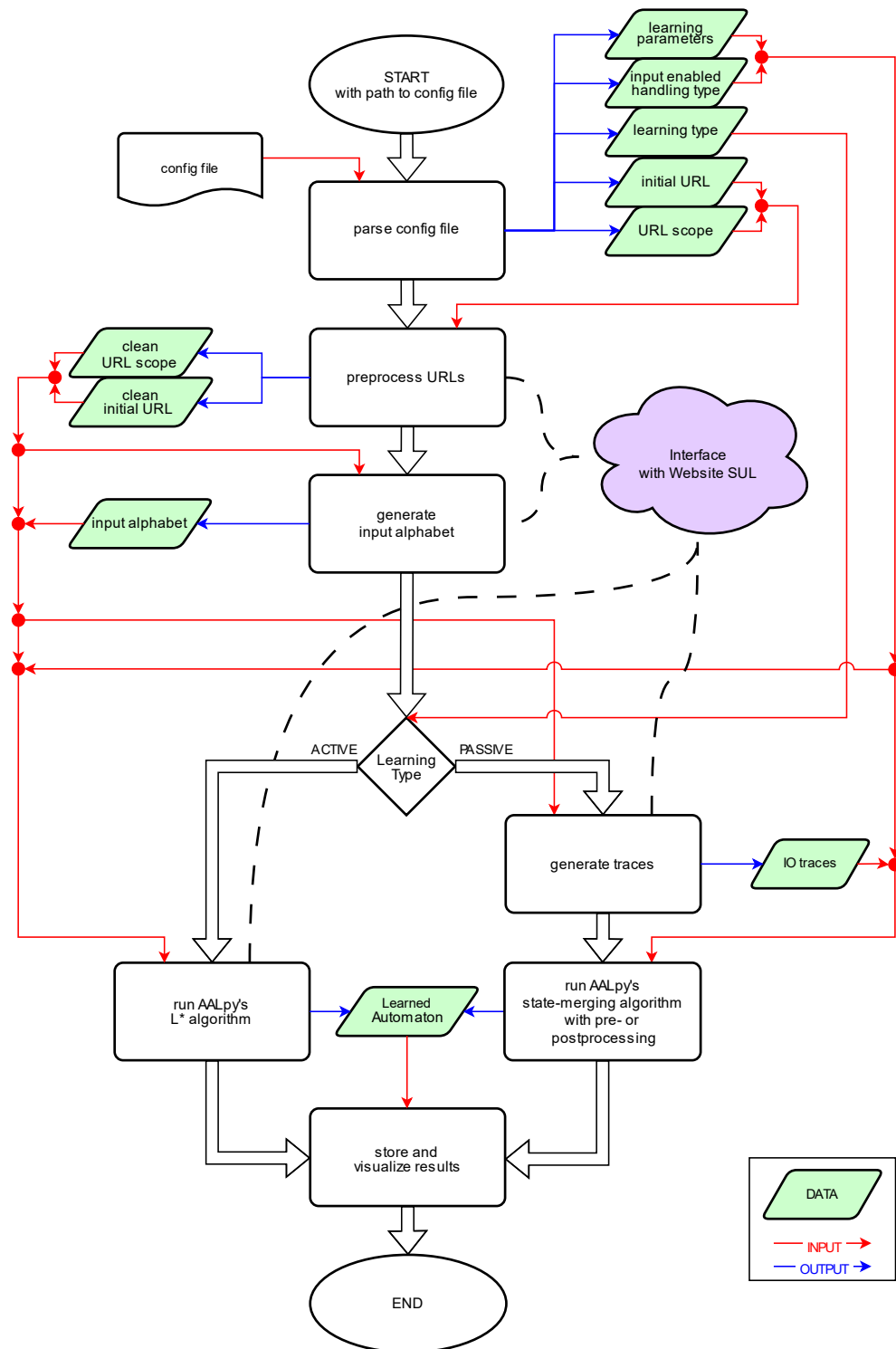


Figure 20: Sequence and data movement of an execution of the learning application

4.1 Tool Versions

For this thesis, AALpy 1.4.0 (for the first prototype, see Chapter 7) and AALpy 1.5.1 (for all further developments) was used as a learning library. The Chrome WebDriver from Selenium 4.17.2 for Python was used to access the website SUL.

4.2 Configuration File

When starting the program, the path to a configuration file in *.yaml* format must be provided. The configuration file must contain information about which website should be learned (*website_to_learn*). This can either be a string matching an internal enum to run one of the examples or demos used in this thesis (see Table 3), or a dictionary specifying *initial_url* and *urls_in_scope* manually (see Table 4).

Other values can or must also be set via the configuration file (see Table 5 and Table 6).

<i>website_to_learn</i>	<i>SUL</i>
<i>RUNNING_EXAMPLE</i>	<i>Loads the example described in Section 3.1.</i>
<i>OUTPUT_EXAMPLE</i>	<i>Loads the example described in Section 5.4.</i>
<i>HOVER_EXAMPLE</i>	<i>Loads the example described in Section 5.5.</i>
<i>MOVING_ELEMENTS_EXAMPLE</i>	<i>Loads the example described in Section 5.9.6.</i>
<i>CAR_ALARM</i>	<i>Loads the whole website described in Section 6.1.</i>
<i>TEST_WEBSITE</i>	<i>Loads the whole website described in Section 6.2.</i>
<i>TW_INDEX</i>	<i>Loads a specific page of the website described in Section 6.2.</i>
<i>TW_BASICS</i>	
<i>TW_MAP_AREAS</i>	
<i>TW_DROPDOWNS</i>	
<i>TW_CHECKBOXES</i>	
<i>TW_SELECTIONS_CHOICES</i>	
<i>TW_ATTRIBUTE_COMBINATIONS</i>	
<i>TW_TABS_WINDOWS</i>	

Table 3: Allowed string values for *website_to_learn* in the configuration file

```

website_to_learn:
  initial_url: https://lklamb.github.io/Test_Website/running-example_page1.html
  urls_in_scope:
    - https://lklamb.github.io/Test_Website/running-example_page1.html
    - https://lklamb.github.io/Test_Website/running-example_page2.html
    - https://lklamb.github.io/Test_Website/running-example_page3.html

```

Table 4: Specifying *website_to_learn* manually in the configuration file

<i>Name</i>	<i>Description</i>
<i>learning_type</i>	<i>This defines which learning type should be used, active or passive learning.</i>
<i>input_enabled_handling</i>	<i>This defines how the SUL should be made input enabled, with self-loops or by inserting a sink state.</i>
<i>include_hover</i>	<i>This is a testing and debugging option that defines whether HTML elements that include attributes that trigger behaviour on hover should be looked for when generating the input alphabet.</i>
<i>display_cursor</i>	<i>This defines whether or not the mouse cursor should be visualized during the learning process.</i>
<i>wait_time</i>	<i>This defines how long Selenium should wait before aborting an element location call using its implicit wait strategy (see Section 4.3).</i>
<i>equivalence_oracle</i>	<i>This defines which equivalence oracle to use during learning, the State Prefix or the Random Walk equivalence oracle.</i>
<i>walks_per_state</i>	<i>This defines how many walks should be executed per state when using the State Prefix equivalence oracle.</i>
<i>walk_len</i>	<i>This defines the length of each walk when using the State Prefix equivalence oracle.</i>
<i>depth_first</i>	<i>This defines whether newly discovered states should be explored first or in a random order when using the State Prefix equivalence oracle.</i>
<i>num_steps</i>	<i>This defines how many steps should be taken in total when using the Random Walk equivalence oracle.</i>
<i>reset_prob</i>	<i>This defines the probability that after a step, the system is reset to its initial state, when using the Random Walk equivalence oracle.</i>
<i>trace_len</i>	<i>This defines the length of traces to be generated when using passive learning.</i>
<i>passive_approach</i>	<i>This defines how the resulting automaton is made input complete during passive learning, with PTA preprocessing and a custom scoring function, or with a custom compatibility function and automaton postprocessing.</i>
<i>logging_level</i>	<i>This defines how detailed the logging should be.</i>
<i>store_results</i>	<i>This defines whether or not results should be copied to a separate directory for permanent storage.</i>
<i>store_dir</i>	<i>This defines where to store permanent results.</i>
<i>seed</i>	<i>If a seed is set, the active learning process, which uses some pseudo-randomness, becomes reproducible. The passive learning does not use any pseudo-randomness and is therefore reproducible by default.</i>
<i>create_pdfs</i>	<i>This option defines whether the learned automaton should be visualized as a pdf. The visualisation might cause issues when learning complex systems. The automaton is saved in a .dot file either way.</i>
<i>headless</i>	<i>This defines whether the browser should be run in headless mode, i.e., without a visible user interface.</i>

Table 5: Configuration options – Description

<i>Name</i>	<i>Allowed Values</i>	<i>Default</i>	<i>Mandatory</i>
<i>learning_type</i>	<i>ACTIVE</i> <i>PASSIVE</i>	-	<i>yes</i>
<i>input_enabled_handling</i>	<i>SELF_LOOP</i> <i>SINK_STATE</i>	-	<i>yes</i>
<i>include_hover</i>	<i>bool</i>	<i>true</i>	<i>no</i>
<i>display_cursor</i>	<i>bool</i>	<i>false</i>	<i>no</i>
<i>wait_time</i>	<i>float</i>	<i>0.0</i>	<i>no</i>
<i>equivalence_oracle</i>	<i>STATE_PREFIX</i> <i>RANDOM_WALK</i>	<i>STATE_PREFIX</i>	<i>no</i>
<i>walks_per_state</i>	<i>uint</i>	-	<i>yes, if</i> <i>equivalence_oracle</i> <i>is STATE_PREFIX</i>
<i>walk_len</i>	<i>uint</i>	-	<i>yes, if</i> <i>equivalence_oracle</i> <i>is STATE_PREFIX</i>
<i>depth_first</i>	<i>bool</i>	-	<i>yes, if</i> <i>equivalence_oracle</i> <i>is STATE_PREFIX</i>
<i>num_steps</i>	<i>uint</i>	-	<i>yes, if</i> <i>equivalence_oracle</i> <i>is RANDOM_WALK</i>
<i>reset_prob</i>	<i>float [0.0, 1.0]</i>	-	<i>yes, if</i> <i>equivalence_oracle</i> <i>is RANDOM_WALK</i>
<i>trace_len</i>	<i>uint</i>	-	<i>yes, if learning_type</i> <i>is PASSIVE</i>
<i>passive_approach</i>	<i>PTA_PREPROCESSING</i> <i>COMPATIBILITY_FUNC</i>	-	<i>yes, if learning_type</i> <i>is PASSIVE</i>
<i>logging_level</i>	<i>DEBUG</i> <i>INFO</i> <i>WARNING</i> <i>ERROR</i> <i>CRITICAL</i>	<i>INFO</i>	<i>no</i>
<i>store_results</i>	<i>bool</i>	<i>false</i>	<i>no</i>
<i>store_dir</i>	<i>string</i>	<i>StoredResults/</i>	<i>no</i>
<i>seed</i>	<i>int</i>	<i>None</i>	<i>no</i>
<i>create_pdfs</i>	<i>bool</i>	<i>true</i>	<i>no</i>
<i>headless</i>	<i>bool</i>	<i>false</i>	<i>no</i>

Table 6: Configuration options – Allowed and default values

4.3 WebDriver Configuration

For all interactions with the website SUL, Selenium's Chrome WebDriver is used. There is some website behaviour that might interfere with the learning process, so the WebDriver is automatically configured as follows by injecting JavaScript code that will run before any on-page script whenever a new webpage is loaded:

During website analysis, JavaScript is disabled.

Websites might include behaviour that automatically loads a new webpage depending on, e.g., the values of cookies. During the initial website analysis, it is necessary to analyse each webpage separately, therefore JavaScript is disabled so that no behaviour will trigger.

JavaScript is later re-enabled for the learning process, where the website's behaviour is relevant.

All alerts are suppressed.

An alert is a dialogue box with a message, a confirmation button, and optionally a dismissal button. An example is shown in Figure 21. When an alert opens, it takes the focus away from the current browser window, and makes all interactions with the website impossible until the alert box is closed [22]. It would be possible to handle alerts by adding the closing of an alert as a separate default input, but since this increases the size of the input alphabet, and alerts are not that commonly used, it was decided to prevent them from opening instead.

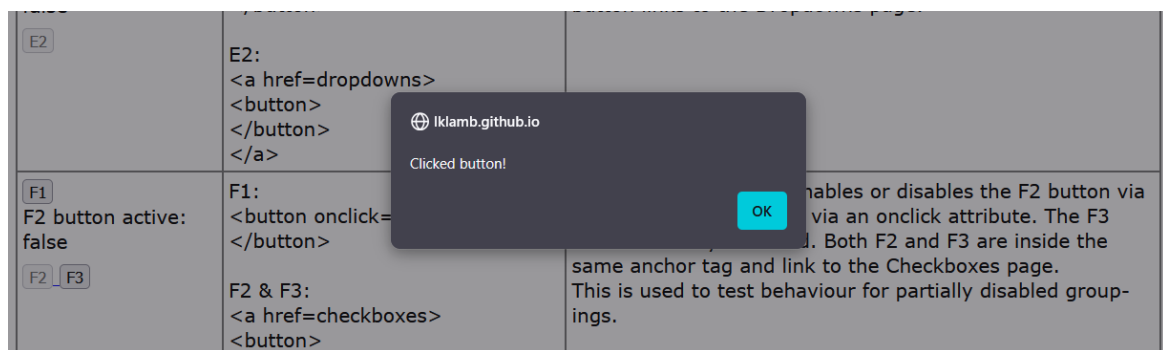


Figure 21: Example of an alert overlay

When loading a webpage, it is forced to open in the same window.

Depending on the browser settings, links might open in new tabs or windows. Tracking behaviour across multiple windows would greatly increase the complexity of the learning, so this is prevented.

The WebDriver waits for elements to be detected.

A common problem when trying to automatically interact with a webpage is whether the webpage is ready to receive this specific interaction. Varying load times lead to race conditions, where sometimes an element may be interactable at the point in time where a Selenium command is executed, and sometimes not. Selenium offers two different mechanisms for synchronization to mitigate this problem. The learning application uses the *implicit wait* strategy, which sets a global timeout that is applied to every element location call during this session. This means, that whenever the WebDriver tries to find an element and cannot locate it, it waits for the specified time frame before raising an exception. If the element is located beforehand, execution continues without further waiting [23].

If the learning application encounters non-deterministic behaviour due to, e.g., an unstable internet connection, it will crash because the learning algorithms depend on deterministic behaviour. Increasing this timeout in the learning application settings might help.

The *explicit wait* strategy is not suitable for this application, because the behaviour of the website is not known in advance, so explicitly waiting for a specific element to be located may lead to waiting indefinitely.

The wait time is set to 0.0 by default. Waiting for a page to fully load after a new URL has been activated is automatically handled by Selenium, so this wait time is only used to wait for internal changes on a page that do not change the current URL.

4.4 Preprocessing of User-Defined URLs

It is not possible to access which document has been loaded directly using Selenium, so the learning application uses URLs instead. Both the initial URL and the list of URLs that define the scope of the exploration are strings defined by the user. Selenium's WebDriver is able to load a webpage given its URL, and also read the URL that is currently displayed in the browser address bar [24].

The web server may be configured to change the displayed URL somehow, causing the displayed URL to not match the user-defined URL. For example, loading the URL `https://lklamb.github.io/Test_Website` will work without problems, but reading the current URL afterwards will return `https://lklamb.github.io/Test_Website/` with an additional forward slash at the end.

It is therefore necessary to load each user-defined URL and read the corresponding displayed URL once in a preprocessing step, so that internally the application can work exclusively with the URLs as they are displayed in the browser address bar.

Additionally, duplicate URLs are removed from the scope.

4.5 Generating the Input Alphabet

Whereas other website automata learning applications such as ALEX (*Automata Learning EXperience*) generate the input alphabet from direct user interactions with the website [4], this learning application includes an additional phase during which the webpages in the scope are automatically analysed to find interactive elements. The reason for this is the different goals: ALEX aims to provide a low threshold tool for website testing for users with little programming experience [4]. In contrast, the goal of the application developed for this thesis is to be as general as possible, so that any website may be learned with as little user-provided information as feasible.

Generating the input alphabet consists of three steps:

1. Searching for interactive HTML elements on each webpage in the URL scope
2. Creating *InputElement* objects corresponding to the found HTML elements within the learning application
3. Creating a unique input letter string for each *InputElement* object

4.5.1 Searching for Interactive HTML Elements

Selenium's WebDriver provides multiple different locator strategies, which can be used to find elements on a webpage within the DOM⁴ [25]. The website is analysed in advance of the learning process, so HTML elements that are dynamically created during the exploration will not be considered.

The first prototype of this website learning application [26], described in detail in Chapter 7, looked for HTML elements by tag name. Since the prototype was limited to buttons only, this worked very

⁴ Document Object Model

well with only a single line of code⁵. When extending the prototype to work with other element types, it quickly became clear that this approach had some drawbacks:

- Lots of HTML element types with different tag names exist [27]. Locating all types separately is a lot of effort both during coding, as well as during execution.
- A website may contain very few of these types of HTML elements. There might be a lot of unnecessary lookups trying to find elements that very rarely occur.
- Elements found using this approach may be interactive, but they also may not be. Trying to filter out superfluous elements using only tag names is not possible. For example, there might be a button⁶, an element type usually used interactively, which in reality does nothing when clicked. On the other hand, a webpage might contain a paragraph of text⁷, which is not commonly an interactive element, that triggers website behaviour when hovered over. Even though this is usually not the case, since one of the advantages of an automaton model of a system is that anomalous behaviour can easily be detected, and it is therefore very useful for testing, these unintuitive cases cannot be just assumed to not exist. This means that non-interactive elements cannot be filtered out, and all existing elements would have to be used to create possible inputs, resulting in a very large input alphabet, and consequently longer execution times.
- Elements may be contained inside other elements. For example, an anchor element might be wrapped around a button element, or vice versa.⁸ This would result in two input letters being generated, even though the elements can only be interacted with at the same time by a human user.
- HTML is very versatile and a browser most often is still able to parse a HTML document that is not well-formed. As a markup language, there is no compilation step that enforces strict rules for the HTML code. Even serious errors, such as not closing an element that has been opened⁹, might not even be visible in the browser. Therefore, elements might be structured strangely, e.g., two hyperlinks wrapped inside a single button element. Although this is not good practice, it cannot be ruled out. Trying to analyse and work with this structure might encounter lots of edge cases.

For these reasons a new approach to generate the input alphabet was implemented. The main motivator was to keep the input alphabet as small as possible. It should only include elements that are actually interactive and trigger webpage behaviour. Additionally, the new strategy covers different ways an HTML element can be interacted with. The prototype's approach assumed that a button would always be clicked, but there is a possibility that, e.g., different behaviour might be triggered when hovering over the button.

The current version looks only for elements that have attributes that are specifically meant to trigger behaviour. These elements are located using Selenium WebDriver's XPath locator strategy. Table 7 lists the attributes that the learning application looks for to find interactive elements.

Other attributes that might interactively trigger behaviour are, e.g., *oncontextmenu*, *ondblclick*, *onmousedown*, *onmousemove*, *onmouseup*, *onkeydown*, *onkeypress*, *onkeyup*, *onfocus*, *onblur*, *ondragstart*, *ondragend*, and many others [28]. The current version of the learning application will not look for these other attributes intentionally, but their behaviour might still be triggered as a side effect when trying to trigger the attributes listed in Table 7, e.g., clicking an element with an *onclick* attribute will also trigger *onmousedown*. Adding these additional attributes to the application as separate inputs in the future can easily be done using the same principles as described in this thesis, if the need arises.

⁵ `WebDriver.find_elements(By.TAG_NAME, "button")`

⁶ `<button></button>`

⁷ `<p></p>`

⁸ `<a><button></button></a>`

⁹ E.g., `<section>` without the corresponding `</section>`

<i>Attribute Type</i>	<i>XPath</i>	<i>Interaction/Behaviour</i>
<i>href</i>	<i>//*[@href and not(self::link[contains(@rel, 'stylesheet')])]</i> ¹⁰	<i>Triggers a hyperlink when clicked.</i>
<i>onclick</i>	<i>//*[@onclick]</i>	<i>Triggers specific behaviour when clicked.</i>
<i>onchange</i>	<i>//*[@onchange]</i>	<i>Triggers specific behaviour when a value associated with the element changes.</i>
<i>onmouseenter</i>	<i>//*[@mouseenter]</i>	<i>Triggers specific behaviour when the mouse pointer enters the element.</i>
<i>onmouseover</i>	<i>//*[@onmouseover]</i>	<i>Triggers specific behaviour when the mouse pointer enters the element.</i> ¹¹
<i>onmouseleave</i>	<i>//*[@onmouseleave]</i>	<i>Triggers specific behaviour when the mouse pointer leaves the element.</i>
<i>onmouseout</i>	<i>//*[@onmouseout]</i>	<i>Triggers specific behaviour when the mouse pointer leaves the element.</i> ¹²

Table 7: Attributes the learning application looks for to find interactive elements

When creating websites, using event handler attributes like these (also called inline event handlers) is more common in older or simpler code, and is considered bad practice now, as it mixes HTML and JavaScript in the same file, and reduces reusability and maintainability. Modern websites mostly use JavaScript event listeners and handlers instead, which allow for attaching behaviour to HTML elements without changing the HTML code directly¹³ [29]. For simplicity, the learning application currently only looks for the specified inline attributes to generate the input alphabet.

The search for HTML elements is done separately per attribute type, so elements that have multiple of these attributes will be found multiple times. This is intentional, as how to interact with an element to trigger the different attribute events might differ depending on the type of attribute, so these need to be treated as different inputs.

Even if both attribute types necessitate the same interaction, if elements that contain both were only tracked once, some behaviour might be lost. If they are both kept as separate inputs, this offers the possibility to analyse the priority in which their effects are triggered. E.g., when clicking an element with both an *href* and an *onclick* attribute that should both load different new webpages, one of them will take priority and be visible in the learned automaton for both inputs. Examples of elements with different attribute combinations can be found on the Test Website (see Section 6.2) [30].

The learning application is currently limited to searching for interactive elements on the root node of the webpage only. There is no support for `<frame>` and `<iframe>` at this time.

4.5.2 Creation of *InputElement* Objects

After all HTML elements that have a specific attribute on a webpage have been found, they, and recursively all their ancestor elements, are made visible on the webpage so that all their information can be accessed. During this phase, the webpage displayed in the Chrome WebDriver might look strange, with elements overlapping etc.

¹⁰ The stylesheet to be used is linked using an *href* attribute. This needs to be filtered out, as this hyperlink cannot be interacted with by the user.

¹¹ The difference between *onmouseenter* and *onmouseover* is that *onmouseenter* is only triggered once when the mouse pointer enters the element that has this attribute. *Onmouseover* is triggered whenever the mouse pointer enters the element or any of its children, so it might be triggered multiple times while moving the mouse pointer within the same element [43]. The learning application treats both these attributes the same.

¹² The difference between *onmouseleave* and *onmouseout* is the same as between *onmouseenter* and *onmouseover* [44]. The learning application treats both these attributes the same.

¹³ E.g., `element.addEventListener("click", ...)`

Then, objects of the class *InputElement* are created for each found HTML element. Depending on the type of HTML element, one or more *InputElement*s might be created. It is not possible to store the references to the found HTML elements directly, as during the learning process webpages will be reloaded, which in turn invalidates the references that Selenium keeps. Trying to access a Selenium HTML element reference that is no longer associated with an HTML element in the Document Object Model of the webpage raises a *StaleElementReferenceException*.

The internally created *InputElement* objects store all data necessary to find the corresponding HTML element again and interact with it:

- the URL to the webpage the element is on
- the XPath that was used to find all elements of this type
- an index of which number element of that type this is on this webpage
- which type of interaction should be used on this element

Additionally, the name of the element is stored if it is accessible. This is not necessary for the learning application to run and not used as a required identifier because it might not be unique or accessible, but it makes the resulting automata much easier to interpret.

Table 8 shows which *InputElement* objects might be created from HTML elements.

<i>Single HTML element with attribute</i>	<i>Nr of created objects</i>	<i>InputElement Type</i>
<i>href</i>	<i>1</i>	<i>Clickable</i>
<i>onclick</i>	<i>1</i>	<i>Clickable</i>
<i>onchange</i>	<i>varies</i>	<i>Clickable or ChangeableSelectOption(s)</i>
<i>onmouseenter</i>	<i>1</i>	<i>HoverStart</i>
<i>onmouseover</i>	<i>1</i>	<i>HoverStart</i>
<i>onmouseleave</i>	<i>1</i>	<i>HoverEnd</i>
<i>onmouseout</i>	<i>1</i>	<i>HoverEnd</i>

Table 8: *InputElement* objects created from found HTML elements

HTML elements with either an *href* or an *onclick* attribute will create a *Clickable* object, because both of those can be triggered with a click on the element.

HTML elements with an *onchange* attribute are an exception to the previously explained rule that the element tag name is not used. This is necessary because *<select>* elements, *<input>* elements of different types, and *<textarea>* elements may have an *onchange* attribute, which are all widely different elements that need to be interacted with in a different way.

A Select is a control element that opens a dropdown of options when clicked, only one of which can be selected exclusively at any point in time. Select elements are filtered out from the list of elements with an *onchange* attribute, and a separate *ChangeableSelectOption* object created for each option that can be selected. From the example of a Select element [31] shown in Figure 22, seven different *ChangeableSelectOption* objects would be created. This includes one for "--Please choose an option--", as this is also just an *<option>* element in the HTML code.

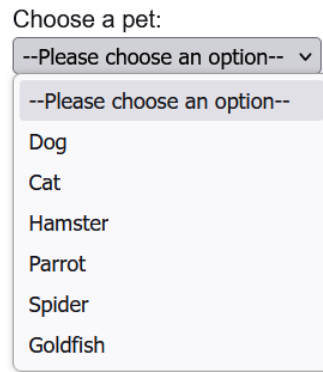


Figure 22: Example of a Select element

Common `<input>` element types that might have an *onchange* attribute are checkboxes and radio buttons. When the learning application finds an element with this attribute, it assumes that it is one of those two types of elements, if it is not a Select element, so a single *Clickable* object will be created. Other `<input>` element types or `<textarea>` elements are currently not handled separately, so they will also simply create a *Clickable* object.

HTML elements with either an *onmouseenter* or an *onmouseover* attribute will create a *HoverStart* object. HTML elements with either an *onmouseleave* or an *onmouseout* attribute will create a *HoverEnd* object.

If an element has multiple of these attributes, multiple *InputElement* objects of different or the same type might be created, because the HTML element is found in multiple searches.

4.5.3 Creation of Unique Input Letter Strings

AALpy's learning algorithms work with strings as inputs and outputs, so a conversion from *InputElement* object to a unique input letter string is necessary.

It would be possible to use opaque non-human-readable IDs here, but since AALpy uses these input letter strings also to create a visualization of the learned automaton, human readable IDs were used for easier interpretation.

An input letter string takes the following format:

*URL;Type;Index;Name;Interaction*¹⁴

If the *InputElement* object is a *ChangeableSelectOption*, the input letter string additionally contains information about the option to be selected:

*URL;Type;Index;Name;Interaction;OptionIndex;OptionName*¹⁵

This string contains all the same information as the corresponding *InputElement* object.

4.6 Reset to Initial State

During active learning and trace generation for passive learning the system is reset to its initial state multiple times. This includes multiple steps, of which (re)loading the initial URL is the final one. If the initial page were loaded without any of the preparatory steps listed below, unintended behaviour might be triggered on loading through information left over from the previous walk, such as the mouse pointer position, scroll position, or stored cookies.

¹⁴ E.g., https://lklamb.github.io/Test_Website/index.html;HREF;0;Basics;CLICK

¹⁵ E.g., https://lklamb.github.io/Test_Website/attribute_combinations.html;ONCHANGE;2;E;SELECT_OPTION;1;Go

1. Scrolling to the top left corner of the webpage that is currently loaded.

Before the first exploratory walk through the system, this is done on the last webpage that was loaded during the generation of the input alphabet. For all subsequent walks, this is done on the webpage loaded at the end of the previous walk.

2. Moving the mouse pointer to the top left corner of the page.

Resetting the mouse position is especially important if the website contains elements that trigger behaviour when the mouse hovers over them. Otherwise, the position of the mouse leftover from the previous walk might influence behaviour on the next walk, so it is essential to reset the mouse pointer before reloading the initial page. It is assumed that the top left corner of the page (coordinates 1 / 1) does not contain an element that triggers behaviour on hover.

3. Deleting all cookies.

This removes the possibility that information stored in the cookies will influence behaviour in subsequent walks. It is important to make sure that all cookies across all domains are deleted, because multiple domains might be reached during exploration. Selenium only offers functionality to delete the cookies for the domain the WebDriver is currently on, so a workaround using the Chrome DevTools Protocol (CDP) was implemented.

4. Loading the initial URL.

The webpage corresponding to the initial URL is loaded using Selenium's WebDriver [32].

4.7 HTML Elements and Interactions

While learning the SUL, the learning application tries different sequences of the letters in the input alphabet as inputs on the system. Each input letter corresponds to an interaction with the system. When a specific input letter is used, first the following cases are checked and handled:

- the system is already in the sink state
- the system is currently on a page that is not inside the defined URL scope
- the system is currently on a page without interactive elements
- the element to interact with is not on the current page

Only if none of these apply will the learning application try to interact with the SUL. A standard interaction consists of two main steps, which are described in the following two sections.

4.7.1 Getting a Selenium Reference to the Element

As already mentioned, it is not possible to store Selenium references for the whole learning process, as they are invalidated when a new page is loaded. Therefore, the process that was originally used to generate the input alphabet (see Section 4.5) needs to be repeated whenever an interaction should happen. First, all HTML elements on the current page of the same type as the element to interact with are found using Selenium's XPath locator strategy. Then, the correct element is accessed using the index that was stored during the input alphabet generation.

If the element contains an attribute that would open a link in a new browser tab¹⁶, this attribute is removed. This step combined with the script that is executed every time a new page loads (see Section 4.1) forces all links to open in the same tab in the same window, which is a design choice to simplify the learning process.

Then, the element is scrolled completely into view to make sure it can be interacted with.

¹⁶ `target=blank_`

4.7.2 Interacting with the Element

The learning application is currently able to handle four different types of interactions with the SUL, which correspond to the four different types of *InputElement* objects that were possibly created during input alphabet generation:

- *Clickable*
- *ChangeableSelectOption*
- *HoverStart*
- *HoverEnd*

When simulating these interactions via Selenium, care was taken to be as close to a real user interaction as possible. Selenium provides functionality to trigger certain behaviour directly, but using this would not portray user interactions accurately, and generate misleading results, which is described in more detail in the following sections.

The learning application currently simulates standard interactions with the website using a mouse and does not take keyboard interactions (e.g., using the *tab* key to navigate) into account. All simulated interactions take place at the centre of the interactable element.

If the element is disabled, the interaction takes place like normal, but does not trigger any behaviour. This is the standard way Selenium handles disabled elements. No specific output for a non-interactable element (*NotInteractable* or *GoSink*) is returned. Instead, the output corresponds to the URL before the interaction, creating a self-loop.

If the element is hidden, the interaction is aborted, because a hidden element cannot be interacted with either by a human user or via Selenium. Selenium throws an *ElementNotInteractableException* and a specific output for a non-interactable element (*NotInteractable* or *GoSink*) is returned.

Selenium's interaction commands by default block until the page is fully loaded before executing [33]. For page internal behaviour, the specified implicit wait time is used to determine the cutoff point for checking if an element can currently be interacted with or not.

4.7.2.1 Clickable Element

When a human user tries to interact with a *Clickable* element, they must first move the mouse pointer to somewhere above the element, and then click the left mouse button. Even this simple interaction offers three possible ways of simulating it:

Method 1

```
element.click()
```

Method 2

```
ActionChains(driver).move_to_element(element).click().perform()
```

Method 3

```
ActionChains(driver).move_to_element(element).perform()  
element.click()
```

If the goal was just to automate an interaction with a certain element on a website, and the element and all its attributes were known, all three methods would work perfectly well. Probably, Method 1 would be chosen, as Selenium's *click()* already contains all additional checks and steps that are needed to click this element [34].

The goal of the learning application on the other hand is to learn the behaviour that would present itself if a user, who does not know anything about the HTML structure “behind the scenes”, interacted with the element. For example, Method 1 will not trigger behaviour that is dependent on the mouse pointer entering the element, which a human user will trigger every time. Method 2 triggers

this behaviour, but sometimes does not register the click correctly afterwards, leading to a self-loop in the learned automaton that should not exist. This non-deterministic behaviour is incompatible with the learning algorithm which depends on deterministic results. The suspicion is that moving the mouse pointer and clicking happens so quickly after each other in this method, that sometimes scripts on the website do not have time to complete.

Method 3 showed the same behaviour every time, including any side effects caused by the mouse pointer entering the element, therefore this method was chosen to separate moving the mouse pointer and clicking the element into two different steps.

4.7.2.2 *ChangeableSelectOption* Element

If the input letter is a *ChangeableSelectOption*, the interaction should select the specified option on the Select element. This might be a different or the same option as the one that is currently selected. The same difficulties as with a *Clickable* element occur here in a more complex form. A human user would move the mouse pointer to the Select element, click it to open the dropdown of options, move the mouse pointer to the specified option, and then click it to select it.

Trying to simulate this sequence of four steps exactly is not possible using Selenium, because browsers render Selects and their options using native widgets, and not standard HTML elements. As a result, the options are not exposed as standard clickable DOM elements, instead the browser controls their user interaction natively [35] [36]. Trying to click an option using Selenium will raise an exception, no matter if the dropdown was previously opened or not.

As a compromise, this interaction is simulated by moving the mouse pointer to the Select element, clicking the Select element, and then changing the selected option using Selenium's Select class. This does not click the option itself, but instead updates the selected option directly at the DOM level. Some website behaviour might be missed due to this discrepancy between a real user interaction and its simulation, but no related problems were noticed during the course of this thesis project.

4.7.2.3 *HoverStart* Element

A *HoverStart* interaction is quite easily simulated. A human user would move the mouse pointer over the element to interact with, and this is exactly what is done in the simulation.

4.7.2.4 *HoverEnd* Element

Simulating a *HoverEnd* interaction is slightly more involved. If a human user wanted to stop hovering over an element, they would move the mouse pointer outside the boundaries of this element. "Outside the element" is usually quite a large area, and the details of this interaction are not clearly defined. As the structure of the webpage is unknown, moving the mouse pointer in a certain direction might trigger different behaviour than in another one, e.g., due to entering a different neighbouring element. To get reproducible results, in the simulation the mouse pointer is not simply moved outside the element. Instead, the page is scrolled to the top left as far as possible, and the mouse pointer moved to the top left corner of the page, same as during a reset (see Section 4.6). The learning application works under the assumption that this position will not contain an element that triggers behaviour on hover.

5 Challenges

This chapter describes some challenging issues that occurred during the course of this thesis, and how they were solved.

5.1 Interaction Speed

Interactions with the website SUL are very slow when compared to the rest of the learning process, which is a common bottleneck in active automata learning. This also affects the trace generation for passive learning, in which the SUL is actively queried. Depending on the number of interactions necessary to learn the behaviour of a specific website SUL, this leads to very long execution times, even for relatively simple website systems.

Operating System	<i>Microsoft Windows 10 Home Version 10.0.19045 Build 19045</i>
CPU Model	<i>Intel(R) Core(TM) i7-7700HQ CPU @ 2.80GHz</i>
CPU Count	<i>8</i>
Download Speed	<i>~45 mbit/s</i>
Upload Speed	<i>~10 mbit/s</i>
Ping	<i>~25 ms</i>

Table 9: Interaction Speed Test – Device specifications and network speed

Performance testing on the running example under the conditions from Table 9 showed that a single valid interaction that causes a new page to load takes about 0.4 seconds, whereas a reset of the SUL takes about 0.35 seconds. Hosting the running example SUL locally instead of accessing it via the internet, or running the browser in headless mode, did not reduce the interaction and reset time significantly. The running example website has a very basic structure. It is assumed that interactions and resets on websites with more elements (e.g., images) take even longer.

Although for the first prototype (see Chapter 7) experiments were run exclusively with direct interactions with the website SUL, this quickly proved impossible for more extensive testing due to hardware and time limitations.

Therefore, a mechanism was implemented in the learning application to use an automaton file in *.dot* format as a replacement representation of the website SUL. When an automaton of a website SUL has been learned with the learning application, such an automaton file is automatically created. If, as for the case study websites, it is possible to confirm that the learned automaton is correct and contains all behaviour that is part of the website SUL, the automaton file can then be used as a replacement SUL for further executions of the learning application on the same website SUL with a different learning type, different type of input enabled handling, and/or different learning parameters. To use an automaton file as a replacement SUL, the additional options listed in Table 10 need to be set in the configuration file.

<i>Name</i>	<i>Allowed Values</i>	<i>Default</i>	<i>Mandatory</i>
<i>system_source</i>	<i>FROM_BROWSER</i> <i>FROM_AUTOMATON_FILE</i>	<i>FROM_BROWSER</i>	<i>no</i>
<i>automaton_file_path</i>	<i>absolute file path as string</i>	<i>-</i>	<i>yes, if system_source is automaton file</i>

Table 10: Additional configuration options for the system source

The website analysis, which is used to generate the input alphabet in advance of the learning process, only uses a limited number of interactions. It therefore always takes place on the actual website SUL,

no matter the *system_source*. The automaton file needs to match this analysis, so it needs to have been created using the same *initial_url* and *urls_in_scope* (or the same *website_to_learn* demo or example).

Execution of interactions on the automaton loaded from a file are much faster. Using this method, much more extensive experiments were possible. As information about execution times is not comparable anymore if using this method, instead of using this metric as was done with the prototype, the number of necessary interactions and resets was tracked for the evaluation of different configurations (see Chapter 8).

5.2 Simulating User Interactions

When interacting with the website SUL via Selenium, it is necessary to simulate real user interactions as closely as possible. Selenium provides functionality to trigger certain behaviour directly, but this would not portray user interactions accurately, and generate misleading results. It was challenging to find the right sequence of Selenium commands to get realistic results.

Specifics of how user interactions were simulated can be found in Section 4.7.2.

5.3 Choice of Initial URL

By choosing an initial URL without specifying which page to load (e.g., https://lklamb.github.io/Test_Website/), the web server will deliver the default page of the website. Which page is loaded depends on the configuration of the web server, but most commonly it will look for an *index.html* file (e.g., https://lklamb.github.io/Test_Website/index.html), which usually corresponds to the website's home page [37]. At first glance, this seems like a good approach to find a well-suited starting point for the exploration of a website.

To improve search engine optimization and usability, the web server may be configured to suppress the file name and extension in the URL displayed in the browser address bar [38]. This behaviour may lead to some problems during the learning process, as Selenium is only able to access the URL that is currently being displayed [24]. If the same page is encountered once with file name and extension displayed in the browser address bar, and once with it suppressed, the learning algorithm will treat this as encountering two different pages, creating duplicate states. As this behaviour is completely dependent on the web server and the structure of the website, it cannot be predicted and mitigated by the learning application. The initial URL must therefore be carefully selected to avoid this duplication of states if possible, or otherwise this phenomenon taken into account when analysing the learned automaton.

This behaviour can, for example, be observed on the Test Website described in Section 6.2 [30]. Figure 23 shows that even though the same HTML document is loaded by the web server, the learning application is confused by the mismatch between displayed URL and loaded HTML document.

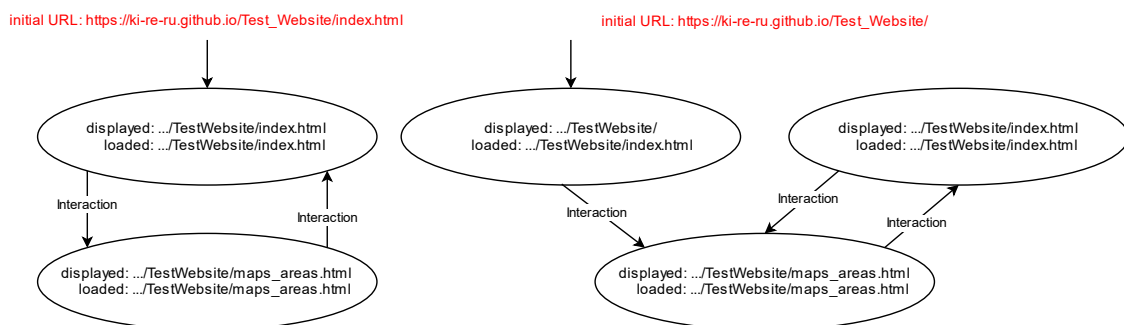


Figure 23: Different partial automata resulting from a different choice of initial URL

5.4 Output of Interactions in Edge States

Pages at the boundary of the scope or without any interactive elements need to be distinguished by either *Boundary* or *DeadEnd* as an output. Otherwise, these two cases would default to returning *NotOnCurrentPage* (for self-loop input enabled handling) or *GoSink* (for sink state input enabled handling) for all inputs.

As per definition in a Mealy machine, the output depends on the current input as well as the current state, it would be possible to have two states that have identical output behaviour for all inputs [39], but both the passive and the active learning process merge states when their future behaviour cannot be distinguished [15] [19].

The example website SUL in Figure 24 is used to illustrate this problem. It consists of four pages. Page 1 contains the three buttons A, B, and C, leading to Page 2, Page 3, and Page 4 respectively. Page 3 contains button D, which leads to Page 1. Page 2 and Page 4 are empty. Since only Page 1 and Page 2 are defined to be in the scope, only A, B, and C are in the input alphabet. This example can be found under https://lklamb.github.io/Test_Website/output-example_page1.html, and loaded in the learning application under the name *OUTPUT_EXAMPLE*.

If pages that are at the boundary of the scope and pages without interactive elements are not handled with separate outputs, all these webpages are merged into a single state (see Figure 25 *VI_loop* and Figure 26 *VI_sink*).

These two cases also need to include the current URL. Otherwise, all webpages on the boundary of the scope would merge into one state, because they would return the same output *Boundary* for all inputs (see Figure 27). The same is true for *DeadEnd*.

If the output for these edge cases includes the current URL, unintended merging of states is impossible (see Figure 28). This is the configuration used by the learning application.

With separate outputs, these two cases will always create self-loops, even if the input enabled handling is configured to use a sink state. This means that *V2* and *V3* of the learned automaton look the same when using a sink state as when using self-loops (Figure 27, Figure 28).

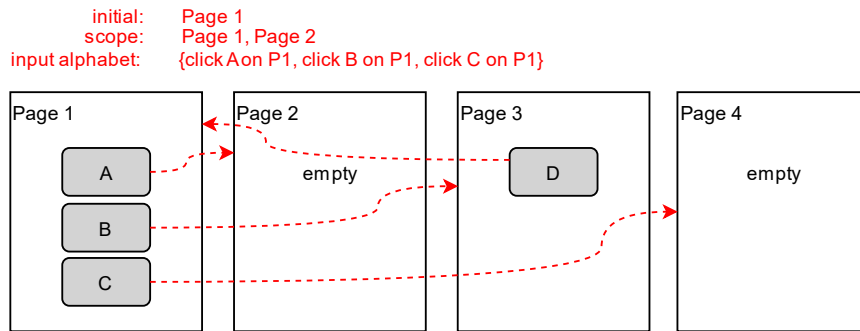


Figure 24: Output Example – Schematic of website SUL

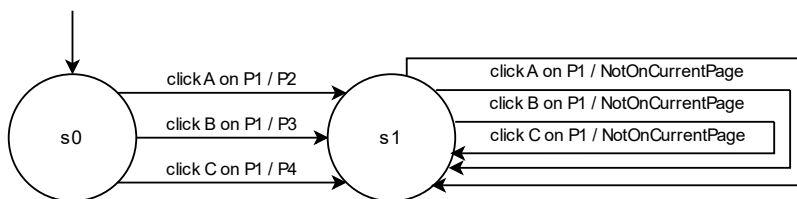


Figure 25: Output Example – Self-loop automaton *VI_loop* resulting if pages that are at the boundary of the scope and pages without interactive elements are not handled separately

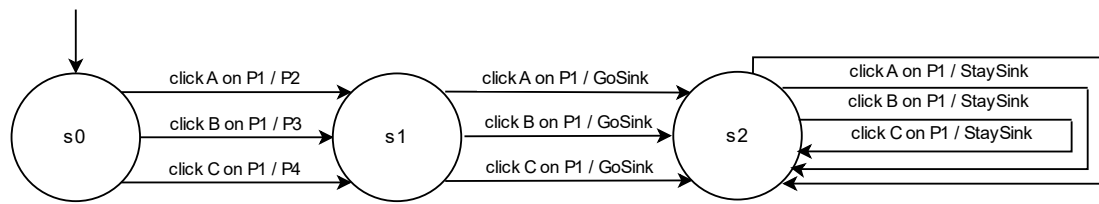


Figure 26: Output Example – Sink state automaton $V1_sink$ resulting if pages that are at the boundary of the scope and pages without interactive elements are not handled separately

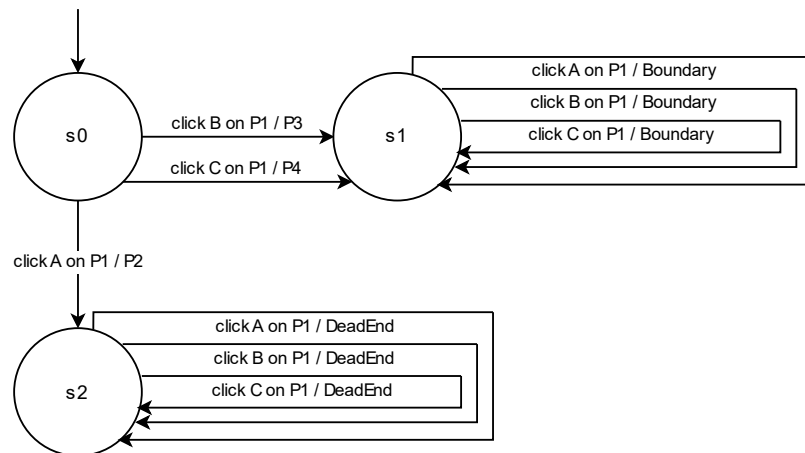


Figure 27: Output Example – Automaton $V2$ resulting if pages that are at the boundary of the scope and pages without interactive elements are handled separately, but their output does not include the current URL

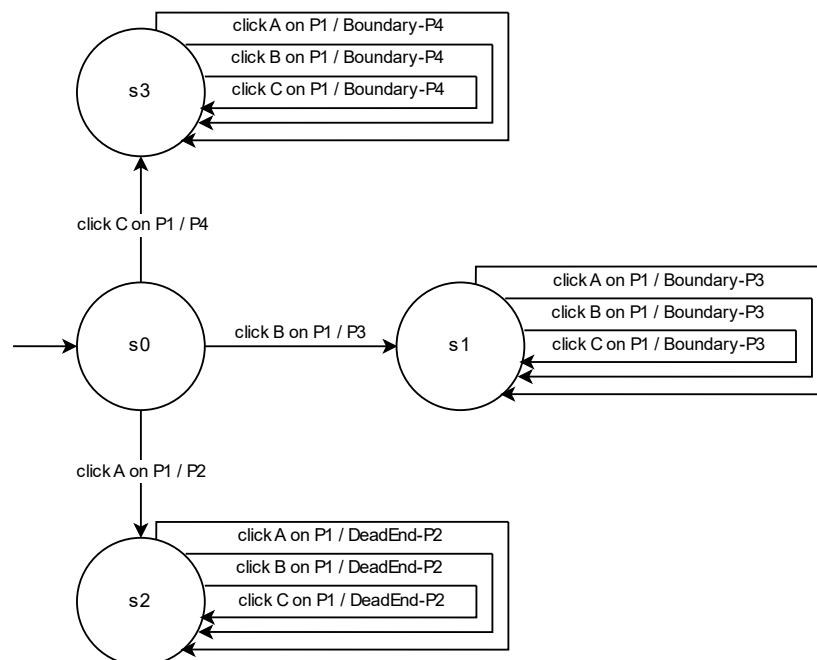


Figure 28: Output Example – Automaton $V3$ resulting if pages that are at the boundary of the scope and pages without interactive elements are handled separately and their output includes the current URL

5.5 Balancing Information Depth and Human Interpretability

As mentioned in the previous section, the choice of output alphabet affects the resulting learned automaton. Using just the current URL as an output, versus using additional output strings for edge cases, versus using additional output strings that also include the current URL will result in a different number of states being found, as well as transitions being labelled differently (compare Section 5.4).

Having too little information in the output string will cause states to merge that make the automaton's behaviour hard to interpret for a human. Having too much information present will increase the length of displayed transition labels, causing overlap and increasing the size of a visualized automaton, which makes readability an issue.

It was attempted to find a balance between these two by using edge case specific outputs including the current URL for some cases (*Boundary*, *DeadEnd*), but without the current URL for others (*InteractionIntercepted*, *NotInteractable*).

5.6 Mouse Reset on *HoverEnd* Interactions

When a *HoverEnd* interaction should be executed on the SUL, there are two different possibilities how the mouse reset can be handled.

The example website SUL in Figure 29 is used to illustrate this. It consists of three pages. Page 1 contains a text H1 and a text H2. Hovering over H1 makes B1 visible. Clicking B1 loads Page 2. Hovering over H2 makes B2 visible. Clicking B2 loads Page 3. All elements are placed in a fixed-layout table, so that the changing visibility will not affect the position of the elements. This example can be found under https://lklamb.github.io/Test_Website/hover-example_page1.html, and loaded in the learning application under the name *HOVER_EXAMPLE*.

In the basic version *V1_hover_basic* without hover detection the mouse pointer is reset every time a *HoverEnd* interaction occurs (see Figure 30), no matter where the mouse was previously. In the more advanced version *V2_hover_advanced* (see Figure 31), the learning application first detects if the mouse pointer is currently hovering above the specific element the *HoverEnd* interaction should be applied to, and then only resets the mouse pointer if so. If not, a self-loop will be created in the learned automaton. *V2_hover_advanced* is currently used by the learning application.

In the figures, page specifiers have been omitted on the inputs, because all input elements are on Page 1.

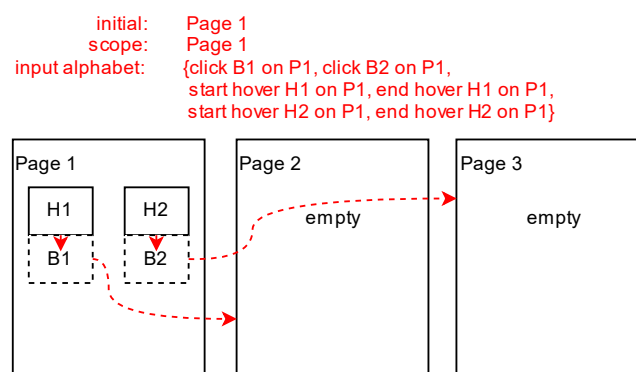


Figure 29: Hover Example – Schematic of website SUL

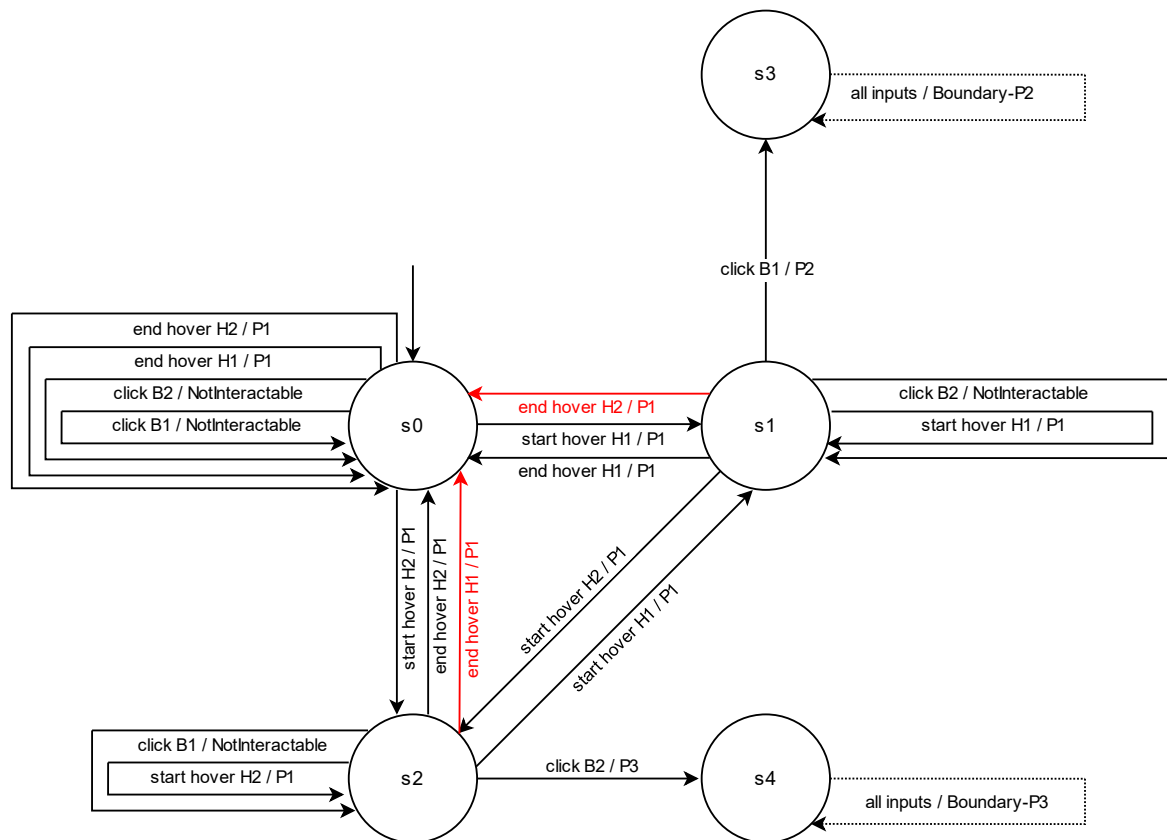


Figure 30: Hover Example – Self-loop automaton *V1_hover_basic* without hover detection

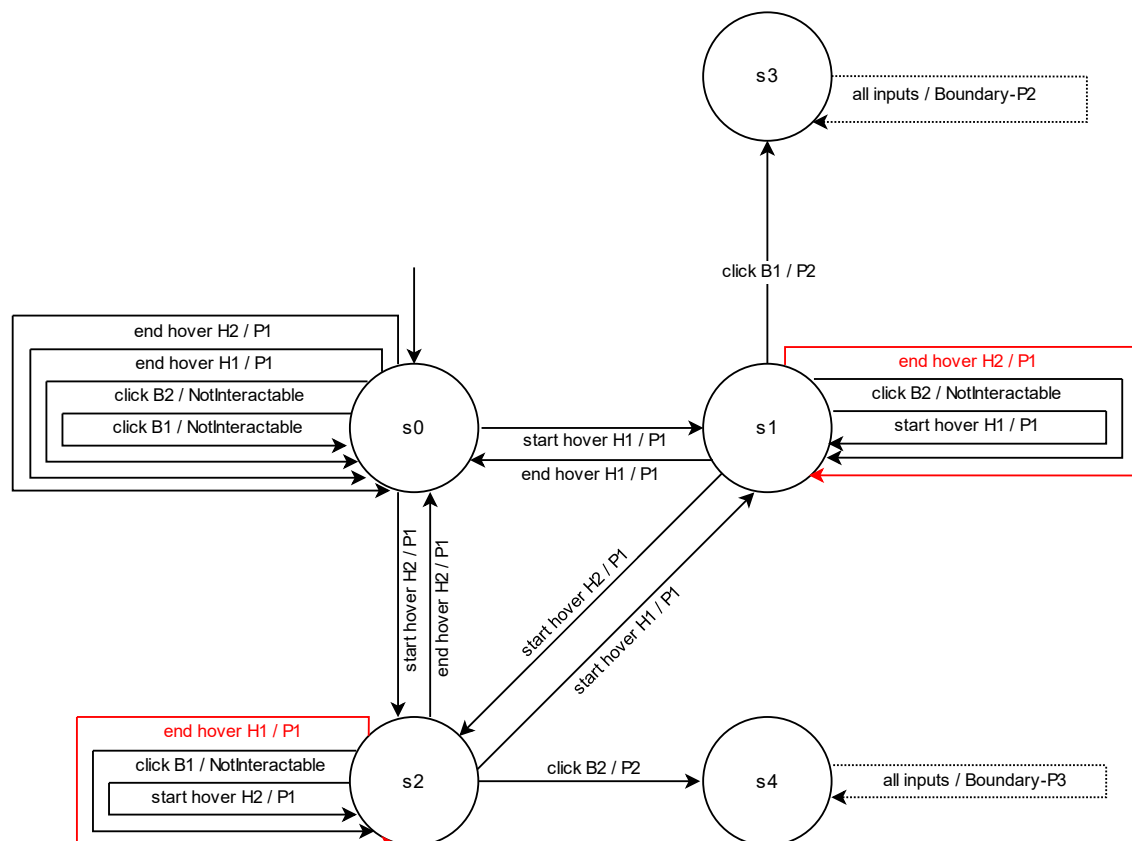


Figure 31: Hover Example – Self-loop automaton *V2_hover_advanced* including hover detection

5.7 Mouse Pointer Tracking

Handling hover interactions requires that the position of the mouse pointer be tracked during the learning process. Selenium interacts with the SUL at a browser level. It does not expose the current position of its “virtual” mouse pointer via its API, so this needs to be tracked manually so it can be used in the process of hover detection. Therefore, whenever an interaction with an element takes place, the centre position of the element is calculated and stored as the current position of the virtual mouse pointer. Whenever the virtual mouse pointer is reset, either during the reset of the SUL or during a *HoverEnd* interaction, this is also tracked. Coordinates are tracked relative to the viewport to closely match user interactions, as a human user also does not automatically move the mouse pointer when, e.g., scrolling up or down.

If an interaction contains multiple steps (e.g., move mouse pointer to element, then click element), the mouse position tracking takes place directly after the movement, so that even if the actual interaction fails, it is always up-to-date. If the learning application tries an interaction that is not possible at all in the current state (e.g., the element is not on the current page) the virtual mouse pointer is not moved and the tracker not updated.

Attempts were made to use PyAutoGUI [40] for mouse movements, which has a much more straightforward API due to its more limited functionality, but since this uses the actual system mouse, a combination with Selenium is not possible. The whole learning application would have to be adapted to use PyAutoGUI exclusively instead.

5.8 Mouse Pointer Visualisation

It is not possible to display the current position of the virtual mouse pointer via Selenium. To make analysing the mouse movements easier, the learning application is able to visualise the mouse pointer as a red dot using a JavaScript script that inserts a new custom `<mouse-pointer>` element on the current webpage. This script must be run every time a new page is loaded. Because it is not known in advance whether an interaction will trigger the loading of a new page, a check whether the mouse pointer element already exists on the page must be run after every interaction and reset. This introduces additional overhead for the execution, so the mouse pointer visualisation is turned off by default and can be turned on via the configuration file. During testing, none of the results indicated that the artificially inserted element interferes with the learning process, but it is recommended to leave the visualisation turned off if it is not necessary for debugging or testing.

Additionally, the visualisation reacts to both Selenium’s virtual mouse pointer and the real system mouse pointer, so moving the real mouse pointer across the WebDriver’s browser window will lead to unpredictable results. The real mouse pointer should be kept away from the browser window regardless to not trigger additional inputs.

5.9 Possible Issues During the Interaction Sequence

Due to the high variability of HTML code and website interactions, a variety of issues might occur when trying to interact with an HTML element. This section describes the issues that were found during the implementation of the learning application and how they were handled, if so.

5.9.1 Non-Determinism Caused by Variable Internet Speed

Variable internet speed might cause non-deterministic behaviour, causing the learning algorithm to crash. This can possibly be mitigated by increasing the implicit wait time in the settings (see also Section 4.1), but this has the drawback of longer execution times, since the application will wait for elements to be located, even if these elements are, e.g., never displayed in the current state.

5.9.2 Interaction with Covered Elements

An element might be interactable, but partially or completely covered by another element. As all simulated interactions take place at the centre of the element, this is the location that will be evaluated, whereas a real user interaction might take place anywhere on the element. If the centre of the element to interact with is covered, *InteractionIntercepted* or *GoSink* will be returned as an output depending on the input enabled handling setting (see also Section 3.3.4). Different shapes and bounding boxes make it hard to check for coverage manually, so a more discerning evaluation was decided against.

5.9.3 Disabled and Enabled Elements Wrapped in Enabled Elements

If multiple elements are wrapped inside another element, and only some of the child elements are enabled¹⁷, this might lead to unexpected behaviour. During some tests, a hyperlink in the parent element could be clicked and would trigger like normal, but this has not been analysed in detail.

5.9.4 Automatic Scrolling Triggers Hover

Elements are automatically scrolled into view before the learning application tries to interact with them. This scrolling might trigger hover behaviour if an element is moved to below the current mouse pointer position. This is currently not handled specifically and might cause undefined behaviour.

5.9.5 *HoverEnd* without *HoverStart*

Elements that only have a *HoverEnd* related attribute, but do not have a *HoverStart* related attribute, will never be hovered over by the learning application. This means that consequently, a *HoverEnd* interaction tried by the learning application will also never trigger any behaviour, no matter the state of the SUL. A possible way to resolve this would be to add an artificial *HoverStart* input if this situation occurs. In the interest of keeping the input alphabet small, this is currently not implemented, so an element with only a *HoverEnd* related attribute will only trigger behaviour if by chance the page loads with the element directly placed beneath the mouse pointer.

5.9.6 Elements Change Position

Elements might change size, shape or visibility due to interactions. This might affect the position of other elements, leading to synchronization problems with the mouse movement.

The example website SUL in Figure 32 is used to illustrate this problem. It consists of three pages. Page 1 contains a text H1 and a text H2. Hovering over H1 makes a button B1 visible. Clicking B1 loads Page 2. Hovering over H2 makes a button B2 visible. Clicking B2 loads Page 3. The difference to the SUL shown in Figure 29 is that the elements are not placed in a fixed-layout table, as well as their position and size. This example can be loaded in the learning application under the name *MOVING_ELEMENTS_EXAMPLE*, and found under https://lklamb.github.io/Test_Website/moving-elements-example_page1.html.

Figure 33 shows how the visibility of B1 and B2, and consequently the position of H2, depends on the position of the mouse pointer. Figure 34 shows the sequence of transitioning from hovering above H1 to hovering above H2. B1 is wrapped inside H1 in the HTML code, so nothing changes as long as the mouse pointer is above B1. As soon as the mouse pointer leaves B1, B1 is hidden, causing H2 to change position. Contrary to a human user, the simulation cannot adapt to this, and the mouse pointer does not end its path above H2.

¹⁷ E.g., `<a href><button disabled></button><button></button></a>`

Figure 35 shows the resulting learned automaton, assuming self-loop input enabled handling. Starting a hover above H2 after hovering above H1 does not lead to B2 being interactable, which was the expected result (compare Figure 29).

In the figures, page specifiers have been omitted on the inputs, because all input elements are on Page 1.

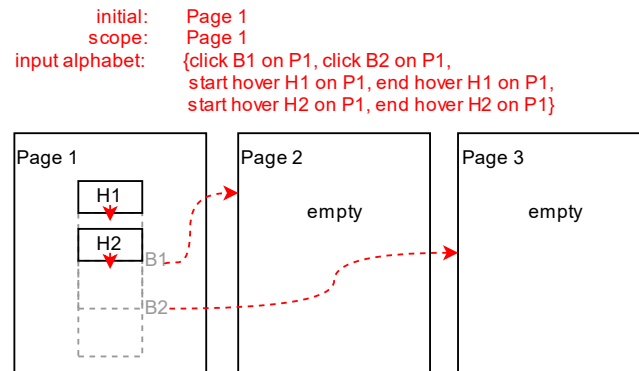


Figure 32: Moving Elements Example – Schematic of website SUL

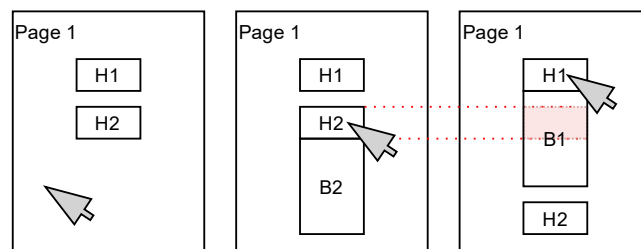


Figure 33: Moving Elements Example – Three different states of Page 1

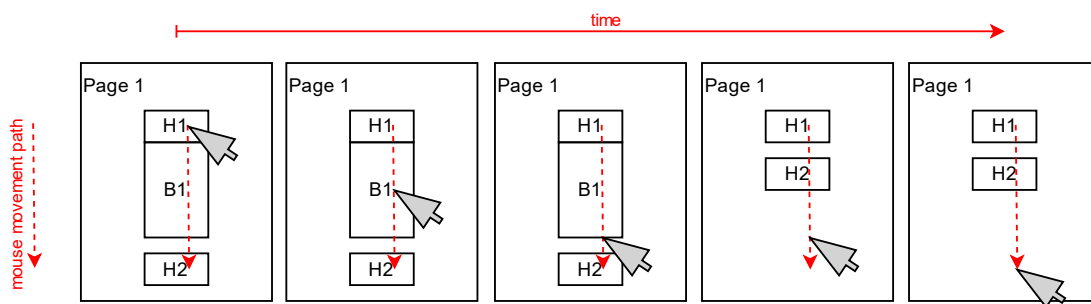


Figure 34: Moving Elements Example – Sequence of transitioning from hovering above H1 to hovering above H2

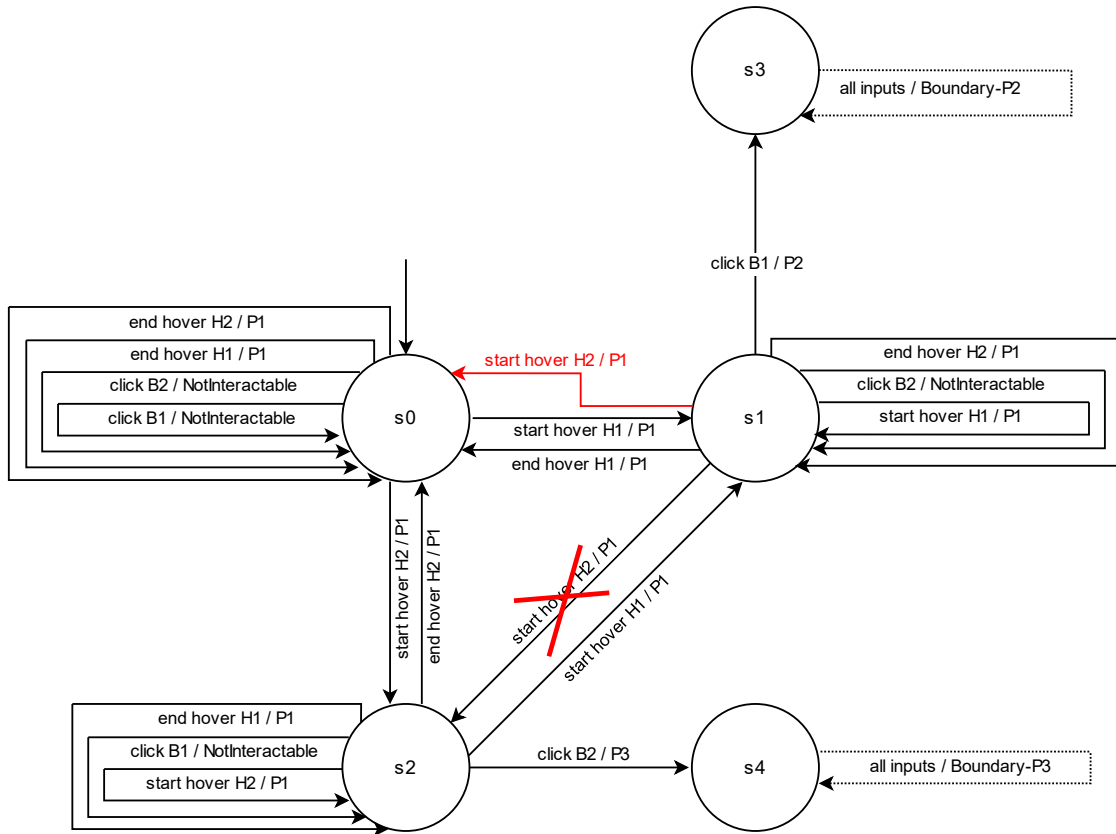


Figure 35: Moving Elements Example – Learned self-loop automaton

5.9.7 Bounding Box Approximation

Some elements, e.g., an `<area>`, do not have a fixed width or height, so their bounding box cannot be calculated automatically using Selenium functionality. *HoverEnd* interactions use the bounding box to determine whether the element is currently being hovered over. For `<area>`s an approximation of the bounding box calculation has been implemented manually in the learning application, but hover detection might fail for other similar elements, leading to *HoverEnd* interactions never triggering behaviour.

5.9.8 User Interference

Selenium does not access the actual operating system's input devices, but instead operates at the browser level, simulating user interactions without interfering with the system's mouse pointer. This allows for running the learning application non-intrusively in the background without disrupting the user's activities [41] [42].

The WebDriver's browser window is not protected from "real" user interactions while the learning application runs, so moving the mouse pointer across this window or interacting with the displayed elements is possible. This might interfere with the learning process if behaviour is triggered, so care must be taken to not unintentionally create additional inputs to the website SUL.

6 Case Study Websites

The learning application and the different automata learning configurations were evaluated on two different websites, which are described in this chapter.

6.1 Car Alarm System Demo Website

This demo website is based on the functionality of a car alarm system. It was implemented using HTML, CSS and JavaScript and hosted via GitHub pages to make it accessible via URL¹⁸ to the automata learning application. The website code can be found in a public GitHub repository¹⁹. Different branches contain different final versions of the website, with full or limited functionality.

The Car Alarm System demo website is based on the state machine shown in Figure 36.

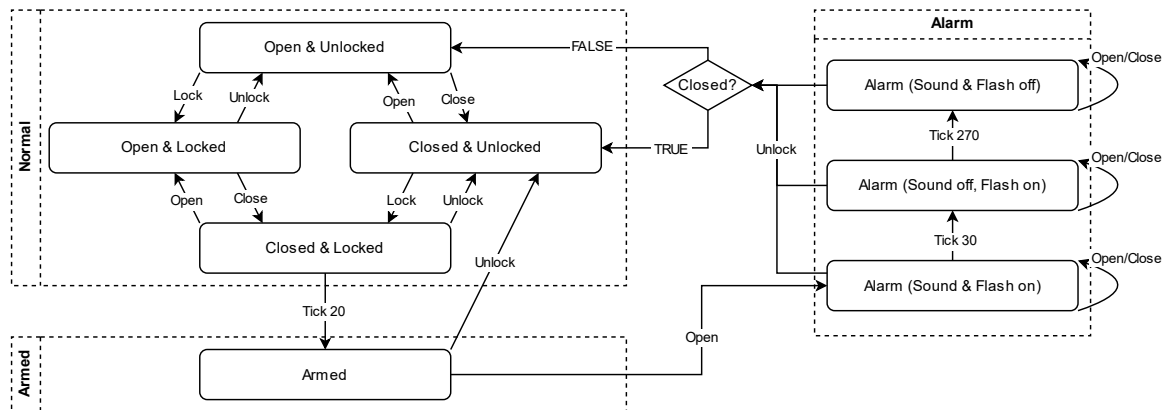


Figure 36: State machine of the Car Alarm System demo website

For simplicity, Figure 36 does not show all the states in the group of states Normal. There are additional states for each combination of doors being open and closed and the car being locked or unlocked. In the figure, “Closed” is a placeholder for all doors being closed. “Open” is a placeholder for at least one door being open. Additionally, intermediate states that are not displayed exist for each “Tick” transition.

The demo website consists of three different connected webpages to model the major groups of states of the system:

- Normal
- Armed
- Alarm

Each webpage contains six buttons with an *onclick* attribute, which trigger the same six inputs on the car alarm system no matter which webpage it is currently on. Internally in the automata learning application, clicking each button on a separate webpage will be handled as a different letter of the input alphabet.

Additionally, another webpage under a different domain can be reached from a hyperlink at the bottom of each of these three pages. This webpage contains the sources of the images used.

¹⁸ https://lklamb.github.io/CarAlarmSystem_Demo [46]

¹⁹ https://github.com/lklamb/CarAlarmSystem_Demo [45]

The possible interactions on each page are therefore:

- Click button “Open/Close front right door”
- Click button “Open/Close front left door”
- Click button “Open/Close back right door”
- Click button “Open/Close back left door”
- Click button “Lock/Unlock car”
- Click button “Tick 10”
- Click hyperlink to sources page

The “Open/Close” and “Lock/Unlock” buttons were implemented as toggles, so that clicking them changes the relevant part of the system to the opposite status that it is currently in.

Clicking the “Tick 10” button advances an internal time value by 10 seconds. Incrementing the time value in chunks was a design choice to reduce the number of steps that need to be taken to affect the system in an observable way and therefore increase learning efficiency.

The website also contains some embedded images and text that display the internal state of the system, and are updated after every interaction. These elements are only used to make the system state easily visible to the user and are never accessed by the automata learning application.

Car Alarm System - Normal



Symbols by NAS from [Noun Project](#) (CC BY 3.0)

Figure 37: Frontend of the Car Alarm System demo website (4-door version)

After every button click, the values of all relevant internal variables (e.g., door status, lock status, timers) are stored as website cookies²⁰.

As shown in the above state machine, when certain conditions are met (e.g., time limit reached in state Normal while all doors are closed and the car is locked, opening a door in state Armed, unlocking the car while in state Alarm), the system changes to a different state. These conditions are checked after every button click. The transitions are implemented in the demo website as loading the URL of

²⁰ Since the state of the website is stored as cookies, when manually exploring the demo website the cookies need to be deleted and the initial URL reloaded to reset the website to its initial state.

the target state/webpage and loading the cookies that have been stored after the last button click, as well as updating the display elements.

Multiple versions of this demo website were implemented:

- 4-door version (as shown in Figure 37)
- 2-door version (with the back door buttons disabled, but still present)
- 1-door version (with all door buttons except the front right door disabled, but still present)
- reduced version (with only one door button present per page, see Figure 38)

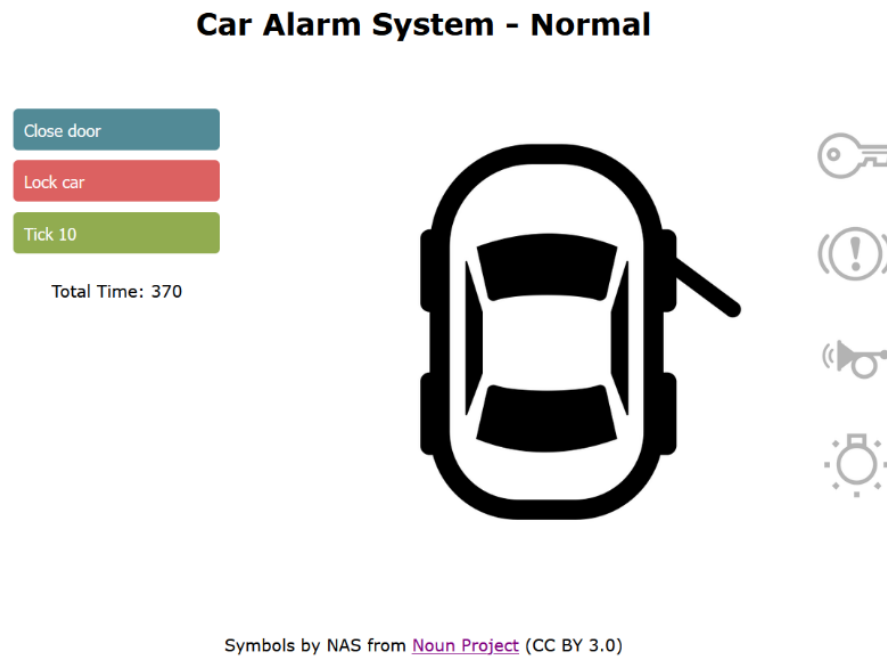


Figure 38: Frontend of the Car Alarm System demo website (reduced version)

The Car Alarm System demo website also contains some unobservable states. The difference between being in an Alarm state with both sound and flash on, with only flash on, or with both sound and flash off is not visible to the learning algorithm, because it does not change the URL or transitions between different URLs in any way. This means that it does not affect the output that is used to distinguish between different states of the website at all.

6.2 Test Website

This demo website contains different HTML elements in various configurations. It was mainly used to test the learning application during implementation, therefore there is no underlying logic to its structure. Instead, its pages and elements can be used as proof-of-concept for how the learning application handles different website structures.

The Test Website was also implemented using HTML, CSS and JavaScript and hosted via GitHub pages to make it accessible via URL²¹ to the automata learning application. The website code can also be found in a public GitHub repository²².

The main Test Website consists of nine pages:

²¹ https://lklamb.github.io/Test_Website/ [30]

²² https://github.com/lklamb/Test_Website [47]

Start Page

The Start Page contains hyperlinks to all other pages. An alert box opens when this page is loaded.

Basics: Hyperlinks & Buttons

This page contains hyperlinks (anchor elements with *href* attributes) and buttons (with and without *onclick* attributes) in different configurations, including different inner/outer element orders, disabled elements that are enabled via script, elements that open an alert when interacted with, etc.

Map & Areas

This page contains a `<map>` element with three circular `<area>` elements. The different area elements, as well as the space inside the map outside the areas link to different pages. One of the area elements is completely covered by another.

Dropdowns

This page contains dropdown menus that can be expanded by either clicking (*onclick*) or hovering (*onmouseenter/onmouseover/onmouseleave/onmouseout*).

Checkboxes

This page contains checkboxes that enable a button when checked, either through an *onclick* or an *onchange* attribute.

Selections & Choices

This page contains `<select>` elements and radio buttons, that enable a button depending on the choice that is made. Behaviour is triggered via the *onchange* attribute.

Attribute Combinations

This page contains HTML elements with multiple attributes. Each element contains a different combination of two attributes that will cause the element to be found during input alphabet generation.

Tabs & Windows

This page contains hyperlinks that try to open in a new tab or window.

Empty Page

This page does not contain any interactive elements.

Additionally, some of the interactive elements link to an example webpage under a different domain, which is therefore also connected to the Test Website system.

The examples used earlier in this thesis (see Figure 24, Figure 29, and Figure 32) are also hosted under the domain of the Test Website, but are not reachable from any of the main pages. Since they are not connected, they will not influence the learning of the Test Website SUL as long as none of their pages are used as an initial URL. To keep the input alphabet small, they should also not be included in the URL scope.

7 First Prototype

To evaluate the feasibility of the thesis project, a first prototype of the learning application with limited functionality was implemented in advance. This prototype [26] was implemented for active learning using the L* algorithm only. Results from the evaluation of the prototype were used to improve the learning application to its current version.

7.1 Configuration

The prototype was configured as follows:

Tools

AALpy 1.4.0 and the Chrome WebDriver from Selenium 4.17.2 for Python were used for the prototype.

Automaton Type

The prototype learned a deterministic Mealy machine, same as the final iteration of the learning application described in this thesis.

Learning Algorithm

The prototype used AALpy's implementation of the L* algorithm.

Equivalence Oracles

For the prototype, experiments were run with the State Prefix equivalence oracle and the Random Walk equivalence oracle with different parameter values. Results for each configuration vary depending on the configuration of the SUL, and will be described further in Section 7.2.

WebDriver Configuration

The only WebDriver configuration to be used in the prototype was Selenium's *implicit wait* strategy. Additional features such as alerts or links opening in a new window or tab were not handled.

Preprocessing of User-Defined URLs

This was not implemented in the prototype.

Input Alphabet – Scope

The prototype used the same general approach as the current version of the learning application to define the scope of the exploration and generate the input alphabet. It was limited to buttons as interactive elements only, and looked for these HTML elements by tag name.

Initial State – Reset

The prototype used a predefined starting point for the exploration of the SUL, which is specified by the user in the form of an initial URL, same as in the current version of the learning application. In the prototype, the reset only included deleting all cookies and loading the initial URL.

Input Enabled Handling

The prototype used self-loops to make the SUL input enabled.

Output – Current URL

The prototype used only the current URL after an interaction as an output. No additional outputs to distinguish edge cases were used.

HTML Elements & Interactions

As the prototype only looked for and handles buttons as interactive elements, interactions were simply simulated as a click in the centre of the button.

7.2 Evaluation of the Prototype

The prototype was evaluated on an early version of the Car Alarm System demo website described in Section 6.1²³ with the setup listed in Table 11. The URLs listed in Table 12 were used to define the SUL.

Operating System	<i>Microsoft Windows 10 Home Version 10.0.19045 Build 19045</i>
CPU Model	<i>Intel(R) Core(TM) i7-7700HQ CPU @ 2.80GHz</i>
CPU Count	<i>8</i>
Download Speed	<i>~45 - 110 mbit/s</i>
Upload Speed	<i>~5 - 15 mbit/s</i>
Ping	<i>~20-25 ms</i>

Table 11: Prototype – Device specifications and network speed

Initial URL	<i>https://lklamb.github.io/CarAlarmSystem_Demo/</i>
URL Scope	<i>https://lklamb.github.io/CarAlarmSystem_Demo/index.html</i>
	<i>https://lklamb.github.io/CarAlarmSystem_Demo/armed.html</i>
	<i>https://lklamb.github.io/CarAlarmSystem_Demo/alarm.html</i>

Table 12: Prototype – URL settings

Experiments were run with the State Prefix equivalence oracle and the Random Walk equivalence oracle with different parameter values.

Additionally, as during the course of these experiments it became obvious that the learning process for the Car Alarm System demo website took a long time, multiple versions of the demo website with reduced functionality were implemented. The number of states of the website was reduced by disabling some of the buttons for opening/closing doors, causing the doors whose buttons were disabled to always stay closed, as was their initial state.

This resulted in a 4-door, a 2-door, and a 1-door version of the system (see Table 13). Disabled buttons were still found during the website analysis and therefore part of the input alphabet, but the implementation of the prototype was adapted to handle disabled buttons by not clicking them and just returning the current URL as an output. This led to more self-loops in the learned automaton, but was a design choice to make the results between different versions more comparable by keeping the input alphabet the same for all versions of the demo website.

Version	Description	Nr. of Observable States
<i>4-door</i>	<i>all four door buttons enabled</i>	<i>50</i>
<i>2-door</i>	<i>two door buttons disabled</i>	<i>14</i>
<i>1-door</i>	<i>three door buttons disabled</i>	<i>8</i>

Table 13: Prototype – Car Alarm System demo website analysis

The number of observable states in Table 13 was determined manually as a basis for comparison with the results of the prototype.

²³ The evaluation took place on the following status:

Prototype code: https://github.com/lklamb/aalpy-selenium-website-learning_prototype/tree/04f7d54a94707ea7c9981e2a98b9b8f8eef791c2
 4-door system: https://github.com/lklamb/CarAlarmSystem_Demo/tree/e65081686d4e0635b7a525797a56d0a32520cf58 (tag: prototype-4doors)
 2-door system: https://github.com/lklamb/CarAlarmSystem_Demo/tree/e69ae3346d75e2a9b0f637f840f22636a483da80 (tag: prototype-2doors)
 1-door system: https://github.com/lklamb/CarAlarmSystem_Demo/tree/e2eb3bcca6bc8e556bdb4b130286b26968782915 (tag: prototype-1door)

7.2.1 Learned Automata

For the 2-door and 1-door versions of the website the prototype was able to learn a correct model of the system. The learned automaton for the 1-door version is shown in Figure 39.

Attempts to learn the 4-door version of the website completely were unsuccessful in a limited timeframe, because the state space was too big and the learning too slow. Nevertheless, the prototype was able to learn parts of the model successfully, leading to the assumption that running the program with different parameters in a larger timeframe would result in a correctly learned model.

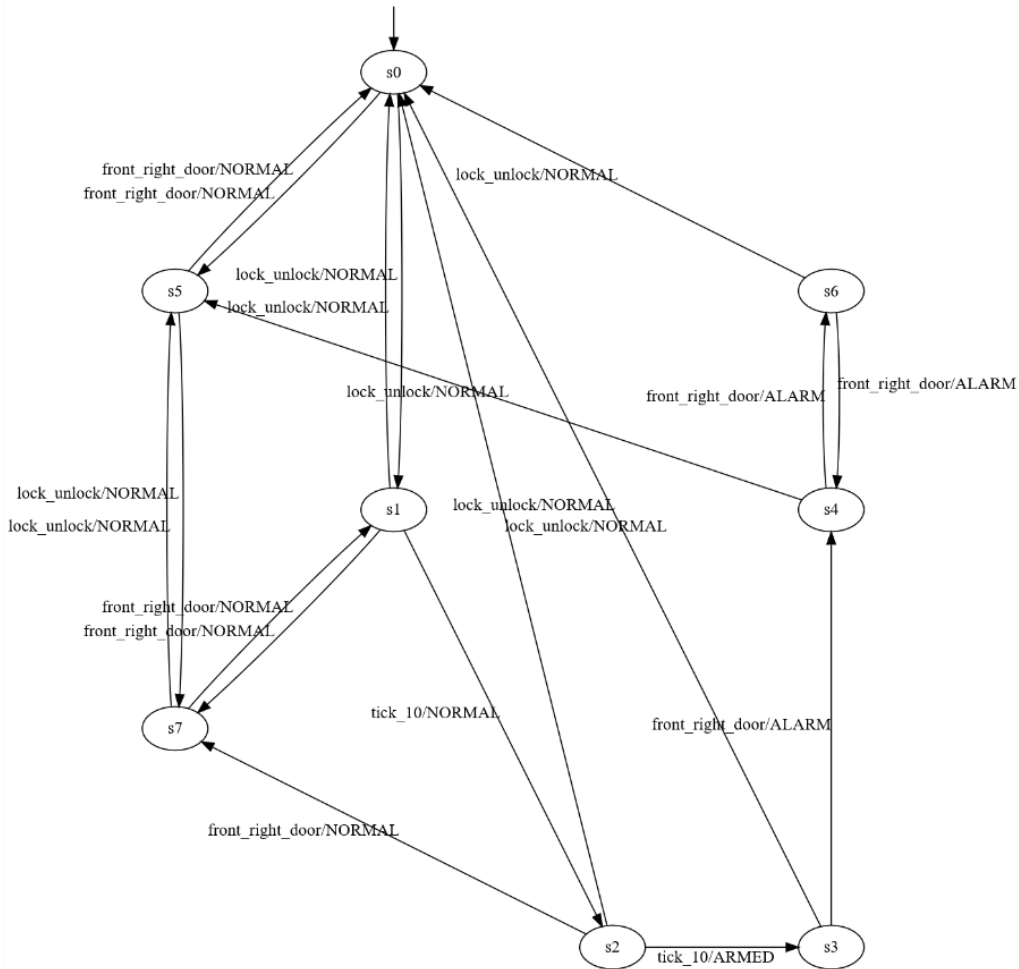


Figure 39: Prototype – Learned automaton for the 1-door Car Alarm System demo website

7.2.2 Impact of Equivalence Oracle Choice on Performance

Experiments were run with the State Prefix equivalence oracle (all versions of the website) and the Random Walk equivalence oracle (2-door version of the website) with different parameter values. For the State Prefix equivalence oracle three parameters were set to different values in different combinations: the number of walks per state, the walk length, and whether to explore newly discovered states first or in a random order. For the Random Walk equivalence oracle, the values for the total number of steps and the reset probability were varied. The results of these experiments are shown in Table 14, Table 15, Table 16, and Table 17.

7.2.2.1 State Prefix Equivalence Oracle

<i>Settings</i>			<i>Results</i>	
<i>Walks per State</i>	<i>Walk Length</i>	<i>Depth First</i>	<i>Found States / Observable States</i>	<i>Total Time [h]</i>
60	20	false	12 / 50	3.5
100	20	false	18 / 50	11.2
150	30	false	29 / 50	40.6
150	30	true	33 / 50	39.1

Table 14: Prototype – 4-door Car Alarm System demo website, performance of State Prefix equivalence oracle²⁴

<i>Settings</i>			<i>Results</i>	
<i>Walks per State</i>	<i>Walk Length</i>	<i>Depth First</i>	<i>Found States / Observable States</i>	<i>Total Time [h]</i>
100	30	false	13 / 14	7.0
100	30	true	11 / 14	4.6
150	20	false	11 / 14	3.6
150	20	true	12 / 14	6.5
150	30	false	14 / 14	10.2
150	30	true	14 / 14	10.9

Table 15: Prototype – 2-door Car Alarm System demo website, performance of State Prefix equivalence oracle

<i>Settings</i>			<i>Results</i>	
<i>Walks per State</i>	<i>Walk Length</i>	<i>Depth First</i>	<i>Found States / Observable States</i>	<i>Total Time [h]</i>
100	20	false	7 / 8	1.4
150	30	false	7 / 8	2.2
150	30	true	8 / 8	3.7
100	30	true	8 / 8	3.1
150	20	true	8 / 8	3.1
100	20	true	8 / 8	1.9
50	20	true	8 / 8	1.4
50	10	true	6 / 8	0.6
40	20	true	7 / 8	1.1

Table 16: Prototype – 1-door Car Alarm System demo website, performance of State Prefix equivalence oracle²⁵

For the number of walks per state and the walk length, choosing higher values increases the probability of learning a correct model, but also increases the learning time. Whether exploring newly discovered states first or in a random order led to better results was highly dependent on the configuration of the website.

Since the 2-door version of the website system has a lot of branching paths leading to lots of different states close to the initial state, exploring newly discovered states first did not increase the number of

²⁴ Only the best results on the 4-door system are shown. To learn this system, much higher settings for walk lengths and walks per state would have been needed, leading to a much longer learning time.

²⁵ Runs are shown in the order they were executed. Parameters were adapted according to the results of the previous runs.

found states significantly and in some experiments even decreased it. It did increase the time needed to learn the correct model, once the other two parameters were fixed.

On the other hand, the 1-door version of the website system has less complexity close to the initial state, so when exploring newly discovered states first, the states at the end of longer paths could be found more quickly without missing the earlier states. The walks per state and the walk length could be reduced significantly while still resulting in a correct model, leading to a reduced learning time.

As already mentioned, the 4-door version of the website could not be learned completely, because the state space was too big.

7.2.2.2 Random Walk Equivalence Oracle

<i>Settings</i>		<i>Results</i>	
<i>Number of Steps</i>	<i>Reset Probability</i>	<i>Found States / Observable States</i>	<i>Total Time [h]</i>
500	0.09	1 / 14	0.02
5000	0.09	6 / 14	0.5
50000	0.09	12 / 14	7.8
100000	0.09	12 / 14	11.9
100000	0.12	12 / 14	15.5

Table 17: Prototype – 2-door Car Alarm System demo website, performance of Random Walk equivalence oracle

Additional experiments with the Random Walk equivalence oracle were abandoned, because the State Prefix equivalence oracle showed much more promising results.

8 Evaluation

In this chapter, the results of executing different configurations of the learning application on the two case study websites described in Chapter 6 are evaluated.

8.1 Active Learning Equivalence Oracle

Because the evaluation of the first prototype showed much more promising results for the State Prefix equivalence oracle than the Random Walk equivalence oracle, active learning of the case study websites with the final version of the learning application was only evaluated using the State Prefix equivalence oracle.

8.2 Seeds for Active Learning

Since the active learning process uses some pseudo-randomness in AALpy’s implementation of the L* algorithm with the State Prefix equivalence oracle, seeds were used to make the results for active learning reproducible. This is possible by setting *seed* in the configuration file. Unless otherwise noted, all results for active learning listed in this chapter have been proven to be the same for the seeds 3, 42, 123, 66, and 1234.

8.3 Test Website

The Test Website can be loaded in the learning application by setting *website_to_learn* to different enum strings in the configuration file. Depending on which string is used, the scope will include the whole website described in Section 6.2, or one of its pages only. This also affects the size of the input alphabet, as shown in Table 18.

Visualisations of the learned automata for the Test Website SULs can be found in the appendix.

<i>Website SUL</i>	<i>Size of URL Scope</i>	<i>Size of Input Alphabet</i>
<i>TEST_WEBSITE</i>	9	65
<i>TW_INDEX</i>	1	8
<i>TW_BASICS</i>	1	8
<i>TW_MAP_AREAS</i>	1	4
<i>TW_DROPDOWNS</i>	1	11
<i>TW_CHECKBOXES</i>	1	4
<i>TW_SELECTIONS_CHOICES</i>	1	9
<i>TW_ATTRIBUTE_COMBINATIONS</i>	1	18
<i>TW_TABS_WINDOWS</i>	1	3

Table 18: Test Website – Size of scope and input alphabet

8.3.1 Single Page Test Website SULs

Learning SULs whose scope only consists of a single page shows slightly different results than learning a system of connected pages, so it needs to be evaluated separately. This section concerns all of the SULs listed in Table 18 except for *TEST_WEBSITE*.

Active learning with the State Prefix equivalence oracle, passive learning with PTA preprocessing, and passive learning with the custom compatibility function result in the same learned automata for all single page SULs. The learned automata have been checked manually for correctness. This proves

that the learning application can correctly handle all elements that occur on the Test Website, and learn the behaviour associated with them.

<i>Website SUL</i>	<i>Self-Loop IEH</i>		<i>Sink State IEH</i>	
	<i>States</i>	<i>Transitions excluding Self-Loops</i>	<i>States</i>	<i>Transitions excluding Self-Loops</i>
<i>TW_INDEX</i>	9	8	9	8
<i>TW_BASICS</i>	7	11	7	11
<i>TW_MAP_AREAS</i>	4	3	4	3
<i>TW_DROPDOWNS</i>	8	32	8	32
<i>TW_CHECKBOXES</i>	5	12	5	12
<i>TW_SELECTIONS_CHOICES</i>	9	36	9	36
<i>TW_ATTRIBUTE_COMBINATIONS</i>	3	16	3	16
<i>TW_TABS_WINDOWS</i>	2	3	2	3

Table 19: Test Website Single Page SULs – Structure of correct learned automata

Table 19 lists the basic structure of each learned automaton. When learning a SUL with a scope of only the initial page as the single URL, all inputs are on the same page by default. Pages at the boundary of the scope are treated as edge cases that always create self-loops. Due to this, when learning a single page, the type of input enabled handling does not have any effect on the resulting learned automaton.

<i>Website SUL</i>	<i>PASSIVE</i>	<i>ACTIVE</i>	
	<i>with PTA Preprocessing or Custom Compatibility Function</i>	<i>with State Prefix Equivalence Oracle</i>	
	<i>Trace Length</i>	<i>Walks per State</i>	<i>Walk Length</i>
<i>TW_INDEX</i>	1	0	0
<i>TW_BASICS</i>	3	0	0
<i>TW_MAP_AREAS</i>	1	0	0
<i>TW_DROPDOWNS</i>	4	0	0
<i>TW_CHECKBOXES</i>	4	0	0
<i>TW_SELECTIONS_CHOICES</i>	5	0	0
<i>TW_ATTRIBUTE_COMBINATIONS</i>	1	0	0
<i>TW_TABS_WINDOWS</i>	1	0	0

Table 20: Test Website Single Page SULs – Minimum parameter values needed to learn correct automata

Table 20 shows the minimum parameter values necessary to learn the correct automata for each single page Test Website SUL.

The passive learning process takes only the length of traces to be generated as a parameter. The necessary trace length is the same whether learning with PTA preprocessing or with the custom compatibility function, as this choice only affects the state-merging algorithm, but not the trace generation.

For active learning, the correct automata for the single page Test Website SULs can be found in the first learning round, meaning the first hypothesis is correct. It can be inferred that the structure of these SULs is such that the observation table of the L* algorithm is never closed and consistent until

the whole behaviour of the SUL has been covered. As the parameters of the active learning process only affect how the algorithm looks for a counterexample (for the State Prefix Equivalence Oracle: walks per state, length of each walk, newly discovered states are explored first or in a random order), and there is no counterexample to be found, the parameters do not matter for achieving the correct automata. The length of walks and the number of walks per state do however affect the execution time of the learning process, and have therefore been set to zero to minimize it for comparison to other configurations. This means that the first hypothesis of the learning algorithm is accepted immediately and no counterexample is looked for.

	<i>PASSIVE</i>		<i>ACTIVE</i>	
	<i>with PTA Preprocessing or Custom Compatibility Function</i>		<i>with State Prefix Equivalence Oracle</i>	
<i>Website SUL</i>	<i>Resets</i>	<i>Interactions</i>	<i>Resets</i>	<i>Interactions</i>
<i>TW_INDEX</i>	8	8	576	1 664
<i>TW_BASICS</i>	160	432	448	1 344
<i>TW_MAP_AREAS</i>	4	4	64	176
<i>TW_DROPDOWNS</i>	14 212	55 308	968	3 267
<i>TW_CHECKBOXES</i>	260	940	80	256
<i>TW_SELECTIONS_CHOICES</i>	5 494	269 874	729	2 592
<i>TW_ATTRIBUTE_COMBINATIONS</i>	18	18	972	2 592
<i>TW_TABS_WINDOWS</i>	3	3	18	45

Table 21: Test Website Single Page SULs – Number of SUL interactions and resets using minimal sufficient parameter values

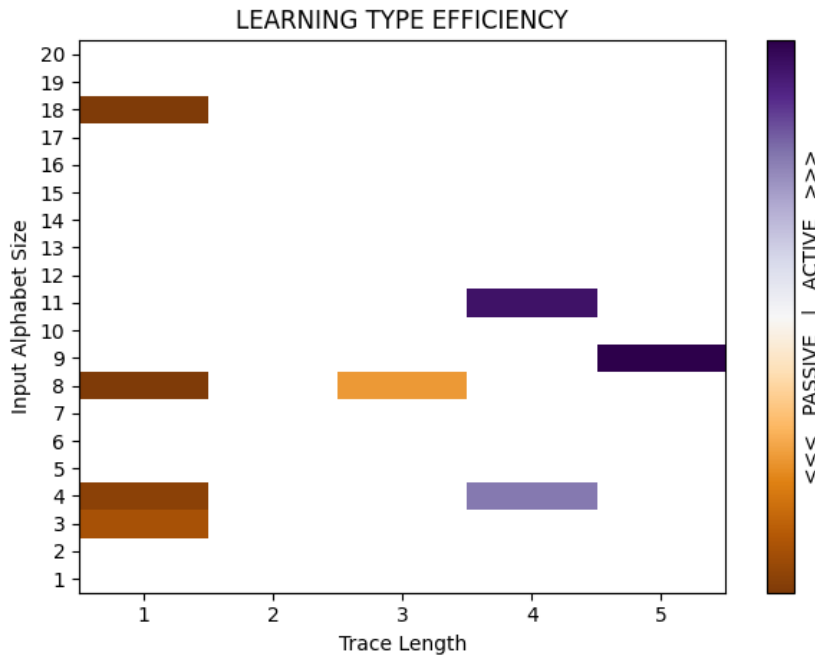


Figure 40: Test Website Single Page SULs – Efficiency of active versus passive learning

Table 21 lists the number of resets of and interactions with the SUL that occur when learning with minimum sufficient parameter values. Figure 40 shows a comparison of active and passive learning on all single page Test Website SULs. The necessary trace length to cover the whole behaviour and the size of the input alphabet are taken as measures for the complexity of the website SUL. The sum

of needed SUL interactions and resets is used as a measure of the efficiency of each learning type, with less interactions and resets being more efficient. The results show that there is a certain tipping point where active learning becomes more efficient than passive learning. The SULs with lower necessary trace lengths are more efficiently learned with passive learning. The ones with higher necessary trace lengths are more efficiently learned with active learning. Increasing the size of the input alphabet seems to intensify the advantage of the more efficient learning type for SULs with the same necessary trace length, though more data points would be needed to sufficiently analyse this effect.

		<i>PASSIVE</i>	
<i>Passive Approach</i>		<i>PTA Preprocessing</i>	<i>Custom Compatibility Function</i>
<i>TW_INDEX</i>	<i>PTA Nodes after Trace Generation</i>	9 (100 %)	9 (100 %)
	<i>PTA Nodes at Start of State-Merging</i>	73 (811 %)	9 (100 %)
	<i>State Merges</i>	64	0
	<i>State Promotions</i>	9	9
<i>TW_BASICs</i>	<i>PTA Nodes after Trace Generation</i>	161 (100 %)	161 (100 %)
	<i>PTA Nodes at Start of State-Merging</i>	845 (525 %)	161 (100 %)
	<i>State Merges</i>	50	10
	<i>State Promotions</i>	7	7
<i>TW_MAP_AREAS</i>	<i>PTA Nodes after Trace Generation</i>	5 (100 %)	5 (100 %)
	<i>PTA Nodes at Start of State-Merging</i>	17 (340 %)	5 (100 %)
	<i>State Merges</i>	13	1
	<i>State Promotions</i>	4	4
<i>TW_ATTRIBUTE_COMBINATIONS</i>	<i>PTA Nodes after Trace Generation</i>	19 (100 %)	19 (100 %)
	<i>PTA Nodes at Start of State-Merging</i>	307 (1 616 %)	19 (100 %)
	<i>State Merges</i>	52	16
	<i>State Promotions</i>	3	3
<i>TW_TABS_WINDOWS</i>	<i>PTA Nodes after Trace Generation</i>	4 (100 %)	4 (100 %)
	<i>PTA Nodes at Start of State-Merging</i>	13 (325 %)	4 (100 %)
	<i>State Merges</i>	5	2
	<i>State Promotions</i>	2	2

Table 22: Test Website Single Page SULs – Passive learning statistics using minimal sufficient trace length for SULs with low necessary trace lengths

For passive learning, the additional overhead to run the state-merging algorithm as well as either PTA preprocessing or postprocessing of the learned automaton needs to be considered, as the SUL interactions and resets only cover trace generation. The impact of this on the execution time is however negligible, because SUL interactions and resets are very slow when compared to executing the algorithm steps. Especially for short trace lengths, where the passive learning shows its advantage over the active learning, the size of the PTA is comparably small, and compatible merges can therefore be found quickly.

Table 22 shows a comparison between the two passive approaches for those Test Website single page SULs where passive learning is more efficient than active learning. It is obvious that passive learning

with the custom compatibility function is preferable to learning with PTA preprocessing, because it does not increase the size of the PTA, which is the main factor that influences the efficiency of the state-merging algorithm. Additionally, postprocessing the learned automaton is more efficient than preprocessing the PTA, because transitions that are supposed to exist in the final automaton are only added once. During PTA preprocessing, they are often added redundantly to nodes that will later be merged into a single state.

8.3.2 Whole Test Website SUL

Active learning with the State Prefix equivalence oracle, passive learning with PTA preprocessing, and passive learning with the custom compatibility function result in the same learned automata for the whole Test Website as well. The learned automata of the whole Test Website are too big for a manual check, but they have been checked for correctness by comparing them to automata created from a merged version of the single pages. This comparison is only valid because it is known in advance that the Test Website does not contain any hidden information that carries over between pages (e.g., cookies) that might affect its behaviour.

This proves that the learning application can correctly learn a system of connected pages.

<i>Website SUL</i>	<i>Self-Loop IEH</i>		<i>Sink State IEH</i>	
	<i>States</i>	<i>Transitions excluding Self-Loops</i>	<i>States</i>	<i>Transitions excluding Self-Loops</i>
<i>TEST_WEBSITE</i>	26	121	27	1 478

Table 23: Whole Test Website – Structure of correct learned automata

Table 23 lists the basic structure of the learned automata of the whole Test Website dependent on the type of input enabled handling. If the scope of the learning consists of multiple pages with interactive elements, the number of states of the correct automaton will always be exactly one higher for sink state input enabled handling than for self-loop input enabled handling, due to the artificially inserted sink state.

The difference in the number of transitions that are not self-loops depends on the structure of the SUL. It can be calculated with the formula

$$t_{SINK} = t_{SELF} + \sum_{p \in scope} I(p) \cdot |s(p)| \cdot (|\Sigma| - |\Sigma(p)|)$$

where

- t_{SINK} is the number of transitions excluding self-loops created with sink state IEH,
- t_{SELF} is the number of transitions excluding self-loops created with self-loop IEH,
- p is a single webpage,
- $I(p)$ is an indicator function to filter out dead end pages, which always create self-loops,

$$I(p) = \begin{cases} 1 & \text{if at least one input letter exists on } p \\ 0 & \text{otherwise} \end{cases}$$

- $|s(p)|$ is the number of distinct states that exist on p ,
- $|\Sigma|$ is the size of the input alphabet, and
- $|\Sigma(p)|$ is the amount of valid input letters that exist on p .

	<i>PASSIVE</i>	<i>ACTIVE</i>	
	<i>with PTA Preprocessing or Custom Compatibility Function</i>	<i>with State Prefix Equivalence Oracle</i>	
<i>Website SUL</i>	<i>Trace Length</i>	<i>Walks Per State</i>	<i>Walk Length</i>
<i>TEST_WEBSITE</i>	6	0	0

Table 24: Whole Test Website – Minimum parameter values needed to learn the correct automata

Table 24 shows the minimum parameter values necessary to learn the correct automata for the whole Test Website SUL.

The necessary trace length for passive learning is one higher than the highest necessary trace length for the individual Test Website pages, because the trace generation starts on the index page and the other pages can only be reached via an additional interaction.

For active learning, the necessary minimum parameters are not affected by learning the whole Test Website versus learning the single pages separately, meaning that here as well the correct automata are learned in the first learning round.

	<i>PASSIVE</i>		<i>ACTIVE</i>			
	<i>with PTA Preprocessing or Custom Compatibility Function</i>		<i>with State Prefix Equivalence Oracle</i>			
	<i>Self-Loop or Sink State IEH</i>		<i>Self-Loop IEH</i>		<i>Sink State IEH</i>	
<i>Website SUL</i>	<i>Resets</i>	<i>Interactions</i>	<i>Resets</i>	<i>Interactions</i>	<i>Resets</i>	<i>Interactions</i>
<i>TEST_WEBSITE</i>	273 124	1 604 821	109 850	430 950	114 075	443 625

Table 25: Whole Test Website – Number of SUL interactions and resets using minimal sufficient parameter values

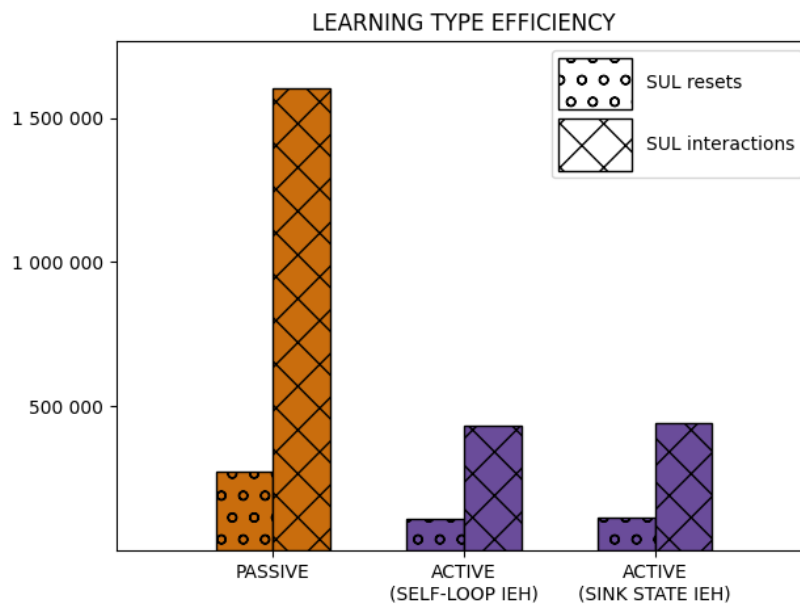


Figure 41: Whole Test Website – Efficiency of active versus passive learning

Table 25 and Figure 41 show the number of resets of and interactions with the SUL that occur when learning the whole Test Website with minimum sufficient parameter values. The results show that to learn the whole Test Website, active learning is much more efficient. This follows the theory resulting from evaluating the single page SULs, that the higher the necessary trace length to cover the whole behaviour of the website SUL, the more efficient active learning becomes when compared to passive learning. From the comparison between the two learning types, it can be inferred that the Test Website SUL as a whole is well above the tipping point where active learning becomes more efficient than passive learning. Active learning with self-loop input enabled handling is slightly more efficient than with sink state input enabled handling on the whole Test Website.

For passive learning, the passive approach and the type of input enabled handling do not have any effect on the number of SUL interactions and resets, as all interactions and resets take place during trace generation, where these choices are not relevant.

They do however heavily affect the number of nodes added during PTA preprocessing, as shown in Table 26, and through that the execution time of the state-merging algorithm if this passive approach is used. Since the input alphabet for the whole Test Website is large, a lot of child nodes are added when using self-loop input enabled handling. This is exacerbated when using sink state input enabled handling, which adds additionally an even larger number of grandchild nodes. Adding such a high number of nodes to the PTA, and then trying to find compatible merge candidates during the state-merging algorithm, takes a long time. Running this configuration exceeded hardware limitations, therefore the number of merges is not given.

This documents again that passive learning with the custom compatibility function and postprocessing is the preferable passive approach, because it does not add any nodes to the PTA. As already mentioned, postprocessing the learned automaton is also more efficient than preprocessing the PTA, because it avoids adding redundant transitions.

		<i>PASSIVE</i>		
<i>Passive Approach</i>		<i>PTA Preprocessing</i>		<i>Custom Compatibility Function</i>
<i>Input Enabled Handling</i>		<i>Self-Loop</i>	<i>Sink State</i>	<i>Self-Loop or Sink State</i>
<i>TEST WEBSITE</i>	<i>PTA Nodes after Trace Generation</i>	273 125 (100 %)	273 125 (100 %)	273 125 (100 %)
	<i>PTA Nodes at Start of State-Merging</i>	11 272 793 (4 127 %)	640 369 638 (234 460 %)	273 125 (100 %)
	<i>State Merges</i>	* ²⁶	* ²⁷	178
	<i>State Promotions</i>	26	27	26

Table 26: Whole Test Website – Passive learning statistics using minimal sufficient trace length

8.4 Car Alarm System Demo Website

The Car Alarm System demo website can be loaded in the learning application by setting *website_to_learn* to *CAR_ALARM* in the configuration file. All results listed in this section were determined on the reduced version of the website (see Section 6.1), which can currently be found under the URLs listed in Table 27.

²⁶ Value not calculated due to hardware limitations

²⁷ Value not calculated due to hardware limitations

Table 28 lists the basic structure of the correct learned automata dependent on the type of input enabled handling. Active learning with the State Prefix equivalence oracle, passive learning with PTA preprocessing, and passive learning with the custom compatibility function result in the same learned automata for the Car Alarm System demo website. Learning the pages of this SUL separately and then combining them is not possible, because hidden information is carried over between the pages. The whole learned automaton has therefore been checked manually for correctness.

This proves that the learning application can correctly learn a system of connected pages where information that affects the SUL behaviour is stored as cookies.

A visualisation of the learned automaton for the Car Alarm System demo website SUL can be found in the appendix.

Initial URL	<i>https://lklamb.github.io/CarAlarmSystem_Demo/index.html</i>
URL Scope (size 3)	<i>https://lklamb.github.io/CarAlarmSystem_Demo/index.html</i>
	<i>https://lklamb.github.io/CarAlarmSystem_Demo/armed.html</i>
	<i>https://lklamb.github.io/CarAlarmSystem_Demo/alarm.html</i>
Input Alphabet (size 12)	<i>click hyperlink to sources page on index.html</i>
	<i>click Open/Close Door button on .../index.html</i>
	<i>click Lock/Unlock button on .../index.html</i>
	<i>click Tick 10 button on .../index.html</i>
	<i>click hyperlink to sources page on .../armed.html</i>
	<i>click Open/Close Door button on .../armed.html</i>
	<i>click Lock/Unlock button on .../armed.html</i>
	<i>click Tick 10 button on .../armed.html</i>
	<i>click hyperlink to sources page on .../alarm.html</i>
	<i>click Open/Close Door button on .../alarm.html</i>
	<i>click Lock/Unlock button on .../alarm.html</i>
	<i>click Tick 10 button on .../alarm.html</i>

Table 27: Car Alarm System Demo Website – Initial URL, scope, and input alphabet

Website SUL	Self-Loop IEH		Sink State IEH	
	States	Transitions excluding Self-Loops	States	Transitions excluding Self-Loops
<i>CAR_ALARM</i>	9	26	10	90

Table 28: Car Alarm System Demo Website – Structure of correct learned automata

8.4.1 Active Learning – Depth First and Input Enabled Handling

As the active learning process is dependent on multiple different parameters via the State Prefix equivalence oracle, a grid search was done to find the optimal configurations for each type of input enabled handling. The results of this grid search are depicted in Table 29, Table 30, and Table 31. The resulting data points belong to one of three classes:

- **CORRECT:** The correct automaton could be learned for all tested seeds.
- **INCORRECT:** The correct automaton could not be learned for any of the tested seeds.
- **MISMATCH:** The correct automaton could be learned for some of the tested seeds, but not others.

The boundaries between the areas where each of these classes occur were approximated using different hyperbolas. In the green area, only **CORRECT** results were found. In the red area, only **INCORRECT** results were found. The yellow area contains a mix of **CORRECT**, **INCORRECT**, and **MISMATCH** results. It can be inferred that if choosing parameters in the yellow area, whether the correct automaton will be learned or not depends on the seed.

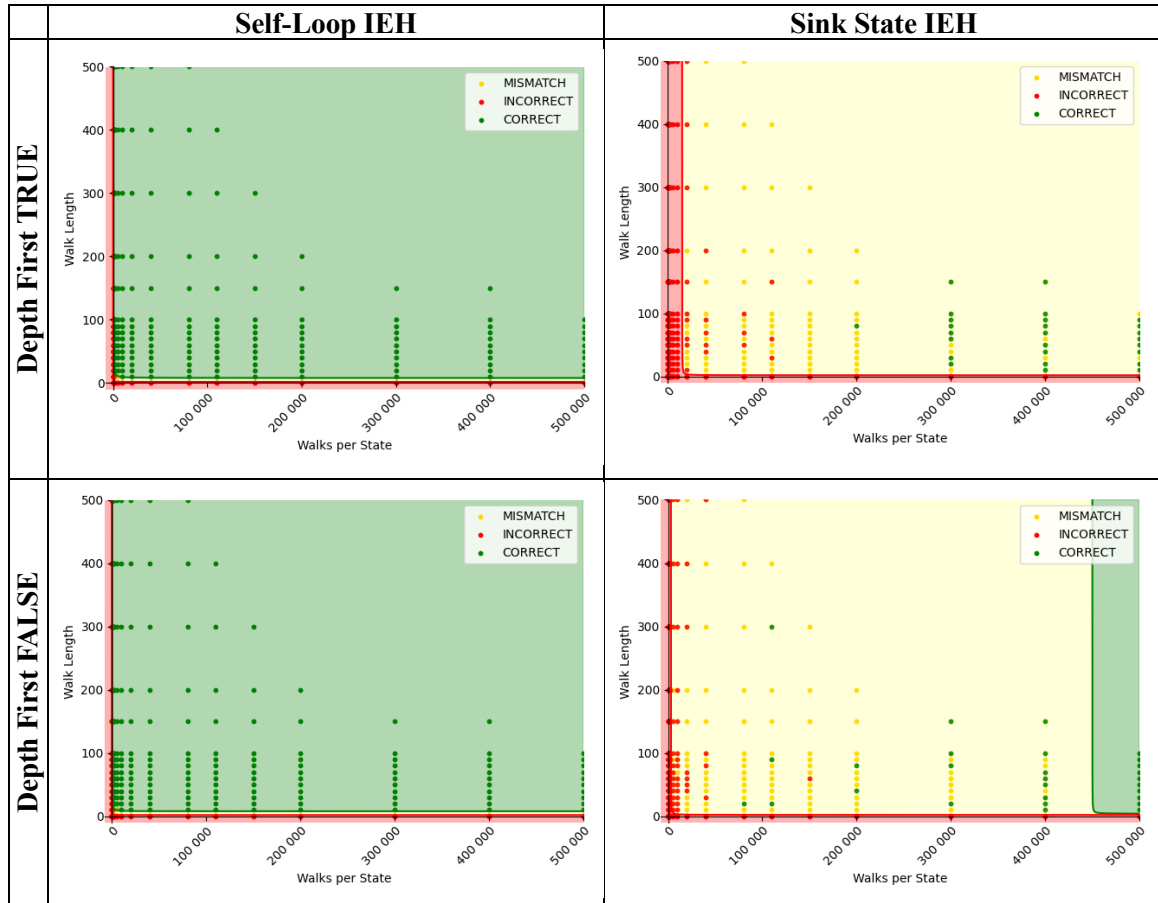


Table 29: Car Alarm System Demo Website – Overview of minimal necessary parameter values for active learning

From the results of the grid search it is immediately obvious that using self-loop input enabled handling is preferable to sink state input enabled handling. At a conservative estimate, with self-loop IEH the learning application is able to consistently learn the correct automaton with walks per state of above 600 and walk lengths above 100 if depth first exploration is used. If states are explored in a random order instead, walks per state of above 400 and walk lengths of above 80 are sufficient.

With sink state IEH, walks per state of 500 000 and walk lengths of 10 are needed to consistently learn the correct automaton, if states are explored in a random order. If newest states are explored first, not even these values are sufficient. The parameters were not increased further, as it was already clear from the existing results that self-loop IEH is preferable.

For both self-loop and sink state IEH, not exploring depth first is more efficient, as can be seen when looking at both the boundary between *INCORRECT* and *MISMATCH*, as well as the boundary between *MISMATCH* and *CORRECT* areas. It is assumed that this is due to a boundary state existing in the learned automaton, which acts as a natural sink state once reached, and therefore makes exploration at the end of paths less efficient.

It is assumed that for sink state IEH, the results are mostly dependent on the number of walks per state, as if the first input is not valid in the current state (meaning that it is not on the current page), it will immediately cause a transition to the sink state and the walk will get stuck there, no matter how long it is and if it contains valid inputs after that. The walk length must however still be high enough to be able to observe a change in the output.

For self-loop IEH, results are dependent on both the number of walks per state and the walk length, as getting a valid input after an invalid input may still lead to new behaviour being discovered.

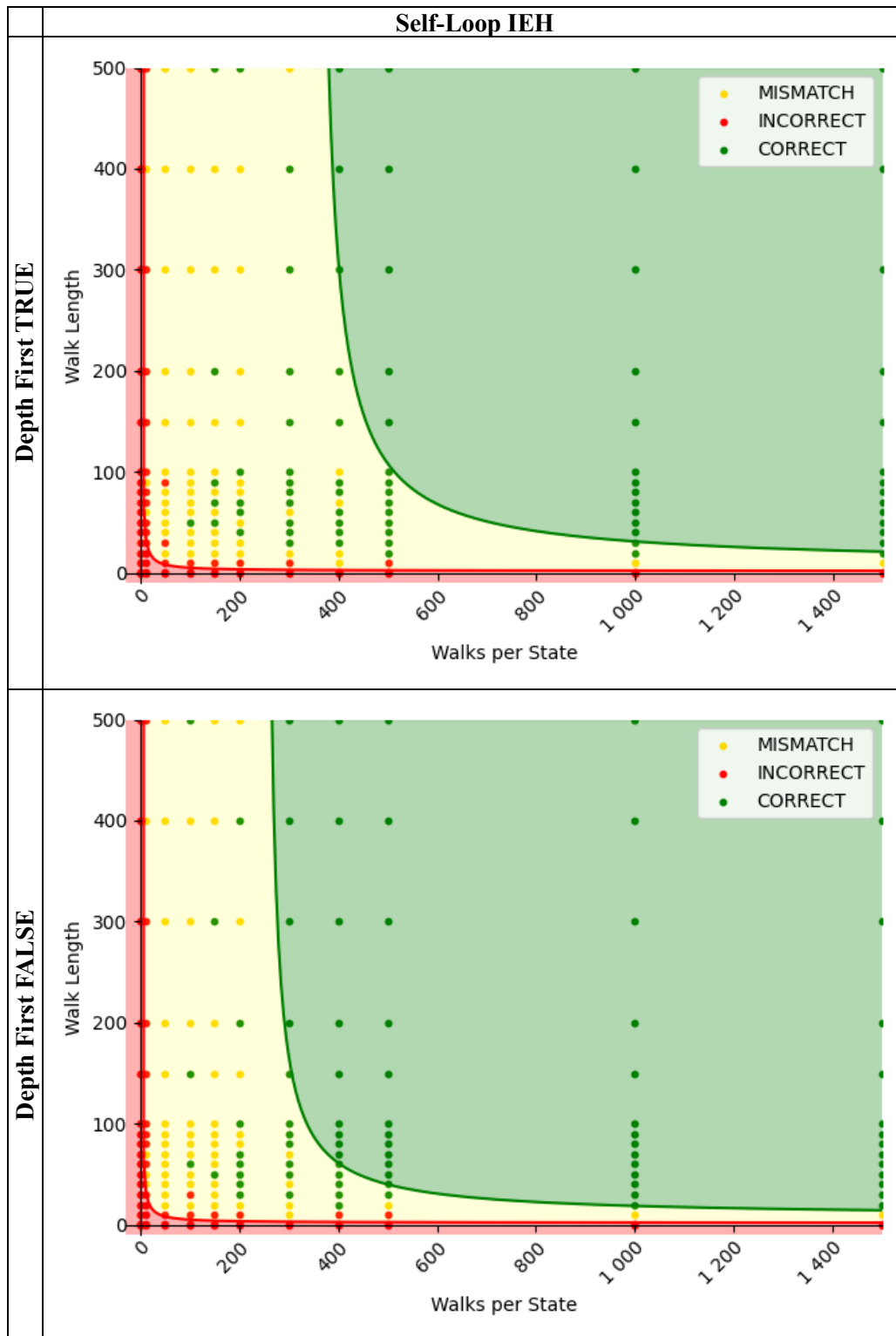


Table 30: Car Alarm System Demo Website – Detailed evaluation of minimal necessary parameter values for active learning with self-loop input enabled handling

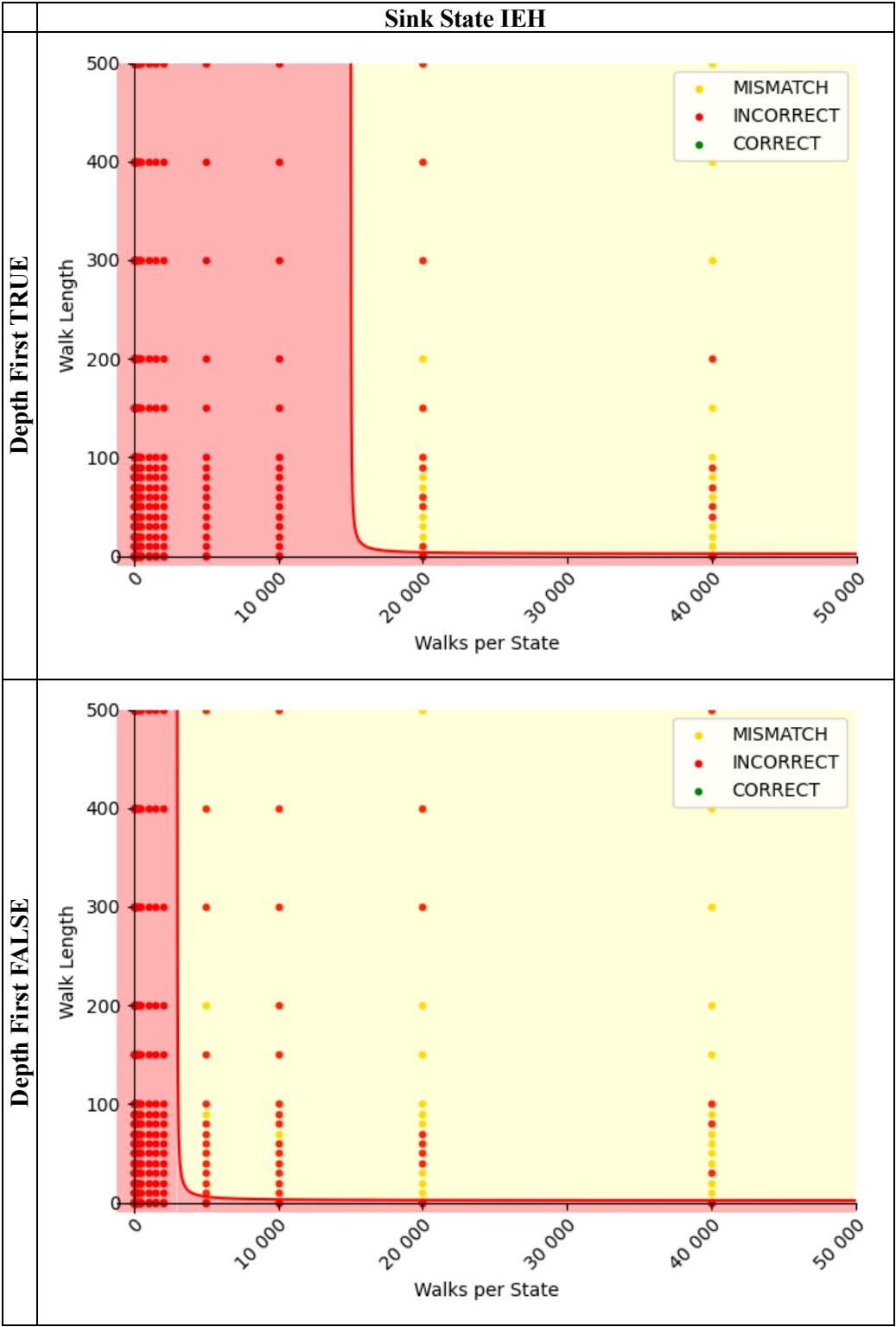


Table 31: Car Alarm System Demo Website – Detailed evaluation of minimal necessary parameter values for active learning with sink state input enabled handling

<i>Website SUL</i>	<i>CAR_ALARM</i>			
<i>Input Enabled Handling</i>	<i>Self-Loop</i>		<i>Sink State</i>	
<i>Depth First</i>	<i>True</i>	<i>False</i>	<i>True</i>	<i>False</i>
<i>Walks per State</i>	600	400	500 000	500 000
<i>Walk Length</i>	100	80	10	10
<i>Min. SUL Resets</i>	7 009	5 192	5 001 243	5 001 090
<i>Max. SUL Resets</i>	7 122	5 307	5 001 509	5 001 463
<i>Min. SUL Interactions</i>	560 474	303 366	60 005 593	60 004 671
<i>Max. SUL Interactions</i>	566 473	307 857	60 006 888	60 006 681

Table 32: Car Alarm System Demo Website – Active learning statistics with chosen parameters (minima and maxima over all tested seeds)

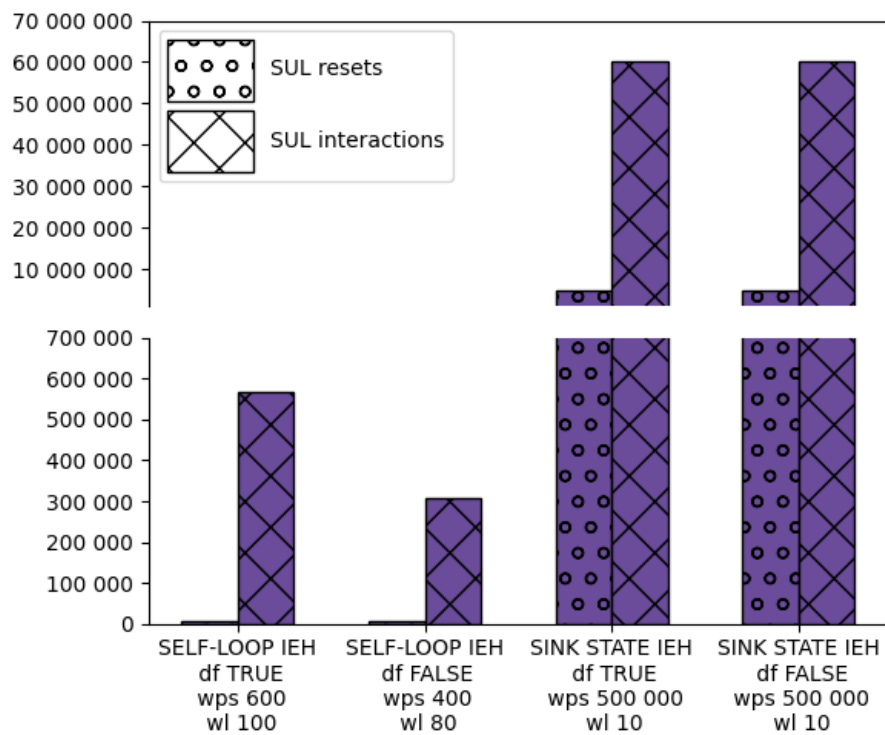


Figure 42: Car Alarm System Demo Website – Efficiency of active learning with chosen parameters (df = depth first, wps = walks per state, wl = walk length)

Table 32 and Figure 42 show the number of resets of and interactions with the SUL that occur when learning the Car Alarm System demo website with active learning with specific chosen parameters. For self-loop IEH, and for sink state IEH without depth first exploration, the number of walks per state and the length of walks was set to the previously mentioned conservative estimate of where results would start to be consistent for all seeds.

As the area where the correct automaton could be consistently learned independent of the seed could not be determined for sink state IEH with depth first exploration, these values represent a specific configuration where the correct automaton was learned with all tested seeds, and may not be generalizable to all seeds. The number of necessary resets and interactions will however definitely be even higher for the area where results become consistent.

Since self-loop IEH necessitates much lower parameter values than sink state IEH, the number of resulting SUL resets and interactions is also much lower. To a lesser degree, this is also true when deciding if newly discovered states should be explored first or not: since the necessary parameter values for exploring states in a random order are lower, the number of SUL resets and interactions are also lower.

8.4.2 Passive Learning – Passive Approach and Input Enabled Handling

<i>Website SUL</i>	<i>CAR_ALARM</i>		
<i>Minimal Sufficient Trace Length</i>	<i>10</i>		
<i>Passive Approach</i>	<i>PTA Preprocessing</i>		<i>Custom Compatibility Function</i>
<i>Input Enabled Handling</i>	<i>Self-Loop</i>	<i>Sink State</i>	<i>Self-Loop or Sink State</i>
<i>SUL Resets</i>	<i>118 096</i>	<i>118 096</i>	<i>118 096</i>
<i>SUL Interactions</i>	<i>1 121 932</i>	<i>1 121 932</i>	<i>1 121 932</i>
<i>PTA Nodes after Trace Generation</i>	<i>118 097 (100 %)</i>	<i>118 097 (100 %)</i>	<i>118 097 (100 %)</i>
<i>PTA Nodes at Start of State-Merging</i>	<i>1 180 969 (1 000 %)</i>	<i>9 683 977 (8 200 %)</i>	<i>118 097 (100 %)</i>
<i>State Merges</i>	<i>100</i>	<i>111</i>	<i>24</i>
<i>State Promotions</i>	<i>9</i>	<i>10</i>	<i>9</i>

Table 33: Car Alarm System Demo Website – Passive learning statistics using minimal sufficient trace length

Table 33 shows statistics of learning the Car Alarm System demo website SUL with passive learning. To learn this SUL, a minimum trace length of 10 is necessary during trace generation, meaning that some behaviour is quite deeply hidden within the system.

The number of SUL resets and interactions as well as the number of PTA nodes before preprocessing are a result of the trace generation only, and are therefore not affected by the choice of passive approach and input enabled handling. Conversely, the number of PTA nodes at the start of the state-merging algorithm is heavily affected by these choices.

If using the custom compatibility function, no nodes are added to the PTA. Learning with the custom compatibility function therefore necessitates less merges during the state-merging algorithm.

If using PTA preprocessing with self-loop input enabled handling, child nodes are added for those inputs that are not on the current page. If sink state input enabled handling is used, even more nodes are added as grandchild nodes. The number of merges of passive learning with PTA preprocessing with either type of input enabled handling is almost the same, meaning that many nodes are merged via implied merges. Nevertheless, the executions have shown that looking for a compatible merge takes much longer for sink state IEH than for self-loop IEH due to the larger size of the PTA.

It is clear that passive learning with the custom compatibility function is much more efficient than passive learning with PTA preprocessing, because it does not add any nodes to the PTA. As already mentioned, postprocessing the learned automaton is also more efficient than preprocessing the PTA, because it avoids adding redundant transitions.

8.4.3 Active versus Passive Learning

	<i>PASSIVE</i>		<i>ACTIVE</i>	
	<i>with Custom Compatibility Function</i>		<i>with State Prefix Equivalence Oracle</i>	
<i>IEH</i>	<i>Self-Loop</i>		<i>Self-Loop</i>	
<i>Parameters</i>	<i>Trace Length</i>	<i>10</i>	<i>Depth First</i>	<i>False</i>
			<i>Walks per State</i>	<i>400</i>
			<i>Walk Length</i>	<i>80</i>
<i>SUL Resets</i>	<i>118 096</i>		<i>5 270²⁸</i>	
<i>SUL Interactions</i>	<i>1 180 969</i>		<i>305 714²⁹</i>	

Table 34: Car Alarm System Demo Website – Number of SUL interactions and resets for most efficient passive and active configuration using minimal consistently sufficient parameter values

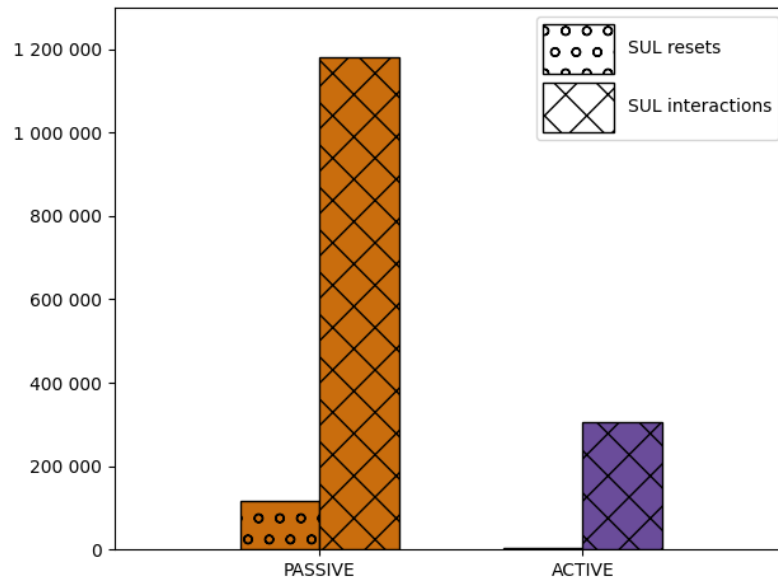


Figure 43: Car Alarm System Demo Website – Efficiency of active versus passive learning

Table 34 and Figure 43 show the number of resets of and interactions with the SUL that occur when learning the Car Alarm System demo website with the most efficient active, as well as the most efficient passive configuration with minimum consistently sufficient parameters. The results show that for this SUL as well, active learning is much more efficient. This proves once again that for high necessary trace lengths to cover the whole behaviour of the website SUL, active learning becomes more efficient than passive learning. The Car Alarm System demo website is well above the tipping point where this occurs.

For passive learning with the custom compatibility function, the additional overhead to run the state-merging algorithm, as well as the postprocessing of the learned automaton, needs to be considered, as the SUL interactions and resets only cover the trace generation. This means that in reality, compared to active learning, passive learning is even less efficient on the Car Alarm System demo website than depicted in Figure 43.

²⁸ Average over tested seeds

²⁹ Average over tested seeds

9 Concluding Remarks

9.1 Summary

An automata learning application was implemented to learn a behavioural model of a website system. It uses AALpy as a learning library and Selenium to access the website. The learning application is set up to learn a deterministic Mealy machine representation of the SUL.

Two types of learning were implemented: active, where during learning the SUL is actively queried with an input and an output is returned, and passive, where the behaviour is learned from preexisting input-output traces.

As a website is not by default input enabled, two different solutions were found to make it so: using self-loops for inputs that are not on the current page, or transitioning to an artificial sink state. These two types of input enabled handling were used for both active and passive learning.

The initial state of the SUL and the scope of the learning is defined by the user in the shape of URLs. During the learning process, the SUL is reset to this initial state multiple times. Exploration of the website SUL continues until the boundary of the scope is reached.

The input alphabet is defined by analysing all pages within the scope for interactive elements. From each interactive element, one or more input letters are created. Inputs are separate per page, as they are in the background HTML structure.

As output, mostly the current URL after an interaction is used. Edge cases as well as necessities of the chosen input enabled handling type are associated with specific output strings.

For active learning, AALpy's L^* algorithm with the Random Walk equivalence oracle or the State Prefix equivalence oracle was used.

For passive learning, first traces were generated in a way that kept the necessary SUL interactions and resets to a minimum. Then, AALpy's state merging algorithm was applied to the resulting PTA. Two different approaches were used to add the information lost during trace generation back in via domain knowledge: In the first approach, the PTA was preprocessed by adding additional nodes, and these nodes merged correctly by using a custom scoring function. In the second approach the PTA was merged as is, with a custom compatibility function being used to keep states on different pages from merging. Then, the resulting automaton was postprocessed by adding missing transitions directly.

Different configurations of the learning application were executed on two case study websites, and the results analysed to find the configuration that is most suited to learning a website SUL.

9.2 Conclusion

In conclusion, the three research questions posed at the beginning of this thesis can be answered as follows:

1. Can an automaton that correctly represents the whole behaviour of a website be learned using AALpy and Selenium?

Yes, on all case study website SULs the learning application was able to learn the correct automata for all configurations with sufficiently high parameter values. This proves that it is able to correctly learn single page SULs with different HTML elements, systems of connected pages, as well as website SULs where hidden information that influences behaviour is transferred via cookies.

2. Is passive or active learning more suitable for learning an automaton of a website system?

This depends on the complexity of the website SUL. For websites where behaviour is deeply hidden within the system, so a high trace length is necessary to observe it completely, active learning is more efficient. For less complex websites where all observable behaviour takes place close to the initial state, passive learning is more efficient.

3. Is it preferable to use self-loops or an artificial sink state to make the website system input enabled?

Using self-loops is much preferable. For all configurations, introducing an artificial sink state reduced efficiency, added additional overhead to the calculations, and hindered learning.

Additionally, it became clear during the course of this thesis that for active learning with AALpy's implementation of the L* algorithm, the State Prefix equivalence oracle is much more suited than the Random Walk equivalence oracle. For passive learning with AALpy's state-merging algorithm, using a custom compatibility function and postprocessing the learned automaton was much more efficient than preprocessing the PTA and using a custom scoring function.

9.3 Future Work

The evaluation of this thesis project shows that it is possible to learn a correct model of a website system using automata learning. However, the implemented learning application still has some limitations.

Especially in the website analysis phase, further improvements would be possible. Below listed are some functionalities that exceeded the scope of this thesis, but would make the learning application more generally applicable to various websites.

- The learning application is currently limited to interacting with elements that can be found directly in the main `<body>` of a webpage. To also find elements in `<iframes>` or `<frames>`, extending some of the functionality to work recursively would be necessary.
- When searching for interactive elements to generate the input alphabet, the learning application currently only looks for specific inline event handlers, which are more common in older or simpler code. To be better suited for analysing more modern websites, extending this step to also look for JavaScript event listeners and handlers would be recommended. Additionally, the set of inline event handlers that are currently looked for is only a small subset of those that exist, and could be extended.
- The learning application could be improved by adding the handling of additional types of HTML elements, such as text input fields, which are commonly used in forms. This would necessitate some type of generator to create the content to be placed in the field. Since text input fields typically impose few restrictions on what can be entered, attempting to test every possible input quickly leads to state explosion, which is an issue that would need to be considered when implementing the content generator.
- Currently, the input alphabet is generated by analysing the elements in webpages that have been defined by the user (URL scope). This means that the user would either in advance have to have some knowledge about which webpages are part of the system, or run the learning application multiple times and extend the URL scope each time new webpages are found. It might be preferable to generate the URL scope automatically from the provided initial URL by using a web crawler.

The following extensions could be made to the learning process itself to cover more scenarios where a website might behave unexpectedly:

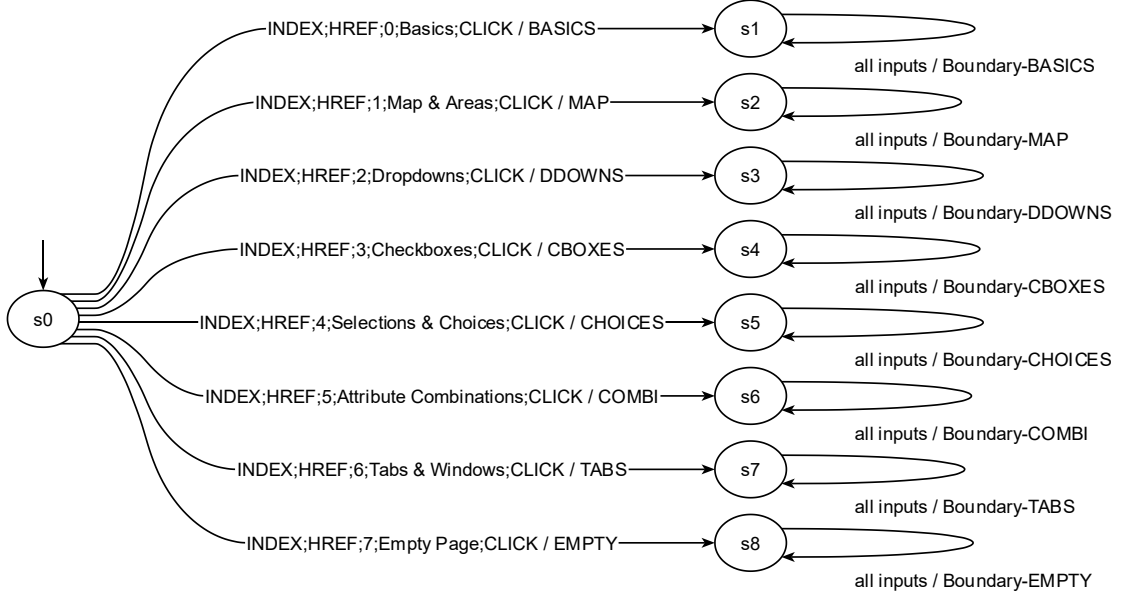
- Using the browser back and forward buttons may influence behaviour. These interactions could be added to the input alphabet. This has the drawback of reduced efficiency for the whole learning process, as they will be tried in any state, even if there is no relevant behaviour.
- Alert boxes are currently prevented from opening. Adding interactions with alert boxes to the input alphabet might unveil additional behaviour. This has the same drawback of reducing efficiency, as these inputs would have to be tried and handled in all states, even if no alert box is open.
- Links are currently forced to open in the same tab of the same window. Opening links in a new tab or window, if they are set up to do so, would more faithfully represent the behaviour of the website SUL, but also greatly increase the complexity of the learning process.

Finally, further experiments and evaluations could provide more insights:

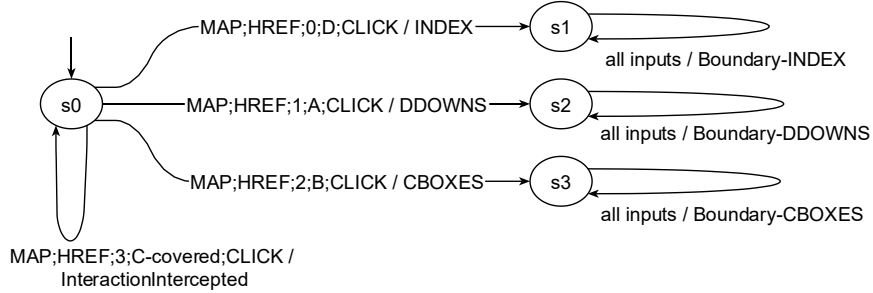
- The learning application has currently been evaluated on two different case study websites which were created specifically for this thesis. Executions on different “real” websites would increase the data available about its suitability and reduce the risk of selection and confirmation bias.
- AALpy’s implementation of the L^* algorithm also offers support for other equivalence oracles than the two used in this thesis, which might give better results.
- The results of the evaluation described in Chapter 8 show that there is a certain tipping point where active learning becomes more efficient than passive learning. It seems like this is directly related to how deeply behaviour is hidden on the website that is learned, so how long an input-output trace through the system must be to cover all behaviour. Increasing the size of the input alphabet seems to only amplify the efficiency of the more efficient learning type. For the case study website SULs, the tipping point where active learning becomes more efficient than passive learning was a necessary trace length of greater than three. Additional experiments with websites which only differ in one aspect would be needed to analyse this effect, and consequently the location of the tipping point sufficiently.

Appendices

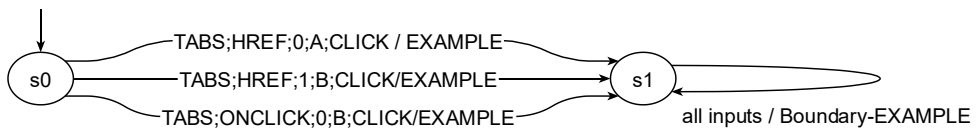
The URLs in the in- and outputs of the learned automata have been replaced with abbreviations for easier readability.



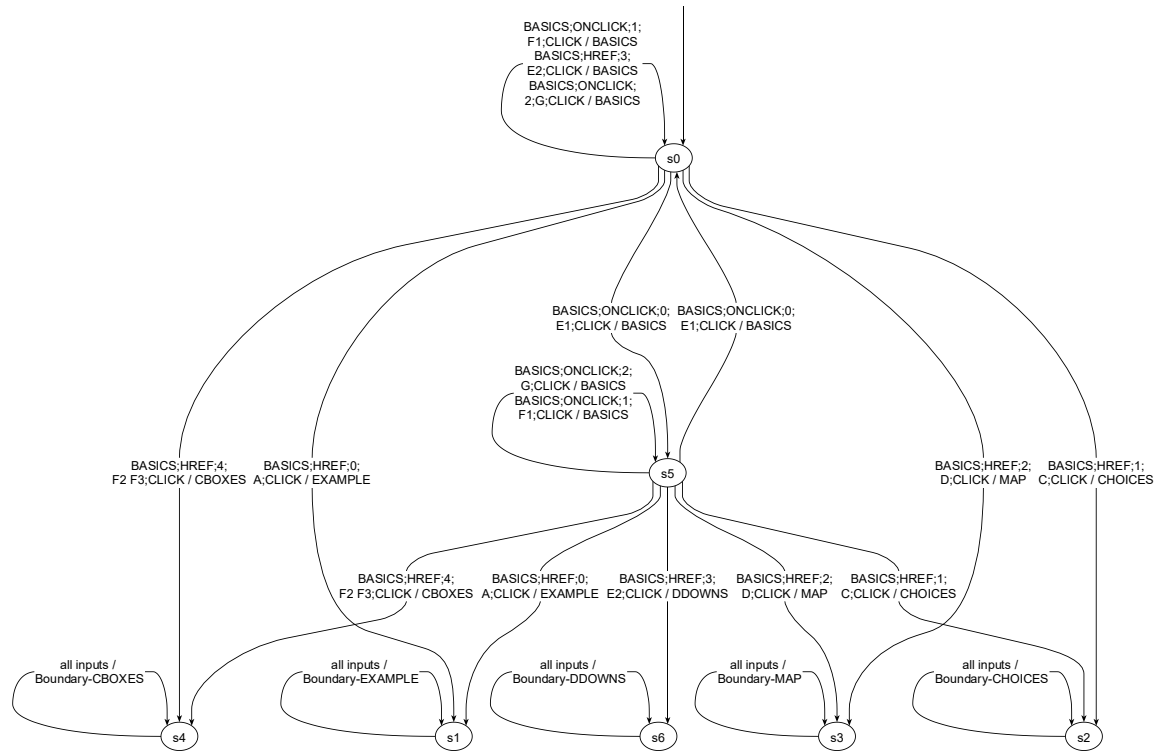
Appendix A: Learned automaton for TW_INDEX (self-loop and sink state IEH)



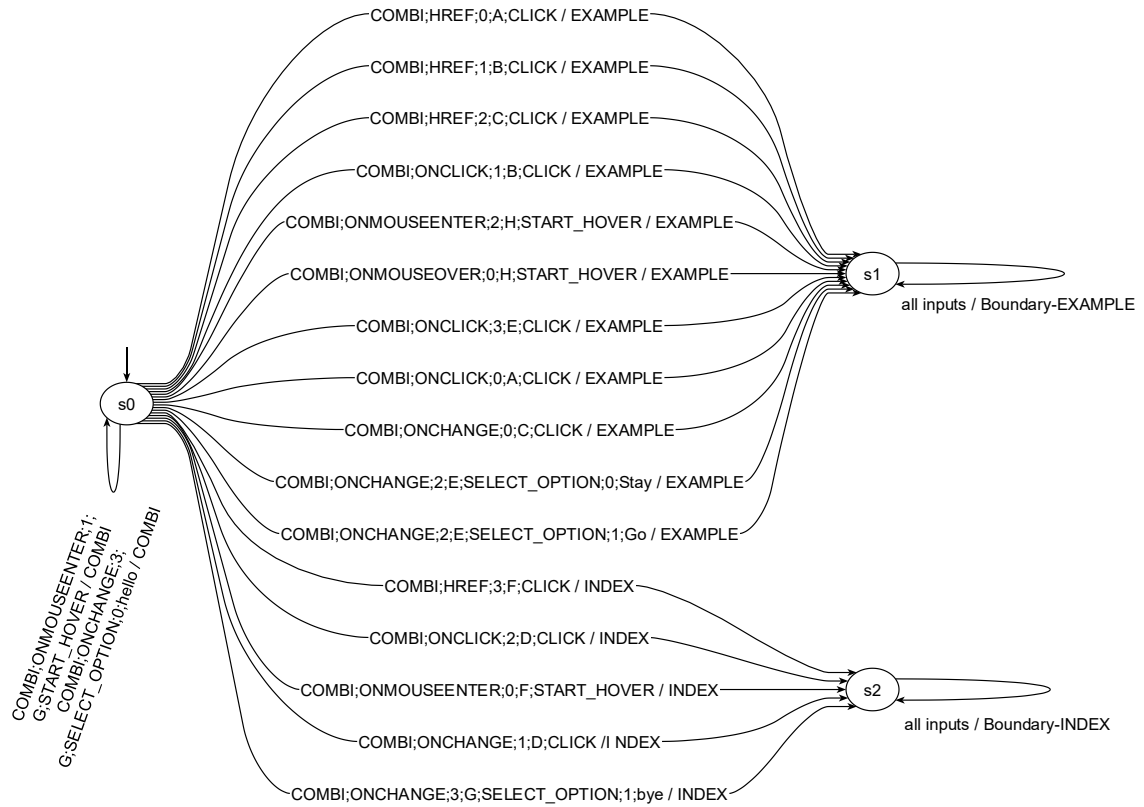
Appendix B: Learned automaton for TW_MAP_AREAS (self-loop and sink state IEH)



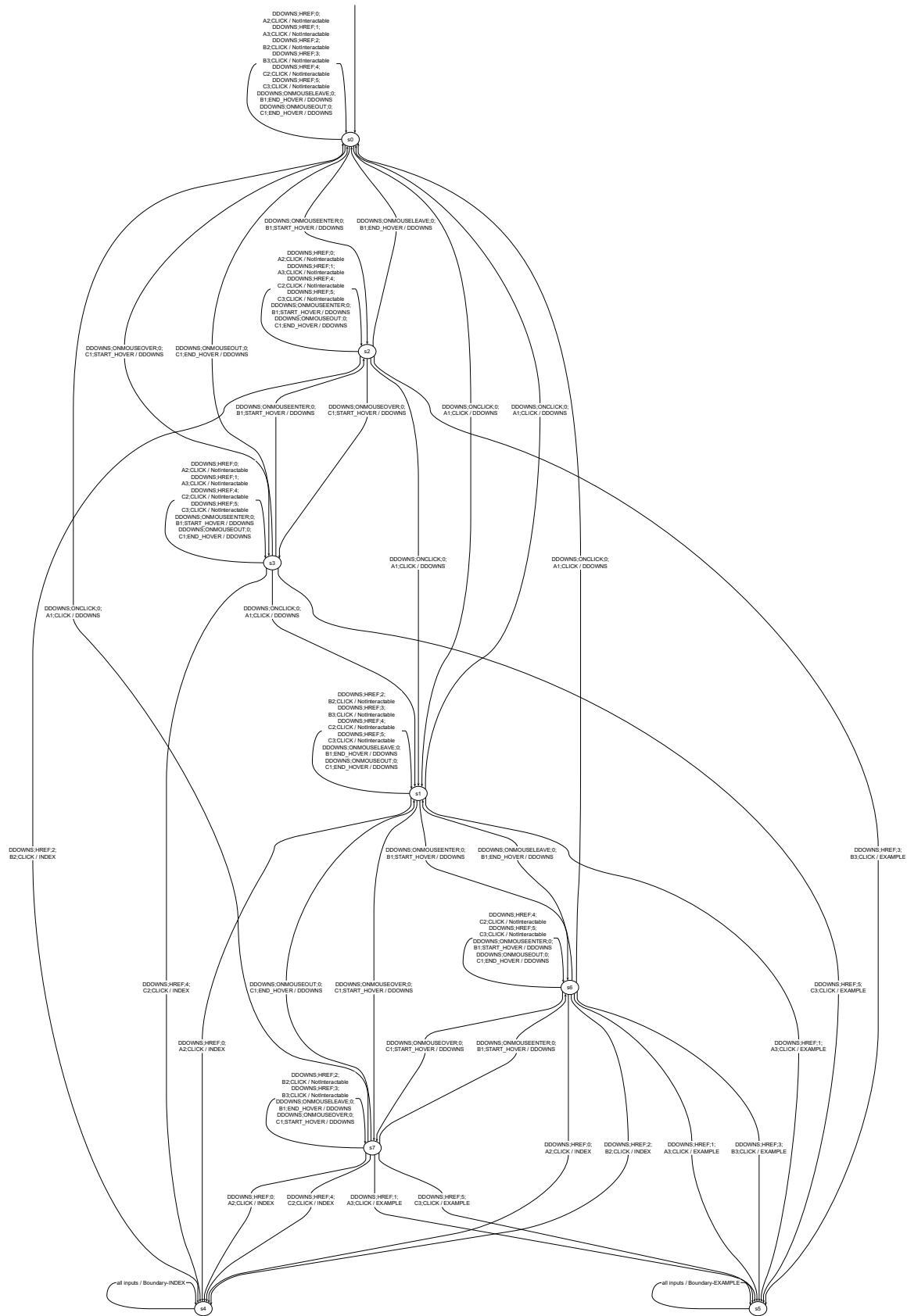
Appendix C: Learned automaton for $TW_TABS_WINDOWS$ (self-loop and sink state IEH)



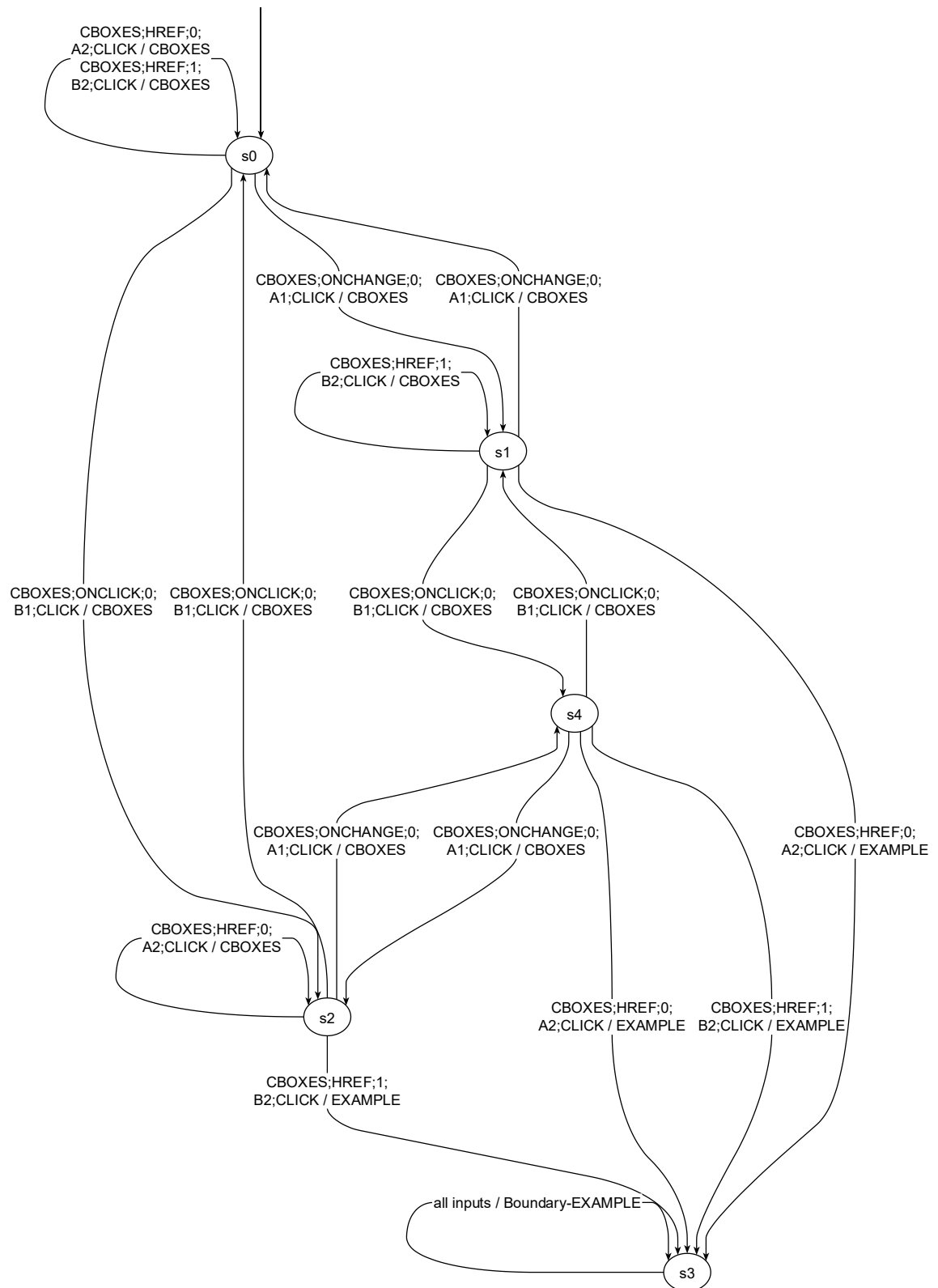
Appendix D: Learned automaton for *TW_BASICs* (self-loop and sink state IEH)



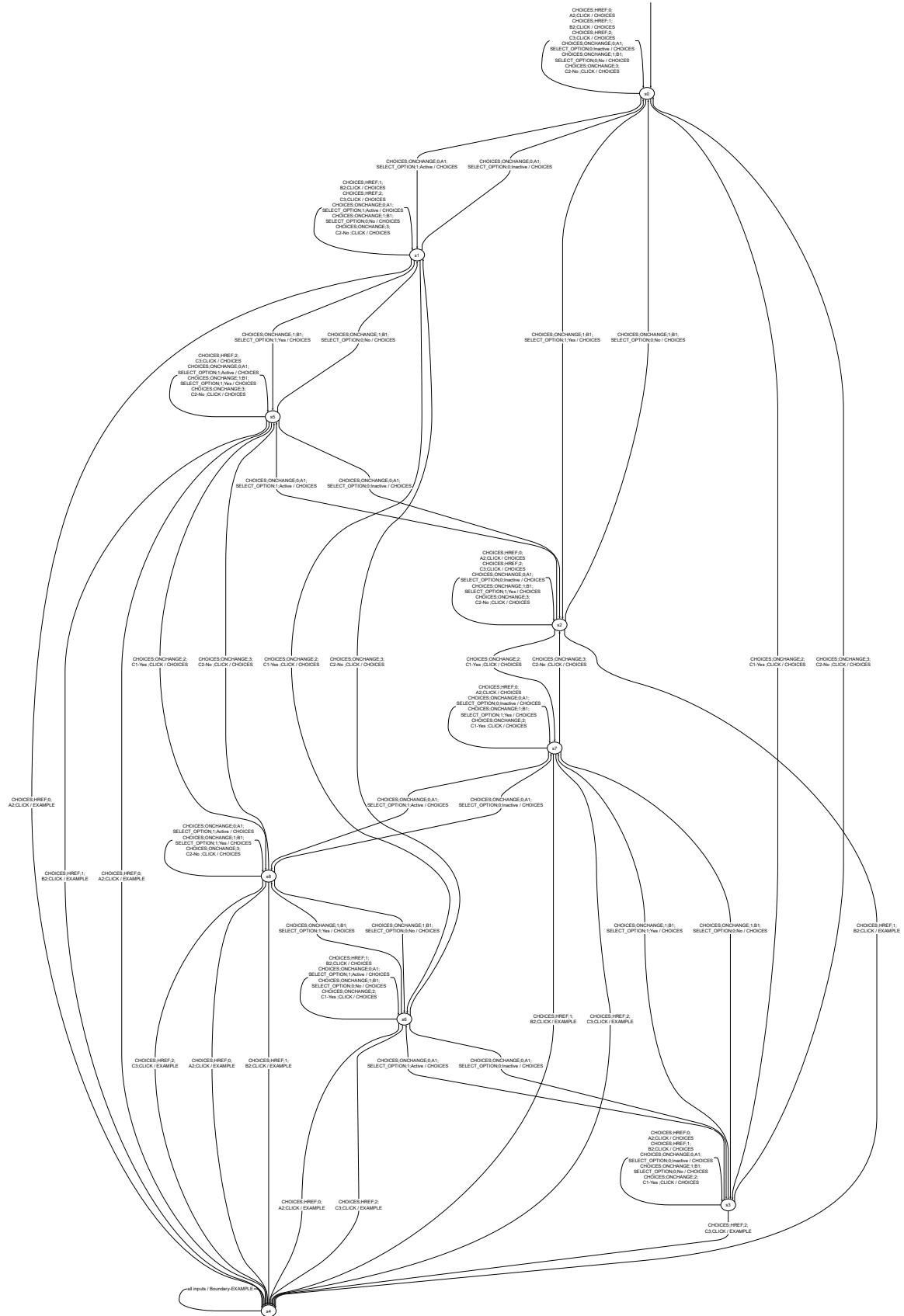
Appendix E: Learned automaton for *TW_ATTRIBUTE_COMBINATIONS* (self-loop and sink state IEH)



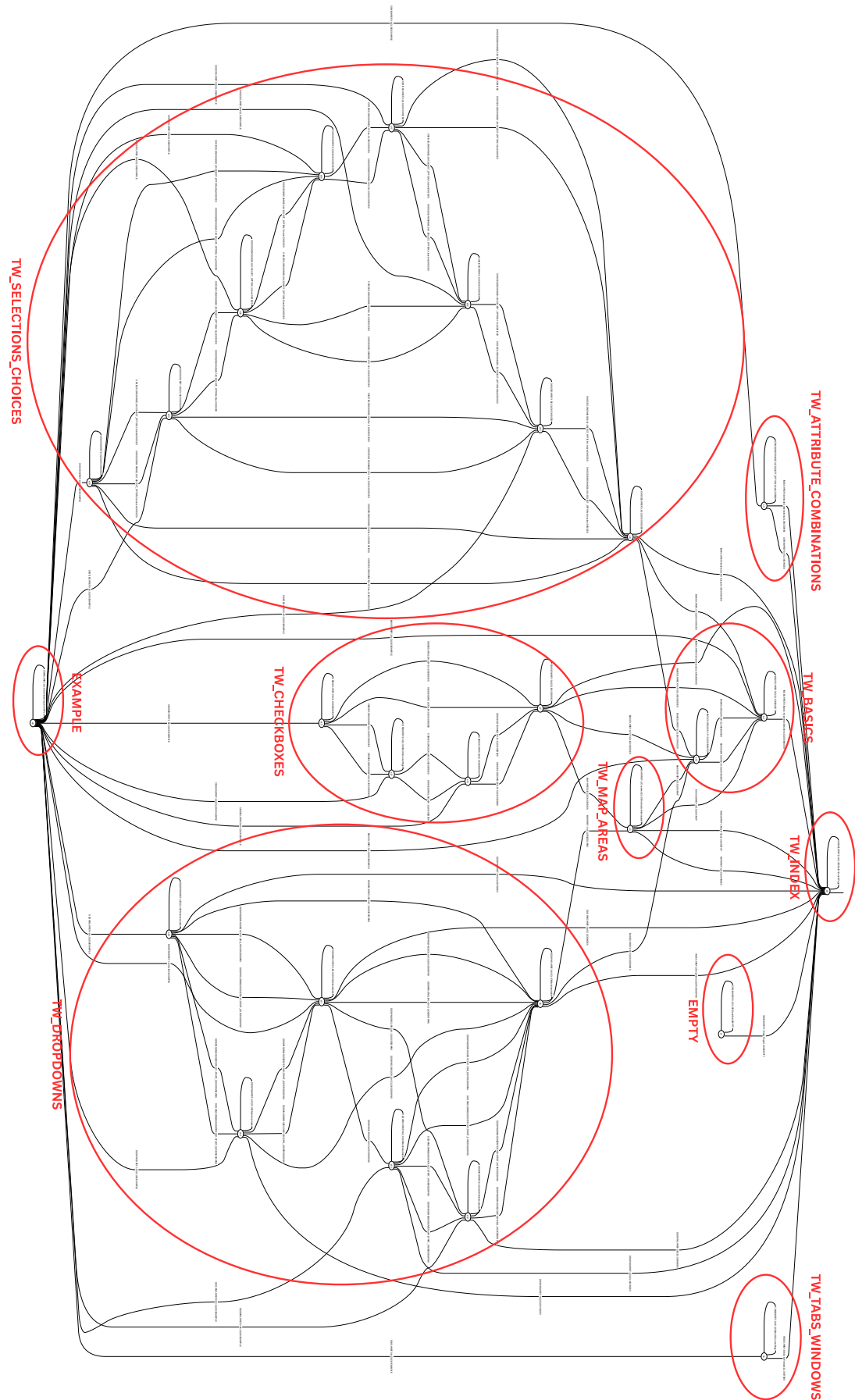
Appendix F: Learned automaton for *TW_DROPDOWNS* (self-loop and sink state IEH)



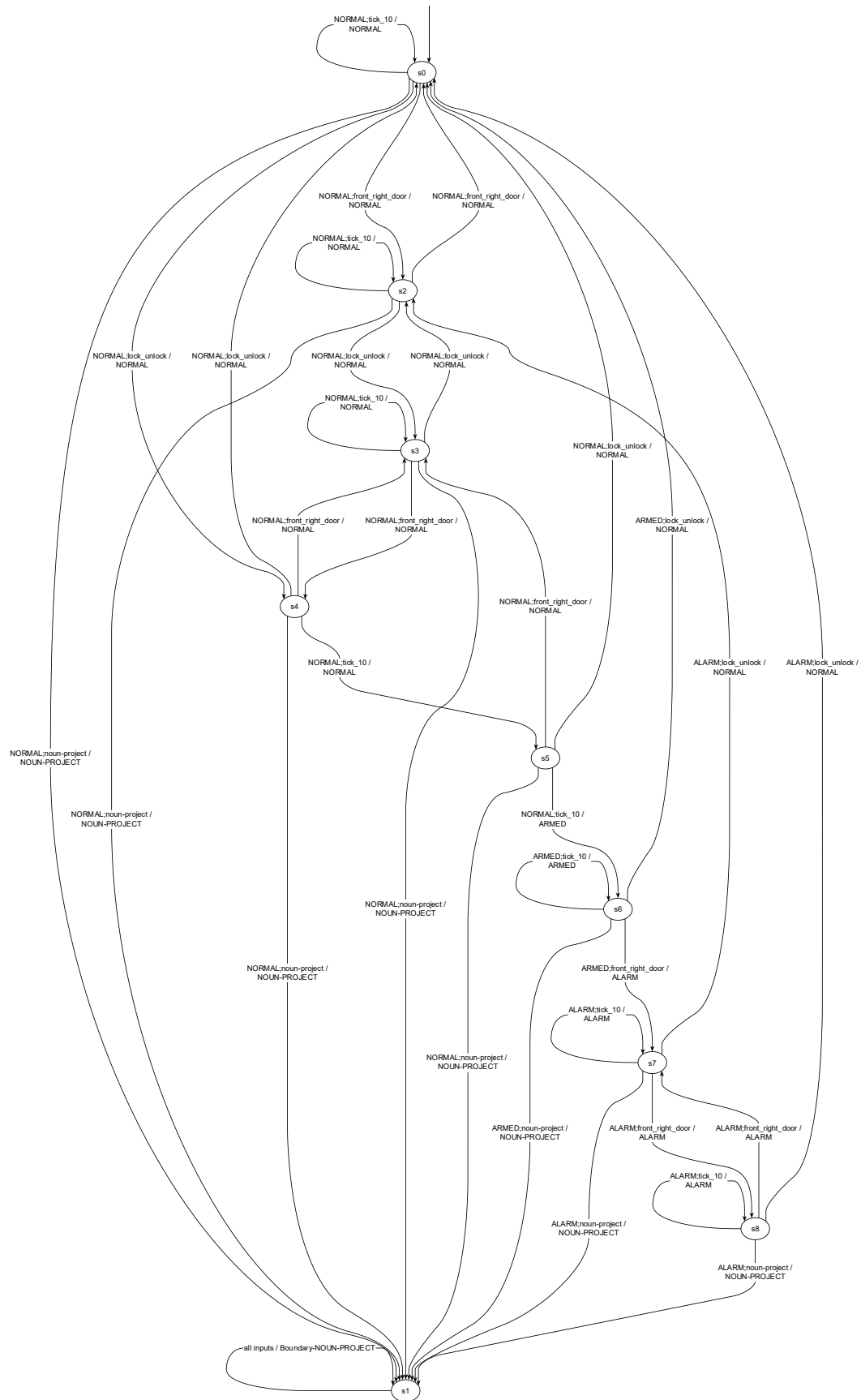
Appendix G: Learned automaton for *TW_CHECKBOXES* (self-loop and sink state IEH)



Appendix H: Learned automaton for *TW_SELECTIONS_CHOICES* (self-loop and sink state IEH)



Appendix I: Schematic of learned automaton for *TEST_WEBSITE* (not all transitions are shown)



Appendix J: Learned automaton for *CAR_ALARM* (IEH specific transitions have been omitted)

Bibliography

- [1] Statista GmbH, “Internet usage worldwide - statistics & facts,” Statista GmbH, 11 April 2025. [Online]. Available: <https://www.statista.com/topics/1145/internet-usage-worldwide/#topicOverview>. [Accessed 20 August 2025].
- [2] Cognitive Market Research, “Web Market Report 2025 (Global Edition),” Cognitive Market Research, June 2025. [Online]. Available: <https://www.cognitivemarketresearch.com/web-market-report>. [Accessed 20 August 2025].
- [3] I. Kertusha, G. Assres, O. Duman and A. Arcuri, “A Survey on Web Testing: On the Rise of AI and Applications in Industry,” *CoRR*, vol. abs/2503.05378, 2025.
- [4] A. Bainczyk, A. Schieweck, M. Isberner, T. Margaria, J. Neubauer and B. Steffen, “ALEX: Mixed-Mode Learning of Web Applications at Ease,” in *Leveraging Applications of Formal Methods, Verification and Validation: Discussion, Dissemination, Applications. ISoLa 2016.*, Springer, 2016.
- [5] A. Pferscher and B. K. Aichernig, “Fingerprinting bluetooth low energy devices via active automata learning,” in *Formal Methods - 24th International Symposium, FM 2021, Virtual Event, November 20-26, 2021, Proceedings*, Springer, 2021.
- [6] P. Ganty, “Learning the State Machine Behind a Modal Text Editor: The (Neo)Vim Case Study,” in *Model Checking Software - 30th International Symposium, SPIN 2024, Luxembourg City, Luxembourg, April 8-9, 2024, Proceedings*, Springer, 2024.
- [7] A. Pferscher and B. K. Aichernig, “Stateful black-box fuzzing of bluetooth devices using automata learning,” in *NASA Formal Methods - 14th International Symposium, NFM 2022, Pasadena, CA, USA, May 24-27, 2022, Proceedings*, Springer, 2022.
- [8] E. Muškardin, B. K. Aichernig, I. Pill, A. Pferscher and M. Tappler, “AALpy: an active automata learning library,” *Innovations in Systems and Software Engineering*, vol. 18, no. 3, pp. 417-426, Springer, 2022.
- [9] Software Freedom Conservancy, “Selenium,” 04 June 2024. [Online]. Available: <https://www.selenium.dev/>. [Accessed 20 August 2025].
- [10] B. K. Aichernig, E. Muškardin and A. Pferscher, “Active vs. Passive: A Comparison of Automata Learning Paradigms for Network Protocols,” in *Proceedings 4th Internat. Workshop on Formal Methods for Autonomous Systems (FMAS) and 4th Internat. Workshop on Automated and verifiable Software sYstem DEvelopment (ASYDE), 4th Internat. Workshop on Automated and verifiable Software sYstem DEvelopment*, Berlin, Germany, 2022.
- [11] H. Rajora, “Basics of Web Testing: A Complete Guide With Best Practices,” Lambdatest, [Online]. Available: <https://www.lambdatest.com/learning-hub/web-testing>. [Accessed 19 August 2025].
- [12] P. Kumar, “Website Testing Techniques,” CodeProject, 28 November 2011. [Online]. Available: <https://www.codeproject.com/Articles/291343/Website-Testing-Techniques>. [Accessed 19 August 2025].
- [13] G. A. Di Lucca and A. R. Fasolino, “Testing Web-based applications: The state of the art and future trends,” *Information and Software Technology*, vol. 48, no. 12, p. 1172–1186, Elsevier, 2006.
- [14] M. Merten, “Active automata learning for real-life applications,” PhD Thesis, TU Dortmund, 2013.
- [15] D. Angluin, “Learning Regular Sets from Queries and Counterexamples,” *Information and Computation*, vol. 75, no. 2, pp. 87-106, Academic Press, Inc., 1987.
- [16] P. Grubelnik, “RALFF - React Automata Learner for Forms,” Master Thesis, TU Graz, 2024.
- [17] E. Muškardin, “AALpy Code - StatePrefixEqOracle.py,” 2021. [Online]. Available: <https://github.com/DES-Lab/AALpy/blob/master/aalpy/oracles/StatePrefixEqOracle.py>. [Accessed 08 July 2025].

- [18] E. Muškardin, “AALpy Code - RandomWalkEqOracle.py,” 2021. [Online]. Available: <https://github.com/DES-Lab/AALpy/blob/master/aalpy/oracles/RandomWalkEqOracle.py>. [Accessed 08 July 2025].
- [19] B. von Berg and B. K. Aichernig, “Extending AALpy with Passive Learning: A Generalized State-Merging Approach,” in *Computer Aided Verification - 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part IV*, Springer, 2025.
- [20] E. Muškardin, “AALpy - Code,” 04 June 2024. [Online]. Available: <https://github.com/DES-Lab/AALpy>. [Accessed 20 August 2025].
- [21] E. Muškardin, “AALpy Wiki,” 19 June 2024. [Online]. Available: <https://github.com/DES-Lab/AALpy/wiki>. [Accessed 05 August 2025].
- [22] Refsnes Data, “Window alert(),” W3Schools, [Online]. Available: https://www.w3schools.com/jsref/met_win_alert.asp. [Accessed 09 May 2025].
- [23] Software Freedom Conservancy, “Selenium - Waiting Strategies,” Software Freedom Conservancy, 03 September 2024. [Online]. Available: <https://www.selenium.dev/documentation/webdriver/waits/>. [Accessed 13 May 2025].
- [24] Software Freedom Conservancy, “Selenium - Browser interactions,” Software Freedom Conservancy, 06 September 2024. [Online]. Available: <https://www.selenium.dev/documentation/webdriver/interactions/>. [Accessed 05 May 2025].
- [25] BrowserStack, “Locators in Selenium: A Detailed Guide,” BrowserStack, 22 September 2024. [Online]. Available: <https://www.browserstack.com/guide/locators-in-selenium>. [Accessed 05 May 2025].
- [26] L. Klambauer, “AALpy-Selenium-Testing Prototype - Code,” 23 May 2024. [Online]. Available: <https://gitlab.tugraz.at/DC0E35FFBBDDB722/aalpy-selenium-testing>. [Accessed 11 September 2025].
- [27] Mozilla Corporation, “HTML elements reference,” Mozilla, [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements>. [Accessed 05 May 2025].
- [28] Refsnes Data, “HTML DOM Events,” W3Schools, [Online]. Available: https://www.w3schools.com/JSREF/dom_obj_event.asp. [Accessed 07 May 2025].
- [29] Mozilla Corporation, “Introduction to events,” mdn web docs, [Online]. Available: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Events. [Accessed 22 May 2025].
- [30] L. Klambauer, “Test Website - Frontend,” 25 July 2025. [Online]. Available: https://lklamb.github.io/Test_Website/. [Accessed 11 September 2025].
- [31] Mozilla Corporation, “<select>: The HTML Select element,” Mozilla, [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/select>. [Accessed 07 May 2025].
- [32] Software Freedom Conservancy, “Selenium - WebDriver,” Software Freedom Conservancy, 07 November 2024. [Online]. Available: <https://www.selenium.dev/documentation/webdriver/>. [Accessed 05 May 2025].
- [33] V. Gupta, “A Complete Guide To Selenium Page Load Strategy,” Lambdatest, 23 January 2023. [Online]. Available: <https://www.lambdatest.com/blog/selenium-page-load-strategy/>. [Accessed 30 July 2025].
- [34] Software Freedom Conservancy, “Selenium - Interacting with web elements,” Selenium, [Online]. Available: <https://www.selenium.dev/documentation/webdriver/elements/interactions/#click>. [Accessed 12 May 2025].
- [35] Mozilla Corporation, “Customizable select elements,” mdn web docs, [Online]. Available: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Customizable_select. [Accessed 12 May 2025].
- [36] I. Mojeed, “Creating a custom <select> dropdown with CSS,” LogRocket, 29 August 2023. [Online]. Available: https://dev.to/logrocket/creating-a-custom-dropdown-with-css-1n9h?comments_sort=oldest. [Accessed 12 May 2025].

- [37] J. Kyrnin, "Understanding the Index.html Page on a Website," ThoughtCo., 07 June 2024. [Online]. Available: <https://www.thoughtco.com/index-html-page-3466505>. [Accessed 05 May 2025].
- [38] A. SRL, "How to Remove index.html from URL," AlexHost, 01 November 2024. [Online]. Available: <https://alexhost.com/faq/how-to-remove-index-html-from-url/>. [Accessed 05 May 2025].
- [39] B. Priya, "Differentiate Between Mealy Machine and Moore Machine in TOC," tutorialspoint, 15 June 2021. [Online]. Available: <https://www.tutorialspoint.com/differentiate-between-mealy-machine-and-moore-machine-in-toc>. [Accessed 09 May 2025].
- [40] A. Sweigart, "PyAutoGUI Documentation," PyAutoGUI, 2019. [Online]. Available: <https://pyautogui.readthedocs.io/en/latest/#>. [Accessed 20 May 2025].
- [41] Moeen Ajaz Khan, "Selenium WebDriver: Detail Guide With Use Cases," LambdaTest, 19 December 2023. [Online]. Available: <https://www.lambdatest.com/learning-hub/webdriver>. [Accessed 20 May 2025].
- [42] Software Freedom Conservancy, "Selenium - Mouse actions," Software Freedom Conservancy, 05 November 2024. [Online]. Available: https://www.selenium.dev/documentation/webdriver/actions_api/mouse/. [Accessed 20 May 2025].
- [43] Refsnes Data, "onmouseenter Event," W3Schools, [Online]. Available: https://www.w3schools.com/JSREF/event_onmouseenter.asp. [Accessed 07 May 2025].
- [44] Refsnes Data, "onmouseleave Event," W3Schools, [Online]. Available: https://www.w3schools.com/JSREF/event_onmouseleave.asp. [Accessed 07 May 2025].
- [45] L. Klambauer, "Car Alarm System Demo Website - Code," 09 August 2025. [Online]. Available: https://github.com/lklamb/CarAlarmSystem_Demo. [Accessed 11 September 2025].
- [46] L. Klambauer, "Car Alarm System Demo Website - Frontend," 09 August 2025. [Online]. Available: https://lklamb.github.io/CarAlarmSystem_Demo. [Accessed 11 September 2025].
- [47] L. Klambauer, "Test Website - Code," 25 July 2025. [Online]. Available: https://github.com/lklamb/Test_Website. [Accessed 11 September 2025].